ChartLibre

User Manual

Application version 0.1.0

A desktop application for exploring data and building scientific charts

Contents

1 Introduction 5
1.1 What it does 5
1.2 Key concepts 5
1.3 How this manual is organised 6
2 Getting started 7
2.1 Installing ChartLibre 7
2.2 Opening a project 7
2.3 Saving, and undoing 7
2.4 Demo projects 8
3 The main window 9
3.1 Navigation rail 9
3.2 The File page 10
3.2.1 Project info 10
3.3 The Developer page 10
3.4 The chart panel 11
3.4.1 The chart menu 11
3.4.2 Measuring, annotating and selecting points 12
3.5 Window size 12
4 Data 13
4.1 The Tables page 13
4.1.1 The preview 13
4.2 Importing data 14
4.3 Editing a table by hand 15
4.4 The Query Builder 15
4.4.1 Useful SQL 16
5 Creating a chart 17
5.1 The New plot window 17
5.2 How a chart reads its data 17
5.3 Figures, axes and series 18
5.4 Customizing a chart 18
5.4.1 Features shared by several chart types 19
6 Chart types 21
6.1 Pairwise data 21
6.1.1 Scatter Plot 21
6.1.2 Time Series 21
6.1.3 Bar Chart and Horizontal Bar Chart 22
6.1.4 Broken Bar and Broken Bar (Vertical) 22
6.1.5 Fill Between 23
6.1.6 Stack Plot 23
6.1.7 Stem Plot 24
6.1.8 Table 24
6.1.9 Text 25
6.1.10 Timeline 25
6.2 Statistical distributions 26
6.2.1 Histogram 26
6.2.2 Box Plot 26

6.2.3 Violin Plot 27
6.2.4 ECDF 27
6.2.5 Stairs 28
6.2.6 Pareto Chart 28
6.2.7 Pie Chart 29
6.2.8 Hexbin 29
6.2.9 Event Plot 30
6.2.10 Fishbone Diagram 30
6.3 Diagnostic plots 31
6.3.1 Q-Q Plot 31
6.3.2 P-P Plot 32
6.3.3 Pair Plot 32
6.3.4 Interaction Plot 33
6.3.5 Mosaic Plot 33
6.3.6 Forest Plot 34
6.3.7 Kaplan-Meier 34
6.4 Gridded data 35
6.4.1 Contour Plot 35
6.4.2 Heatmap 35
6.4.3 Quiver 36
6.4.4 Stream Plot 36
6.4.5 Wind Barbs 37
6.5 Irregularly gridded data 37
6.5.1 Contour Plot (Scattered) 37
6.5.2 Triangular Color Mesh 38
6.5.3 Triangular Mesh 38
6.6 3D and volumetric data 39
6.6.1 Surface Plot 39
6.6.2 Surface Plot (Scattered) 40
6.6.3 Scatter Plot (3D) 40
6.6.4 3D Line Plot 41
6.6.5 3D Bar Chart 42
7 Data operations 43
7.1 The Data operations page 43
7.2 How every operation window works 44
7.2.1 What an operation writes 45
7.2.2 One series or several 45
7.2.3 Curves, surfaces and fields 45
7.3 Analysis 46
7.3.1 Peaks — find and measure peaks 46
7.3.2 Roots — find where a series crosses a level 46
7.3.3 Calculus — differentiate or integrate 47
7.4 Statistics 48
7.4.1 Statistics — tests and summaries 48
7.4.2 Outliers — detect anomalies 48
7.4.3 Clustering — group similar data 49
7.4.4 Control Chart — monitor process stability 50
7.4.5 Transform — rescale or reshape a distribution 51
7.4.6 Decomposition — several series together 52
7.5 Geometry 53

7.5.1 Geometry — move a series' coordinates ... 53
7.6 Signal processing ... 53
7.6.1 Smoothing — reduce noise ... 53
7.6.2 Spectral Analysis — analyse frequencies ... 55
7.6.3 Filtering — filter, detrend or demodulate ... 56
7.6.4 Baseline Correction — subtract a background ... 57
7.7 Modeling ... 58
7.7.1 Fit — fit a model to data ... 58
7.7.2 Interpolation — fill gaps and resample ... 60
7.7.3 Function — plot a function ... 61
7.7.4 Regression — robust and machine-learning regression ... 61
7.7.5 GP Regression — a fit with its own uncertainty ... 62
8 Analysis ... 63
8.1 How every analysis window works ... 63
8.2 The analyses ... 63
8.3 Guided analysis ... 66
9 Statistics ... 68
9.1 Using the window ... 68
9.2 Models on values ... 69
9.2.1 Descriptive statistics ... 69
9.2.2 One-sample tests ... 69
9.2.3 Normality / shape tests ... 69
9.2.4 Paired-sample tests ... 70
9.2.5 Independent-sample tests ... 70
9.2.6 Group comparison (ANOVA) ... 70
9.2.7 Correlation / association ... 70
9.2.8 Distribution fit ... 71
9.3 Models on tables ... 71
9.3.1 Contingency table (chi-squared, Fisher) ... 71
9.3.2 Two-way ANOVA ... 71
9.3.3 Survival (Kaplan-Meier, log-rank) ... 72
9.4 Reliability of the results ... 72
10 Saving, reports and templates ... 73
10.1 Saving a figure ... 73
10.2 Reports ... 73
10.3 Templates ... 73
10.4 History ... 74
11 Settings, styles and the log ... 75
11.1 Settings ... 75
11.2 Chart styles ... 75
11.3 The log ... 75
11.4 Credits ... 76
12 Advanced: extending ChartLibre ... 77
12.1 Writing a custom chart renderer ... 77
12.2 Writing a custom data operation ... 78
13 Equivalents in other programs ... 81

Introduction

ChartLibre is a desktop application for Windows and macOS for importing tabular data, querying it with SQL, analysing it and turning it into scientific charts — from histograms to survival curves and 3D surfaces — without writing code. It is free and open source (MIT licence). It was written mostly by Claude, Anthropic's AI model, in conversation with its author, who designed it, set its direction and tested it. It is young software: test it on your own data before relying on it. It should also work under Linux but it has never been tested. It uses the power of SQLite to store the data, the Qt6 GUI multiplatform library, the charting capability of Matplotlib and powerful mathematical and statistical analysis tools such as SciPy, NumPy, scikit-learn and statsmodels.

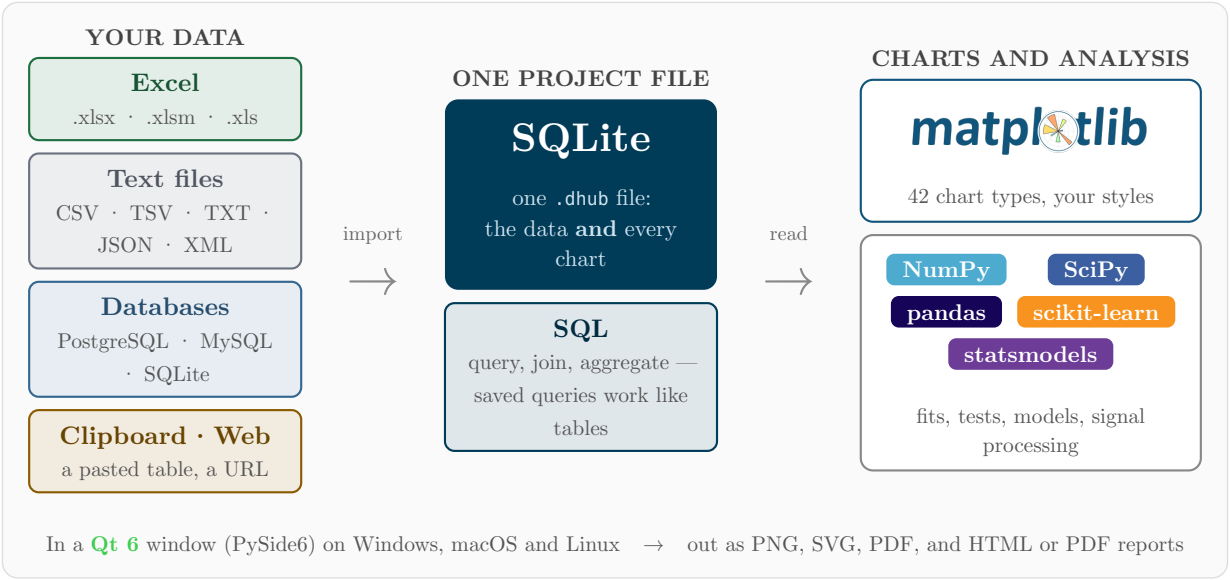


Figure 1: From your data to charts and results: what ChartLibre is built on.

What it does

- **Import** data from files (CSV, Excel, JSON, XML), the clipboard, other databases (SQLite, PostgreSQL, MySQL) and web addresses.
- **Query** it with SQL to filter, join and aggregate, without copying it.
- **Chart** it with 42 chart types, from bar charts to Kaplan-Meier curves, each fully customizable.
- **Analyse** it with 19 data operations on a chart's series — curve fitting, smoothing, filtering, spectral analysis, peak finding, statistics and hypothesis tests, control charts, clustering, outlier detection and more — and with models of a table's columns: Fit Model, Partition, Discriminant, Partial Least Squares.
- **Ask** it questions in everyday words: **Guided analysis** suggests what to use, and the quick answers reply in a sentence.
- **Report** it: save figures for publication, export a chart or the whole project as an HTML or PDF report, and reuse a chart's look as a template.

Key concepts

Project (`.dhub`)	The file you work in: a SQLite database that holds the imported data **and** every chart built on it. It is self-contained — copy or e-mail the file and everything travels with it.

Table	A set of rows and columns: imported, produced by an operation (their names start with _), or edited by hand.
Saved query	A named SQL `SELECT`, usable anywhere a table is. Its rows are computed when read, so it always reflects the current data.
Figure	Matplotlib object: a page of the chart panel, listed under **Charts** in the navigation rail. It holds one or more axes.
Axis	Matplotlib object: a chart inside a figure, with a chart type and one or more series.
Series	Matplotlib object: the data drawn on an axis: a SQL query whose columns are named after the **roles** the chart type reads — `x` and `y`, `value`, `time` and `event`, …
Chart type (renderer)	The code that turns a series' rows into a drawing. See Section 6.
Data operation	A calculation on the data of one or more series, whose result is added to the chart. See Section 7.
Analysis	A model of a table's columns — a response and what might explain it. See Section 8.

How this manual is organised

Section 2 covers installation, projects and demos. Section 3 tours the window. Section 4 explains importing, editing and querying data. Section 5 and Section 6 cover charts: how to make one, then every chart type in detail. Section 7 describes every data operation, Section 8 the analyses of a table and Guided analysis, and Section 9 the statistical tests. Section 10 covers saving and reporting, and Section 11 the preferences, the log and the credits. The last chapter, Section 12, is for those who want to extend ChartLibre with their own chart types, operations or functions.

Getting started

Installing ChartLibre

Nothing needs installing first — not even Python. Copy the `ChartLibre` folder anywhere you like (unzip it, if you downloaded it), then start it:

macOS	Double-click `ChartLibre.app`.
Windows	Double-click `ChartLibre.exe` (or `ChartLibre.bat`).
Linux	Open a terminal in the folder and run `./ChartLibre.sh`. The first launch also adds ChartLibre to the applications menu, so later it starts from there.

The first launch asks, then downloads what ChartLibre needs into the folder itself: its own copy of Python and the scientific libraries (Qt, Matplotlib, NumPy, SciPy, …) — about 400 MB to download, 1.7 GB on disk. It needs an internet connection and takes a few minutes; ChartLibre opens by itself when it is ready. Every later launch starts straight away and needs no internet. Nothing is installed anywhere else, so deleting the folder removes ChartLibre completely.

> **The first time**, macOS may say it cannot check the app: open **System Settings → Privacy & Security** and click **Open Anyway** (or right-click the app and choose **Open**). Windows may show **Windows protected your PC**: click **More info**, then **Run anyway**.
>
> **On Linux**, Qt needs a few system libraries that a desktop usually has. If one is missing, the first launch says which and how to install it — typically `sudo apt install libxcb-cursor0`.
>
> **If ChartLibre stops starting**, delete the hidden `.venv` folder inside the ChartLibre folder (on macOS, Cmd+Shift+. shows hidden files in Finder) and open ChartLibre again: it reinstalls the libraries.

Opening a project

On first launch, or whenever no project is open, ChartLibre asks what to open:

New	Creates an empty `.dhub` project at a path you choose.
Open	Opens an existing `.dhub` file.
Load demo	Opens one of the demo projects (see below).

The last project is reopened automatically next time, and the **File** page lists the recent ones.

Saving, and undoing

A project is a database, and every change is written to it as it is made: there is no unsaved work to lose. The commands on the **File** page are:

Save	Makes sure every change is in the `.dhub` file itself right now (Ctrl+S / Cmd+S). Useful before copying the file elsewhere.
Save As...	Saves the project under a new name and switches to the copy, leaving the original untouched.

Undo (in the main menu, and in the macOS Edit menu) takes back the last recorded action — an import, a table edit, a data operation, deleting a figure, hiding points, applying a template — up to the last ten, and names the action it is about to undo. Settings changed in Chart properties are not recorded: change them back by hand.

Demo projects

Load demo (on the **File** page) opens a ready-made project with data and charts already built. Twenty-six demos ship with ChartLibre. The window lists them with a picture of each one's first chart and a short summary, so you can see what you are opening.

Between them they cover every chart family and give most data operations a dataset built for them: a sparse curve for **Interpolation**, a drifting spectrum for **Baseline Correction**, a three-tone signal for **Filtering** and **Spectral Analysis**, a Gaussian bump and a saddle for the surface operations, the NIST reference data sets for **Fit**, and a **Diagnostic plots** demo with Q-Q and P-P plots, a survival study, a forest plot, a mosaic and a two-factor experiment — whose notes suggest which **Statistics** model to try on each.

A demo opens as a copy in your home folder, under the demo's own name, so you can change it freely. Loading the same demo again replaces that copy with a fresh one.

> **When to use it.** Exploring ChartLibre before you have data of your own; copying a chart's settings or a series' query into a real project.

The main window

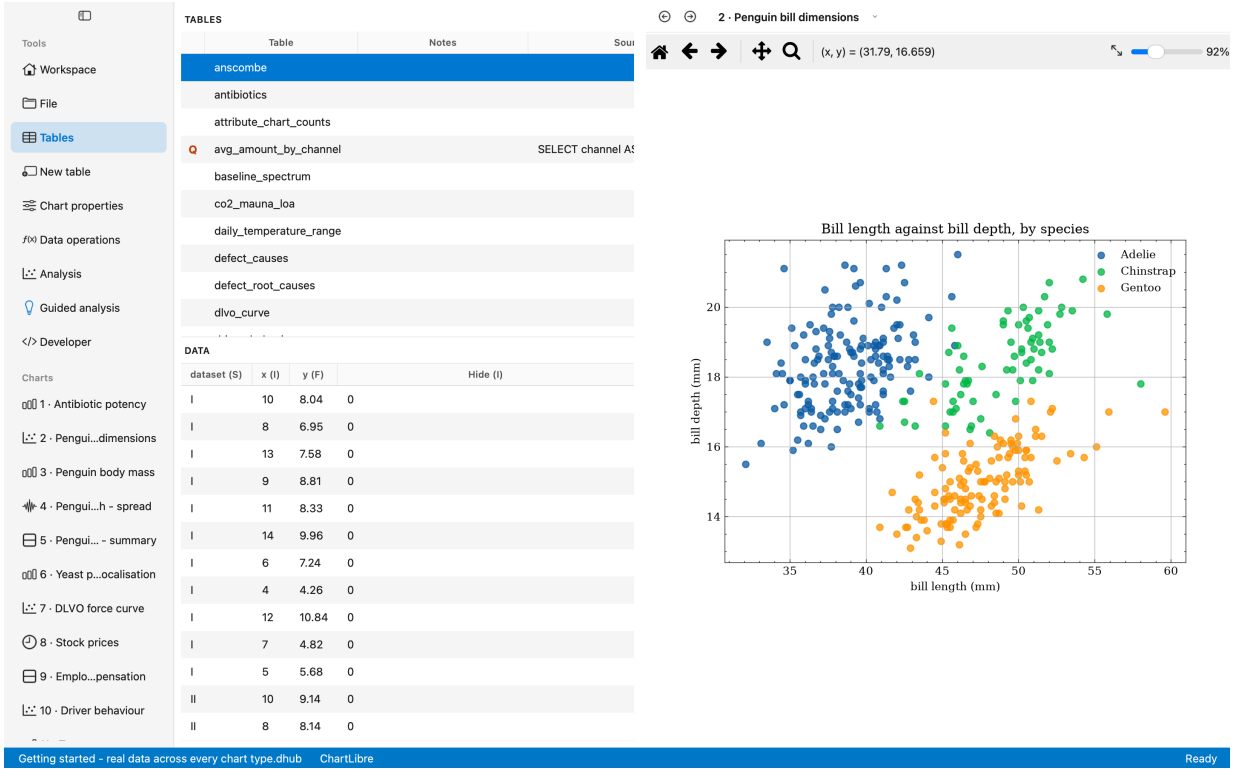


Figure 2: The main window: the navigation rail on the left, the page it selects beside it, and the chart panel on the right.

> The figures in this manual are taken in the **macOS native** style. On Windows the same screens wear the Fluent style: the panels, the pages and every control are the same and in the same place, but the navigation rail is a column of square tiles and the window draws its own title bar. On Linux the window follows the desktop's Qt style.

The window has three parts: the **navigation rail** at the far left, the **page** it selects, and the **chart panel**, which takes the rest of the window. The page and the chart panel are separated by a splitter you can drag.

Navigation rail

Workspace	Collapses the page to give the chart panel the whole window; click again to bring the last page back.
File	Projects, demos, import, save and export (see below).
Tables	The tables and saved queries of the project, with a preview of the selected one (see Section 4).
New table	A blank table, a designed experiment or a table built from others (see Section 4).
Chart properties	The settings of the selected figure, axis and series (see Section 5.4). A click on the chart selects what it landed on: outside the axes the figure, on an axis that axis, on a line, points or bar that series.

Data operations	New plot, the Query Builder and the operations on a chart's series (see Section 7).
Analysis	Models of a table's columns: Fit Model, Partition, Discriminant, Partial Least Squares (see Section 8).
Guided analysis	Search, quick answers and suggestions, starting from what you want to find out (see Section 8.3).
Developer	The project's history and database information, and the tools for extending ChartLibre (see below).

On Windows and Linux the rail ends with **Settings** and a **Menu** button (Undo, Help, Credits, …); on macOS these live in the menu bar.

The File page

Workspace	**New** and **Open** a project, and **Project info...** (see below).
Demo	**Load demo** (see Section 2.4).
Import	**Import** data into the project (see Section 4.2).
Save	**Save** and **Save As...** (see Section 2).
Export	**Project report** — the whole project as one HTML or PDF document (see Section 10).
Open recent	The projects opened recently. Click one to see its preview, author, dates, size and the start of its notes — read from the file without opening it — then **Open** it (or double-click). **Clear list** forgets them.

Project info

Project info... holds what a project says about itself, kept inside the `.dhub` file so that it travels with it:

Author	Who made it. Remembered on the computer, and filled in for every new project.
Created	When the project was created (for a project older than this record, when its file was).
Notes	What the project is about, where the data came from, what is left to do.
References	The bibliographic sources of the data and methods: a citation, and optionally a DOI and a web link each. Reports list them with working links.
Preview	A picture of the project's first figure, `<project>.preview.png` beside the project file, drawn again each time the project is saved. The Load demo window shows the demos' previews the same way.

The window opens by itself when a project is created, and when a project is saved without an author or notes — once per session: close it to fill it in another time.

The Developer page

Despite its name, the first part of this page is for everyone:

History	**Project history...** lists every operation applied in the project, oldest first: what was run, on which series, with which settings (see Section 10.4).

Database information	The project file's path and size, everything SQLite reports about it (tables and rows, page usage and how much **Optimize DB** would reclaim, journal mode, SQLite version, file dates), and every table with its row count and import link. From here a table can be exported to CSV or Excel or refreshed from its link.
Optimize DB	Checks the project file for problems, reports them, and compacts it — it can shrink a project noticeably after large tables were deleted.

The rest of the page holds the tools for extending ChartLibre — **Edit Localization**, **Data Operation Builder**, **Function Creator**, **Renderer Helper** (see Section 12) — and the **Log viewer** (see Section 11.3).

The chart panel

The chart panel shows one **figure** at a time. The strip above the chart names the current figure: click it for the list of every figure of the project, or use the ‹ and › arrows to step to the previous or next one. Creating a chart, or running an operation that adds a figure, shows the new figure. Above the chart, a toolbar offers Matplotlib's navigation tools and shows the pointer's coordinates:

Home	Back to the initial view.
Back / Forward	Step through the zoom and pan history.
Pan	Drag to move the view; right-drag to zoom each axis separately.
Zoom	Drag a rectangle to zoom in.
Zoom to fit	Bring the whole chart back within the panel.

Resting the pointer on a point shows a label with its series name and its x and y values. Very large series are **downsampled** for drawing (see the Figure options); the data itself is never changed.

The chart menu

Right-clicking the chart opens its menu:

Reload	Redraw the chart from the project, discarding zoom and pan.
Copy	Copy the chart as a picture to the clipboard.
Save figure	Save the chart as PNG, JPEG, SVG or PDF, at the figure's own DPI (see Section 10).
Export report...	This chart alone, with its notes, as an HTML or PDF report (see Section 10).
Export view as CSV...	The rows actually on screen now — after zoom and downsampling — as a CSV file, each row tagged with its axis and series.
Templates	**Save this look as a template...**, **Apply a template...**, **Delete a template...** (see Section 10.3).
Crosshair	A guide line under the pointer, shared by every axis of the figure.
Link zoom/pan across axes	Zooming or panning the x range of one axis applies it to every axis of the figure — for stacked time series.
Select points	Turns the pointer into a rectangle selection tool (see below).
Live updates	Redraws the chart on a timer, for a table that an external process keeps filling.
Delete	Deletes the figure, after confirmation.

Right-clicking **inside an axis** adds actions for the exact point clicked: **Add vertical line at x = …**, **Add horizontal line at y = …**, **Add annotation here…**, and **Measure from here** / **Measure to here**.

Measuring, annotating and selecting points

Measure from here and then **Measure to here** record a two-click measurement — distance, Δx, Δy and slope — kept on the chart like a reference line. Annotations, reference lines and measurements each have a tab in **Overlay properties** (in Chart properties) where they can be edited by exact values or deleted. An annotation can also be dragged on the chart.

With **Select points** on, drag a rectangle over the chart: every point it covers, from every series of that axis, is selected. The menu then offers:

Hide N selected point(s)	Removes the points from the chart without touching the table: the series' query is narrowed to exclude them. Undo brings them back.
New series from selection	Adds a series holding only the selected points — to isolate a peak or a cluster for its own analysis.

Window size

Every window of ChartLibre opens fully visible on the screen it appears on, shrinking if needed to fit a small laptop display. Each dialog remembers its size and position between sessions.

Data

The Tables page

The **Tables** page lists the project's tables and saved queries (marked with a **Q**), and shows a preview of the selected one below the list.

The list has four columns:

(no title)	A narrow first column with a badge: a chain for a table linked to its source (it can be refreshed with **Update link**), **Q** for a saved query, nothing otherwise.
Table	The table's name.
Notes	Free text: double-click to write what the table is, where it came from, what was done to it. Notes are kept in the project and appear in reports.
Source	Where its data comes from: a file name for a file or web import, "connection → table" for a database import, the SQL itself for a saved query (the full text in the tooltip), nothing for a table with no link.

Tables written by data operations start with **_**. **Show generated tables** hides or shows them, so the list can show only your own data.

Right-clicking a table offers:

Plot	Opens New plot on this table.
Query Builder... / Edit...	Opens the Query Builder; on a saved query, opens that query for editing.
Rename..., **Duplicate table**, **Delete...**	Manage the table. Several tables can be selected and deleted at once.
Export → CSV... / Export → XLSX...	Save the table as a CSV text file or an Excel workbook.
Update link	Re-read the table from its source — the same file, database table, query or URL — replacing its contents.
History...	The operations applied to or from this table (see Section 10.4).

The preview

The preview shows the selected table's rows. Each column header carries the column's type in brackets: (S) text, (I) whole numbers, (F) decimal numbers. Right-click a cell for:

Copy	Copy the selected cells.
Statistics of a column	Count, empty cells, distinct values, minimum, maximum, mean and median of the column.
Histogram and statistics	With exactly one column selected: a new figure with that column's histogram, shown at once, and its statistics — mean with its 95% confidence interval, standard devia-

	tion, quartiles, skewness, kurtosis — in the figure's notes. Hidden rows are left out.
Hide rows / Select rows	Mark the rows equal to the selected cell, different from it, higher, lower, or where the column is empty — in the `Hide` column, which every chart skips, or in the `Selected` column, which only marks them. The rows stay in the table.
Export rows...	Save the selected rows as CSV or Excel.
New chart from selected columns	Opens New plot with the selected columns as x, y and z.
Edit table...	Opens the table editor.

Importing data

Import (on the **File** page) opens the import window with four sources:

Open	A local file: `.csv`, `.tsv`, `.txt`, `.xlsx`, `.xlsm`, `.xls`, `.json`, `.xml`. For a workbook with several sheets, you choose the sheet.
Paste	Whatever table is on the clipboard — for example cells copied from a spreadsheet.
Database	A table, or the result of a query, from another database: SQLite, another ChartLibre project, PostgreSQL or MySQL.
Web	Whatever an http(s) address returns — a CSV download, a JSON API, a spreadsheet.

Whatever the source, the window shows a preview before anything is written, and the left panel sets how it is read:

Source	Where the data comes from.
Read options	The destination table name, the header row, rows to skip, the delimiter and the text encoding. These describe how to read **text**, so they are disabled for a database source, which has its own column types.
Columns	The type of each column (number, text, date, …) — change it when a column was guessed wrong.

Database opens its own window: pick an engine, give a file path or a host, port, user and password (**Connect** then lists the server's databases, and picking one lists its tables), choose a table — or tick **Use a query** and write a `SELECT` for a join, a filter or an aggregate — and confirm. The last connection is remembered, except its password.

Web has a menu of ready-made public datasets grouped by subject. The menu is the file `web_sources.json` in the ChartLibre folder, beside `config.json`: a list of entries with a `name`, a `url`, and optionally a `category` and a `description`. Open it in any text editor to add, change or remove datasets — the menu reads it again each time it opens. A mistake in the file (a missing comma, say) leaves its entries out and is named in the log; it never stops the import window. Picking one fills in the address without downloading, so it can be checked first; **Fetch** downloads it. **Add source** saves an address of your own to the menu, and **Delete source** removes one you added.

> Importing is a one-time read: the table does not change when the original file, database or web page changes. To refresh it, right-click the table and choose **Update link**. Every source except a paste can be refreshed this way.

A PostgreSQL or MySQL password is never saved — not in the project, not anywhere on disk — because a `.dhub` file is easily copied or shared. **Update link** asks for it again. **Web** only fetches `http://` and `https://` addresses, never a local file.

Editing a table by hand

The preview is read-only. To change the data, right-click it and choose **Edit table...**:

Cells	Double-click and type. An emptied cell becomes NULL (not measured) rather than an empty text, which would turn a numeric column into text.
Rows	**Add row** at the end, **Insert row above** the selected one, **Delete rows** with a selected cell.
Columns	Type a name and pick a type, then **Add column** (at the end) or **Insert before selected**; **Rename selected**; **Delete selected**.
Managed columns	`Hide` marks rows every chart skips; `Selected` marks rows for your own queries and operations (`WHERE "Selected" = 1`, or **Group by** `Selected` in New plot), with no effect on drawing; `ClusterId` is written by Clustering. Mark rows by a comparison on the selected column, or create, reset or invert the columns, from the editor's last button.

Every change is written as it is made, yet **Cancel** still puts the table back: the editor takes a snapshot before the first change. **OK** keeps the changes (and the snapshot stays in the undo history), **Cancel** restores the snapshot after asking.

Inserting a row **above** another renumbers the rows below, and inserting a column **before** another rebuilds the table. Neither is offered when the table's row id is one of its own columns (an `INTEGER PRIMARY KEY`); add at the end instead.

The Query Builder

The **Query Builder** (in the **Data** section of Data operations, or from the table list's menu) writes, tests and saves SQL queries. A saved query appears in the table list and can feed any chart, exactly like a table — but its rows are computed each time it is read, so it never goes out of date and takes no space.

The window has:

- on the left, the **saved queries** (add, delete, pick one to edit) and the project's **tables** with their **fields**;
- in the middle, the **SQL** editor and the query's **name**;
- **snippet** buttons that insert ready-made SQL for the selected table and field:

Select	`SELECT` the chosen fields from the table.
Join	Opens **Build a JOIN**: the main table and field, the table and field to join on, and the join type.
Union	Opens **Build a UNION**: two tables, stacked, using the fields they have in common.
Summary	Count, average, minimum and maximum, grouped by a field.
Order	`ORDER BY` the field, with an explicit direction.
Filter	`WHERE` with the comparison written out.

Placeholders in a snippet are written in `<angle brackets>`, which is not valid SQL, so a snippet left unfinished fails instead of running on the wrong column. **Run** shows the first 500 rows of the result below the editor. **Save query** saves it under its name; **OK** saves and closes the window — and stays open if the query could not be saved, showing why.

> By default ChartLibre refuses any SQL that would change the database — `UPDATE`, `DELETE`, `DROP` and the like — when a query is saved or typed. The data is changed only through the import window, the table editor and the operations, which all keep the undo history. The guard can be switched off in Settings (see Section 11).

Useful SQL

```
-- Filter and convert units
SELECT time AS x, temperature - 273.15 AS y FROM log WHERE sensor = 'A'

-- Monthly means from a dated table
SELECT strftime('%Y-%m', date) AS x, AVG(value) AS y FROM readings GROUP BY x

-- Recode a number into categories for a Box Plot
SELECT value, CASE WHEN dose < 10 THEN 'low' ELSE 'high' END AS "group" FROM trial
```

Names that are SQL keywords, such as `group`, must be quoted with double quotes.

Creating a chart

A chart in ChartLibre is never a picture of a spreadsheet range: it is a **query**. Every series on every chart is a SQL `SELECT` whose result columns are named after the **roles** the chart type reads — `x` and `y` for a scatter plot, `value` for a histogram, `time` and `event` for a survival curve. The chart is redrawn from that query every time it is shown, so it always reflects the data as it is now.

The New plot window

Plot, the first entry of the **Data operations** list (see Section 7), opens the chart-creation window. Building a chart takes three decisions:

1. **The chart type.** The picker lists all the chart types grouped by family, with a search box that matches a name or a whole family ("stat" brings up the statistical charts at once). Selecting one shows its description, the roles it needs and a link to the corresponding Matplotlib documentation.
2. **The data.** Pick a data source — a table or a saved query — then pick which of its columns plays each role. The window writes the SQL for you (`SELECT pressure AS x, flow AS y FROM operating_points`) and shows it; you can edit it directly for a filter, a join or a computed column. Only reading queries (`SELECT`, `WITH`) are accepted.
3. **Group by** (optional). Pick a column the chart does not already use — a species, a batch, a site — and the series becomes one series per value of that column, each with its own colour and legend entry. Each one is the same query with `WHERE "column" = value` added, and rows marked hidden stay out. Up to 50 groups.
4. **Where it goes. Create new figure** adds a new figure to the project; **Use current figure** puts the chart in the figure on screen, either as a new axis (**Add new axis**) or as one more series on an axis it already has (**Use existing axis**). Figure, axis and series can be named here or later.

When the window closes, the chart panel shows the new figure, so what you just made is what you see.

How a chart reads its data

A chart type never sees your raw table — it sees the rows the series' query returns, with columns named after its roles. A Scatter Plot needs `x` and `y`, so its series query is written:

```
SELECT pressure AS x, flow AS y FROM operating_points
```

Optional roles unlock extra behaviour when the query provides them: `color` and `size` on a Scatter Plot drive a colour map and marker sizing; `yerr` adds symmetric error bars and `yerr_low`/`yerr_high` asymmetric ones; a `group` column on a Box Plot splits one series into several boxes. A role the query does not return is simply not used.

Because the source is SQL, everything SQLite can compute is available without touching the data: `WHERE` to filter, `GROUP BY` to aggregate, `strftime` to bin dates, arithmetic to convert units (`SELECT t, v * 1000 AS y`), `CASE` to recode categories. The Query Builder (Section 4.4) is the place to write and test longer queries.

> Rows a chart cannot use are reported, not hidden. A value that is not a number where a number is needed, a missing grid cell for a chart that needs a complete grid, a negative value on a logarithmic axis: each is logged with the series it came from, and the chart draws what it can. Open the log (see Section 11.3) when a chart looks emptier than expected.

Figures, axes and series

A **figure** is one page of the chart panel, listed under **Charts** in the rail. It holds one or more **axes** — individual charts — laid out in a grid, and each axis holds one or more **series**. Every axis has its own chart type, so a figure can put a histogram, its Q-Q plot and a box plot side by side.

Most chart types accept any number of series per axis and draw them together (several lines on one Time Series, several samples in one Histogram). A few accept only one — a Pie Chart, a Heatmap, a Surface Plot, a Mosaic Plot — because two of them on the same axis would hide each other; for these the application offers a new axis instead of a second series that would be silently dropped.

Add series in Chart properties adds a series to the selected axis. To add an axis to a figure, open **Plot** with **Use current figure** and **Add new axis**.

Customizing a chart

Chart properties edits the selected figure, axis and series, each in its own tab - a click on the chart opens the tab of what it landed on:

Figure name (also the tab title), **notes** about the figure (kept with it in the project and printed under it in reports; saved as you type), its creation and last-modified dates, the grid of axes (rows and columns), the **figure layout** (tight, constrained, compressed, or manual spacing), size and DPI, frame, a Matplotlib style for this figure alone, and **downsample** (draw at most N points per series, for very large tables).

Axis chart type, title, axis labels, ranges, scales (linear, log with its base, symlog with its threshold) and direction (invert), shared x/y/z with neighbouring axes, grid, ticks, spines, the axis position and span in the grid, and the chart type's own **options** (number of bins, which distribution to compare with, whether to draw a colour bar, …).

Series name, SQL query, colour, line style, marker, legend label, visibility, sort by x, and the **plot arguments** passed to Matplotlib (line width, transparency, hatch, …).

Every setting has a description shown in its tooltip and beside it, so Matplotlib's own documentation is rarely needed. Settings are split into two families:

- **Options** are read by ChartLibre's own chart code: whether to draw a legend, how many bins, which reference line, how to colour tiles.
- **Plot arguments** are handed to the Matplotlib drawing call unchanged: colours, widths, markers, transparency, drawing order.

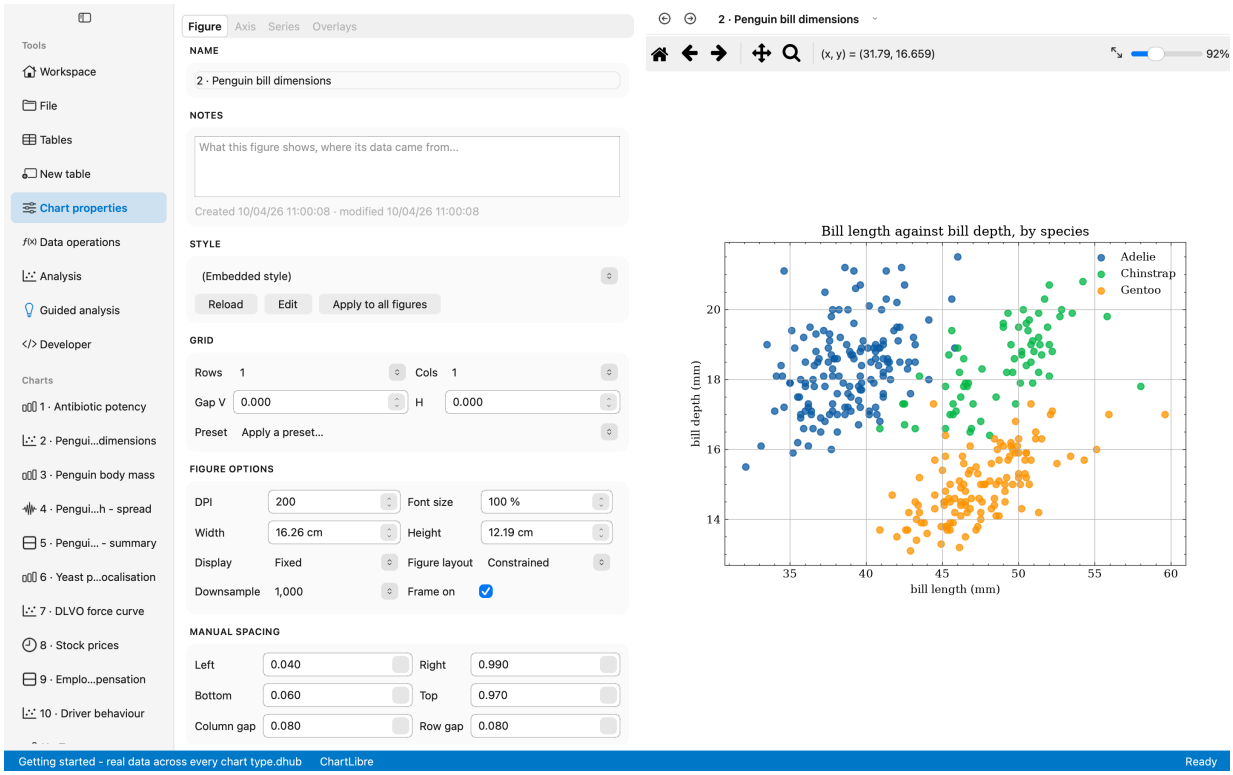


Figure 3: Chart properties: the figure, axis, series and overlay property panels, one collapsible section each, over the chart they apply to.

Overlay properties, also in Chart properties, lists the annotations, reference lines and measurements of an axis — one tab each — as editable tables, so they can be positioned by exact value rather than only by dragging.

Features shared by several chart types

Some settings appear on many charts and work the same way everywhere:

Error bars	The roles `xerr/yerr` draw symmetric error bars; `xerr_low/xerr_high` and `yerr_low/yerr_high` draw asymmetric ones. **Cap size**, **cap thickness**, **error line width**, **error colour** and **error every N** (draw a bar on every Nth point only, to keep a dense series readable) style them.
Trend line	**Trend degree** fits a least-squares polynomial through the points (1 is a straight line, 0 draws none). **Trend band** shades either the **confidence** band — the uncertainty of the fitted line itself, narrow — or the **prediction** band — where the next observation would fall, much wider. **Trend sigma** sets the band's half-width in standard deviations.
Confidence ellipse	The covariance ellipse of the point cloud, **N** standard deviations across (2 covers about 95% of a bivariate normal sample). It shows correlation as a tilt, which a cloud of points alone does not make obvious.
Distribution fit	On Histogram and ECDF: fit continuous distributions to each sample and draw their curves over the data. **best** draws the top-ranked candidate, **top3** or **top5** that many, or name one (`norm`, `lognorm`, `gamma`, `weibull_min`, …).
Colour mapping	When a `color` role holds numbers, **colormap**, **norm** (linear, log, symlog, logit), **vmin** and **vmax** control how values become colours. Integer colour columns are treated as category ids rather than a continuous scale.

Colour bar	On contour, heatmap, hexbin and mesh charts: add a colour bar with an optional label. It takes space from its own axis, so a **constrained** or **compressed** figure layout places it best.
3D view	On 3D charts: **elevation**, **azimuth** and **roll** fix the camera angle in degrees; leave them empty to use the angle you set by dragging.
Rasterized	Draw a dense series as pixels inside an otherwise vector export (SVG, PDF), so a million points do not make the file unopenable.

Chart types

ChartLibre ships **42 chart types**, grouped into six families. The first five follow the taxonomy of Matplotlib's own documentation; **Diagnostic plots** gathers the charts statisticians use to check assumptions and report models. For each one below: what it shows, when to use it, the roles it reads, the Matplotlib functions that draw it — each linked to its page in Matplotlib's documentation, where every keyword the chart's **Kwargs** accept is explained — and the settings that matter most.

Role names are written as the query must return them. Where a role is shown in capitals (`Y`, `X`, `YError`), that is its exact spelling.

Pairwise data

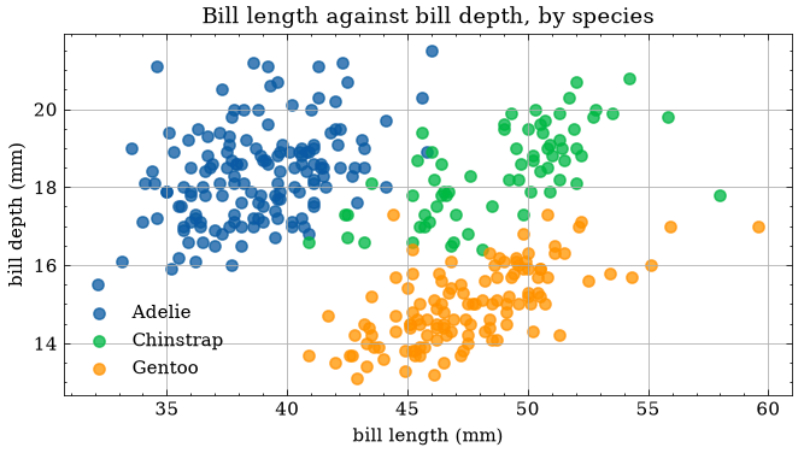
Charts of one quantity against another: the everyday x/y family.

Scatter Plot

One mark per row at (x, y). Optional columns colour each point (`color`: numbers through a colormap, integers as categories) and size it (`size`). The basic chart for two measured quantities, and the one most data operations add their results to.

Required: `x`, `y` **Optional:** `color`, `size`, `xerr`, `yerr`, `xerr_low`, `xerr_high`, `yerr_low`, `yerr_high`

Matplotlib: `Axes.scatter`, `Axes.errorbar`



> **When to use it.** Looking for a relationship between two measurements, spotting clusters and outliers, comparing data with a fitted curve.

Key settings: marker shape and size, the shared **trend line**, **confidence ellipse** and **error bar** settings described above.

Time Series

A line through (x, y) in x order, where x is a number or a timestamp. A real date/time column gets date ticks automatically. A **rolling average** line (window **N** samples) can be drawn over the raw line, and **gap threshold** breaks the line wherever two samples are further apart than a given time (`30min`, `2h`, `1D`), so an outage reads as a gap instead of a straight line across it.

Required: `x`, `y` **Optional:** `color`, error-bar roles as Scatter Plot

Matplotlib: `Axes.plot`, `Axes.errorbar`



When to use it. Measurements over time: sensor logs, prices, monitoring data.

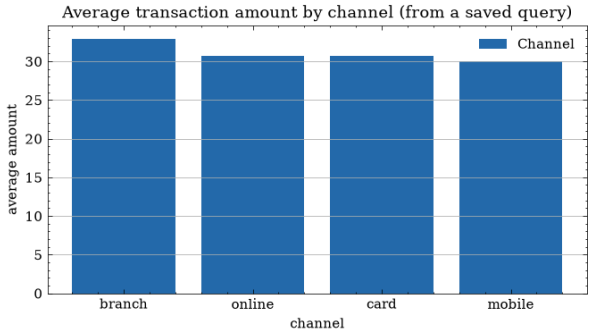
Key settings: **rolling window**, **show raw / show rolling**, **rolling line style**, **gap threshold**.

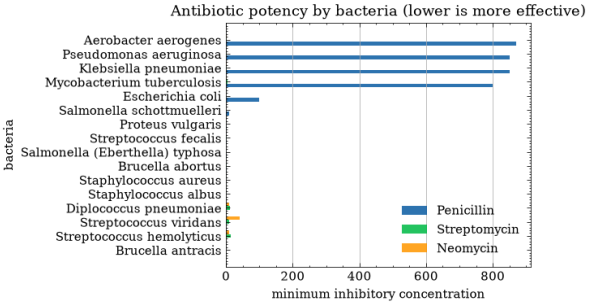
Bar Chart and Horizontal Bar Chart

One bar per row, of height `Y`, at category `X` (or at the row number when there is no `X`). Several series are drawn as groups of bars side by side within each category. `YError`/`XError` add error bars, and `Bottom`/`Left` let bars start somewhere other than zero (for floating or stacked bars). The horizontal version is the same chart turned on its side, better when category names are long.

Required: `Y` **Optional:** `X`, `color`, `YError`, `XError`, `Bottom`, `Left`

Matplotlib: `Axes.bar`, `Axes.barh`





When to use it. Comparing a quantity across categories: sales per region, counts per class, means per group.

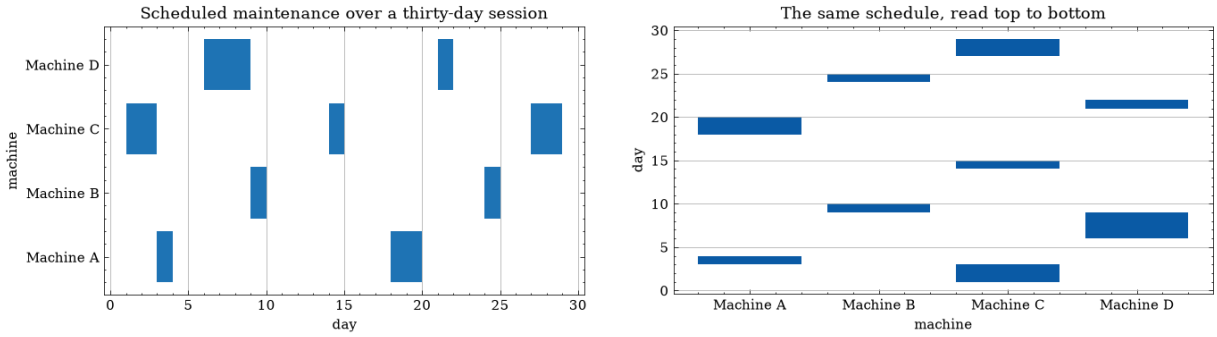
Key settings: **width** (or **height**) of each group, **align**, **hatch**, **log** value axis, and three settings for crowded category axes: **max tick labels**, **tick label rotation**, **tick label font size** (each with an automatic mode).

Broken Bar and Broken Bar (Vertical)

Gantt-style interval bars: each row is an interval from `start` lasting `duration`, drawn on the band of its `category`. Several intervals in the same category share one band. The vertical version stacks the bands in columns.

Required: `category`, `start`, `duration` **Optional:** `color`

Matplotlib: `Axes.broken_barh`, `patches.Rectangle`



When to use it. Schedules, machine states over time, shifts, any set of intervals grouped by resource.

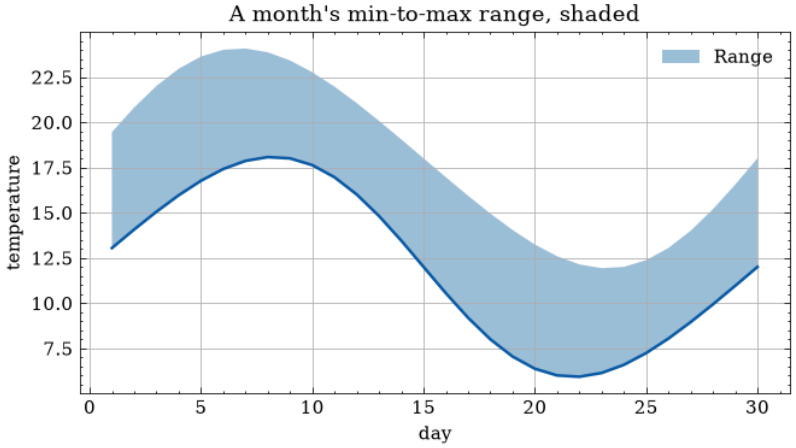
Key setting: **band height** (thickness of each band, as a fraction of the space between categories).

Fill Between

Shades the region between two curves sharing an x: `y` and `y2`, or `y` and a fixed **baseline** when there is no `y2`. The upper curve is drawn as a line over the fill (**edge line**), so a band reads as an estimate with its uncertainty.

Required: `x`, `y` **Optional:** `y2`

Matplotlib: `Axes.fill_between`



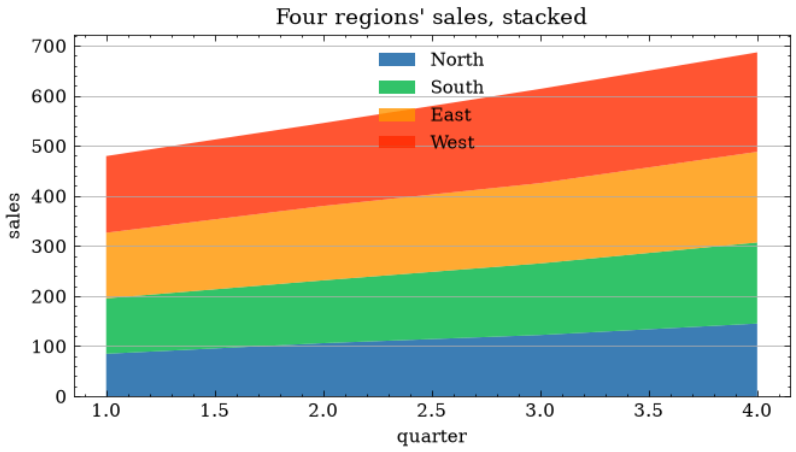
When to use it. Confidence or tolerance bands, min–max envelopes, the area between two scenarios.

Stack Plot

Every series on the axis is stacked into filled bands over a shared x, each band starting where the one below ends, so the top edge is the total.

Required: `x`, `y`

Matplotlib: `Axes.stackplot`

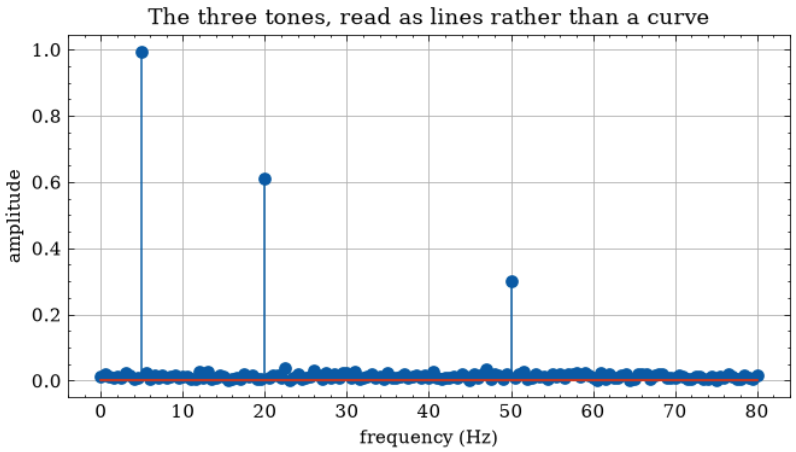


When to use it. How a total divides between its parts over time: energy mix, market shares, traffic by source.

Key setting: **baseline** — `zero` (usual), `sym` (centred on zero) or the two `wiggle` forms that make a streamgraph.

Stem Plot

A vertical stem from a baseline to each value, topped by a marker.
Required: x, y
Matplotlib: `Axes.stem`



When to use it. Values defined only at discrete x: impulse responses, discrete spectra, sampled signals, probability mass functions.

Key settings: **line**, **marker** and **base format** strings (`C0-`, `C0o`), **bottom** (the baseline value), **orientation**.

Table

Draws a table of text on the axis instead of a plot: up to eight columns, chosen with the roles `column_1` … `column_8`, in that order.
Required: none **Optional:** `column_1` … `column_8`
Matplotlib: `Axes.table`

Per-machine summary

Machine	n	mean_mm	min_mm	max_mm
Machine A	30.0	25.0008	24.9936	25.0079
Machine B	30.0	25.0018	24.9892	25.0073
Machine C	30.0	25.0058	24.9992	25.0155
Machine D	30.0	25.0014	24.9795	25.0252

When to use it. Putting a few key numbers beside a chart in the same figure, for a report or a slide.

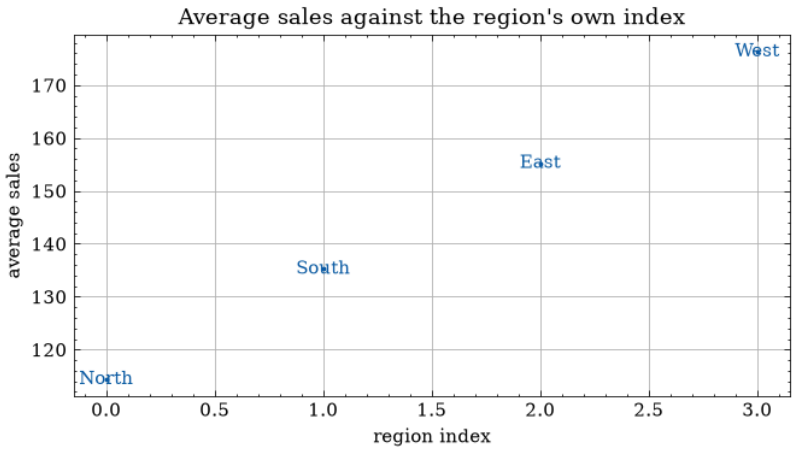
Key settings: **max rows** (default 200 — a large table is unreadable long before it is slow), **font size**, **row labels from index**, cell and header alignment, placement in the axis.

Text

Writes the text in `text` at each (x, y). When the optional **adjustText** library is installed, overlapping labels are pushed apart and joined to their point by a thin leader line.

Required: `x`, `y`, `text` **Optional:** `color`

Matplotlib: `Axes.text`



When to use it. Labelling points by name: countries on a scatter, sample ids, landmarks on a map-like plot.

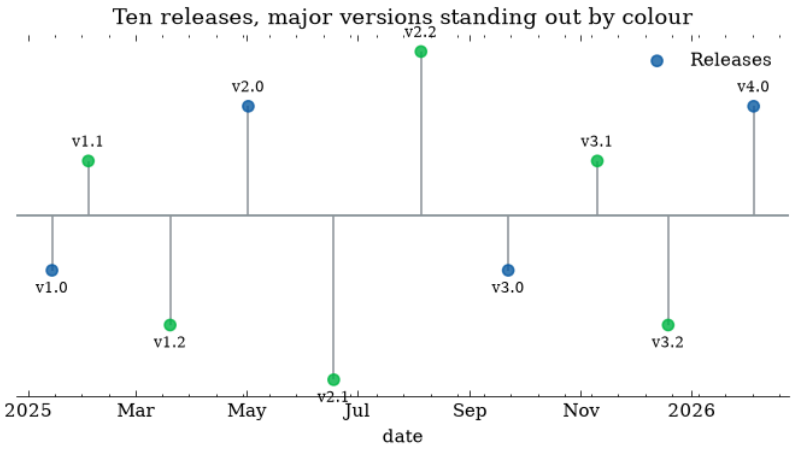
Key settings: **font size**, horizontal and vertical alignment, **show markers** at the original point, **leader colour**.

Timeline

Dated events along a horizontal baseline, each on its own vertical stem with its label at the top. Stems alternate between several heights so neighbouring labels do not collide. `y` overrides the automatic stem height, `color` and `size` work as on a scatter.

Required: `x` (the date) **Optional:** `label`, `y`, `color`, `size`

Matplotlib: `Axes.vlines`, `Axes.annotate`



> **When to use it.** Release histories, project milestones, incident lists — anything that is a list of moments.

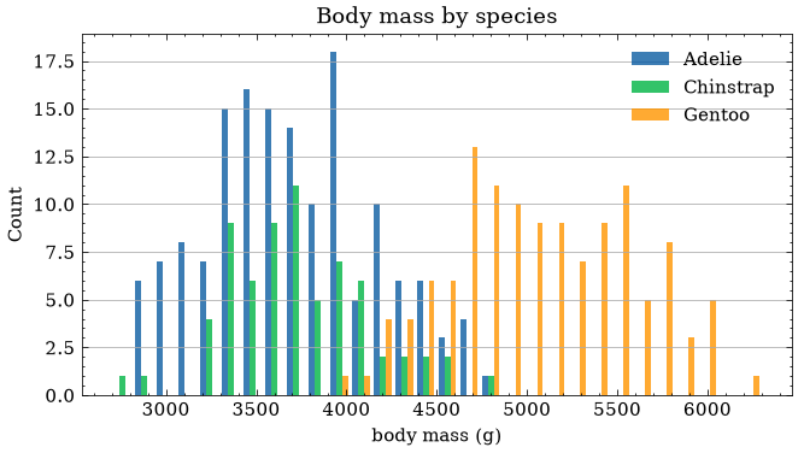
Key settings: **levels** (how many stem heights to cycle through), **annotate**, **annotation font size**, **stem colour**.

Statistical distributions

Charts of how values are distributed, rather than of one value against another.

Histogram

Counts values into equal-width bins. Several series — or one series split by a `dataset` column — share the same bins, so they can be compared directly; `weight` counts each row that many times.
Required: `value` **Optional:** `dataset`, `weight`, `color`
Matplotlib: `Axes.hist`



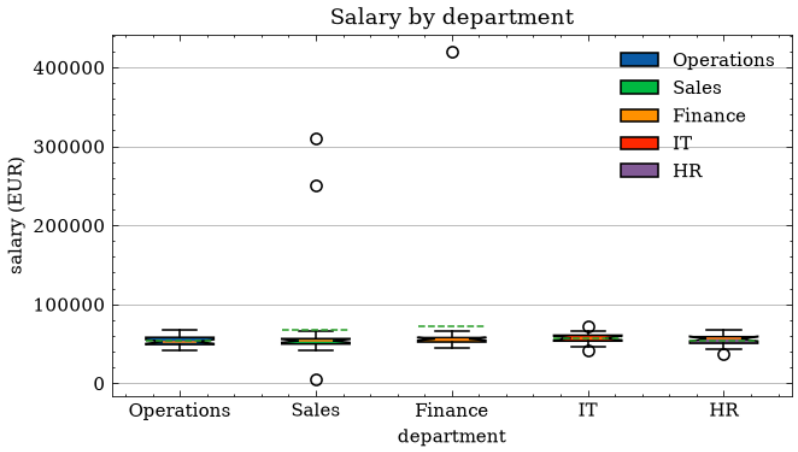
> **When to use it.** The first look at the shape of a sample: centre, spread, skew, several peaks.

Key settings: **bins**, **range min/max**, **histogram type** (side-by-side bars, stacked bars, step outline, filled step), **density** (normalise to compare samples of different sizes), **cumulative**, **log** count axis, and **distribution fit** to overlay fitted densities.

Box Plot

The five-number summary of each sample: a box from the first to the third quartile, a line at the median, whiskers reaching 1.5 × IQR (adjustable) and individual points beyond them. A `group` column turns one series into one box per group; without it, each series is a box.

Required: `value` **Optional:** `group`, `color`
Matplotlib: `Axes.bxp`

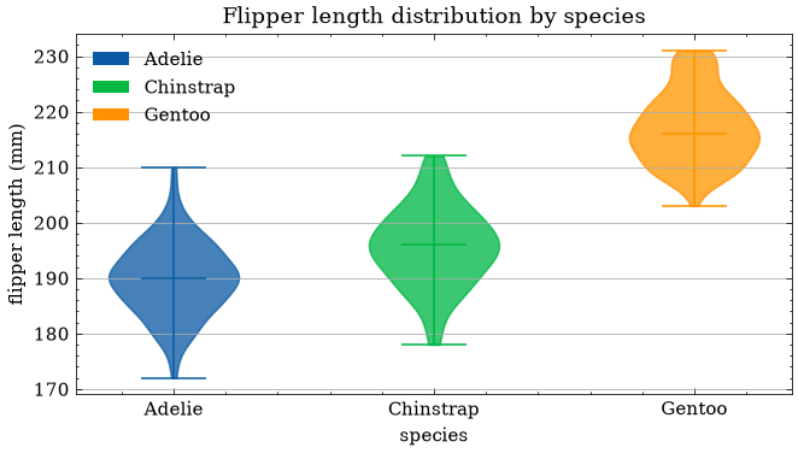


When to use it. Comparing the distribution of several groups at a glance; spotting outliers.

Key settings: **whisker reach**, **notch** (a confidence interval around the median: notches that do not overlap suggest different medians), **show means**, **show fliers**, **direction**, and **statistical annotations** (median, mean, n, or a custom format string, placed outside or on the box).

Violin Plot

Like a box plot, but each sample is drawn as its estimated density (a kernel density estimate), mirrored — so two peaks show as two bulges where a box would hide them. Same roles and grouping as Box Plot, so an axis can be switched between the two.
Required: `value` **Optional:** `group`, `color`
Matplotlib: `Axes.violinplot`

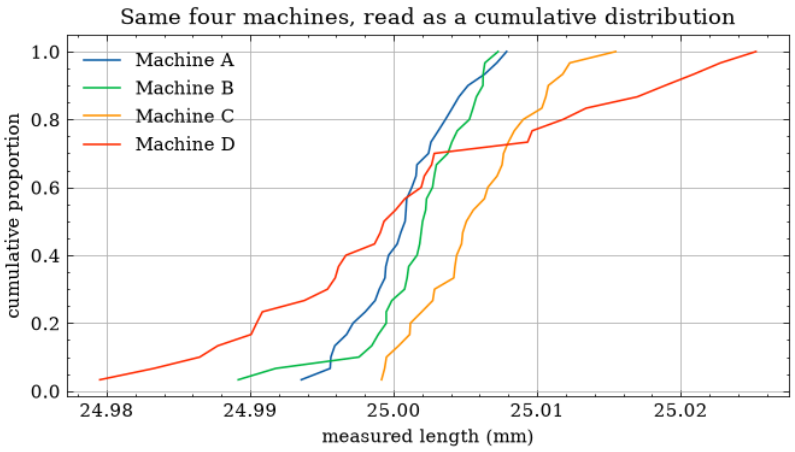


When to use it. Comparing distributions that may be skewed or have more than one peak.

Key settings: **bandwidth** (smaller follows the data more closely), **show medians/means/extrema**, **quantiles** to mark (e.g. `0.25, 0.5, 0.75`).

ECDF

The empirical cumulative distribution function: for every value, the fraction of the sample at or below it. No bins to choose, every point is drawn, and samples of different sizes compare directly.
Required: `value` **Optional:** `weight`
Matplotlib: `Axes.plot`



> **When to use it.** Comparing distributions honestly, reading percentiles directly ("90% of parts are below 4.2 mm"), judging a distribution fit.

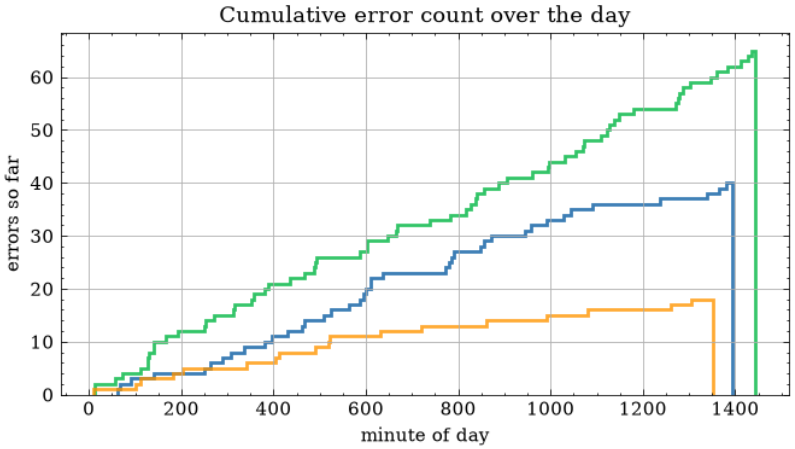
Key settings: **complementary** (draw 1 − F, the fraction above each value — better for upper tails), **as percent**, **distribution fit** (the gap between the steps and the fitted curve is what the Kolmogorov–Smirnov test measures).

Stairs

A step outline: each value is held constant between two edges. With an `x` column, x holds the **edges**, so it has one more row than the values.

Required: `y` **Optional:** `x` (bin edges)

Matplotlib: `Axes.stairs`



> **When to use it.** Data that is already binned (a histogram computed elsewhere) or a rate that holds constant between readings, where a line would suggest a slope that is not there.

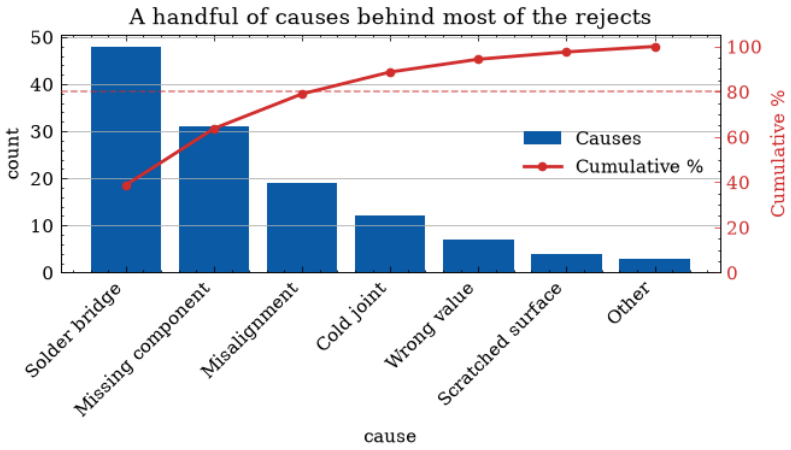
Key settings: **fill**, **orientation**.

Pareto Chart

A bar chart with categories sorted from largest to smallest, plus a line of the cumulative percentage on a second axis and a reference line (80% by default).

Required: `Y` **Optional:** `X`, `color`, error and offset roles as Bar Chart

Matplotlib: `Axes.bar`, `Axes.twinx`, `Axes.plot`



When to use it. Quality and root-cause work — "which few causes account for most of the defects?" Read how far along the axis the cumulative line crosses 80%.

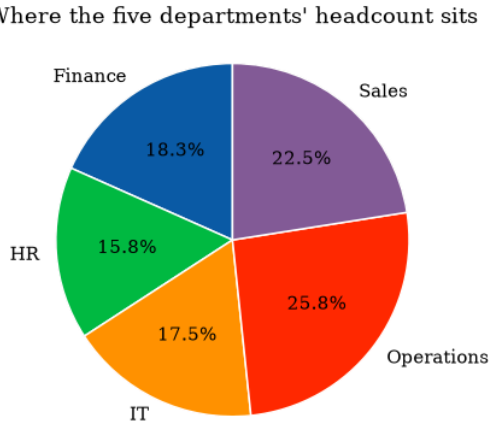
Key settings: **cumulative line**, **reference percent**, **max categories** (lump the rest into "Other"), **ascending**.

Pie Chart

Wedges proportional to `value`, labelled by `label`, with the percentage inside each. `explode` pushes chosen wedges outward. A pie draws only one series.

Required: `value` **Optional:** `label`, `explode`

Matplotlib: `Axes.pie`



When to use it. Showing parts of a whole when there are few parts (up to five or six). For comparisons between parts, a bar chart reads better.

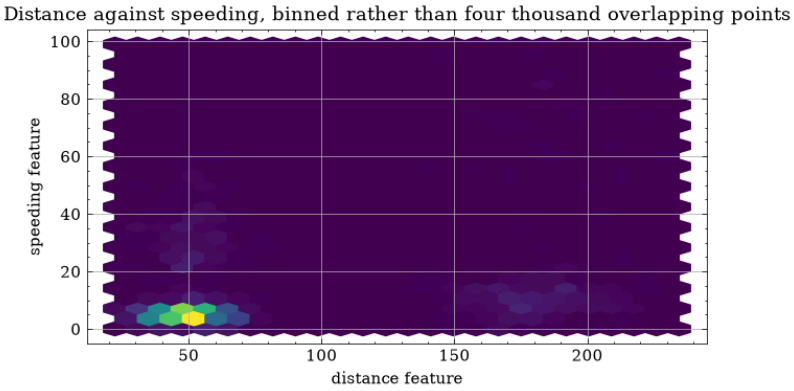
Key settings: **wedge width** (below 1 makes a donut), **colormap**, **start angle**, **percentage format**, **show legend** instead of labels.

Hexbin

Bins the (x, y) plane into hexagons and colours each by how many points fell into it — or, with a `value` column, by the mean, sum, max, min or count of those values.

Required: `x`, `y` **Optional:** `value`

Matplotlib: `Axes.hexbin`



When to use it. Scatter plots with thousands to millions of points, where markers pile on top of each other and hide where the data really is.

Key settings: **grid size**, **minimum count** (blank out sparse hexagons), **count scale** linear or log, **reduce function**, **colour bar**.

Event Plot

One row of short tick marks per series, one tick at each x where an event happened.
Required: `x`
Matplotlib: `Axes.eventplot`

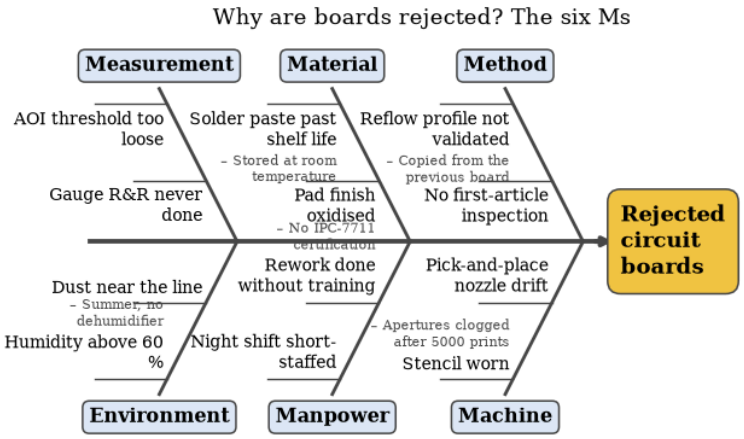


When to use it. Discrete events with no value attached: a digital signal's edges, error log lines, neuron spikes, door openings.

Key settings: **orientation**, row spacing (**line offsets**), tick length (**line lengths**), **row labels**.

Fishbone Diagram

An Ishikawa cause-and-effect diagram: a spine leading to the problem (the head) with one bone per `category` (Method, Machine, Material, …), each listing its `cause` rows and, optionally, a finer `subcause` under each.
Required: `category`, `cause` **Optional:** `subcause`
Matplotlib: `Axes.plot`, `Axes.annotate`



When to use it. Structuring a root-cause analysis in a quality or engineering review.

Key settings: **problem** text (defaults to the series name), **font size**, bone, head and category colours.

Diagnostic plots

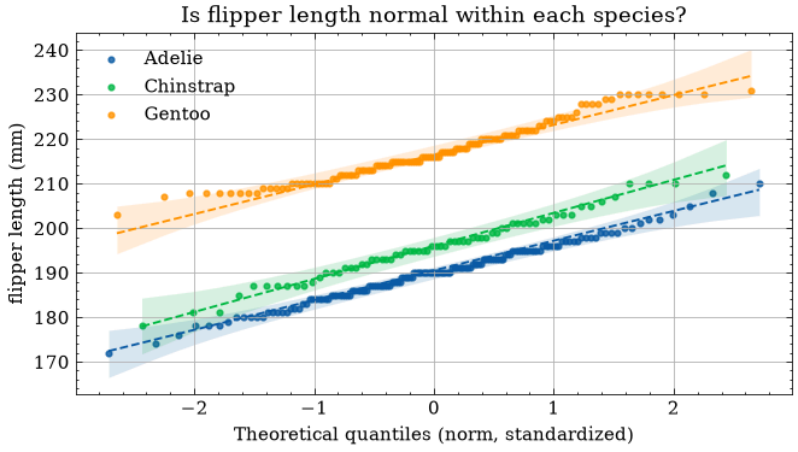
Charts that check a statistical assumption or report a model. Several are drawn for you by **Statistics** (see Section 9) when **Add a chart** is ticked.

Q-Q Plot

Quantile–quantile plot: the sample's sorted values against the quantiles the chosen distribution predicts, after fitting that distribution to the sample. If the distribution fits, the points lie on a straight line. A shaded band shows where a sample truly drawn from that distribution would stay with 95% probability, so points outside it are real departures, not noise. A Q-Q plot is most sensitive in the **tails**.

Required: `value`

Matplotlib: `Axes.plot`, `Axes.fill_between`



When to use it. Checking normality (or any other distribution) before a t-test, an ANOVA or a control chart; seeing **how** a sample departs — an S-shape means heavy or light tails, a curve means skew.

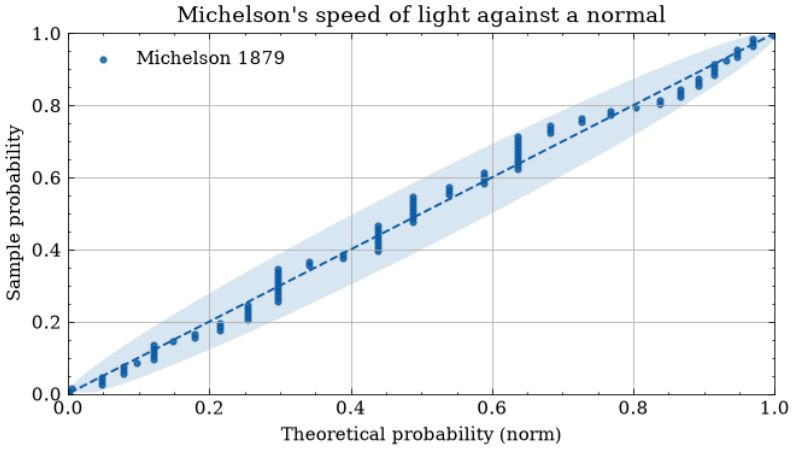
Key settings: **distribution** (normal, lognormal, exponential, gamma, Weibull, logistic, Laplace, Gumbel, Student's t, uniform), **standardized** (theoretical quantiles as z-scores, like the classic normal plot, or in data units where a perfect fit lies on y = x), **reference line** (through the quartiles — not pulled by outliers — least squares, the fitted distribution, or none), **confidence** of the band.

P-P Plot

Probability–probability plot: the sample's cumulative probabilities against those of the fitted distribution. A perfect fit lies on the diagonal. Where the Q-Q plot is sharpest in the tails, the P-P plot is sharpest in the **middle** of the distribution.

Required: `value`

Matplotlib: `Axes.plot`, `Axes.fill_betweenx`



When to use it. Together with the Q-Q plot, when the centre of the distribution matters most.

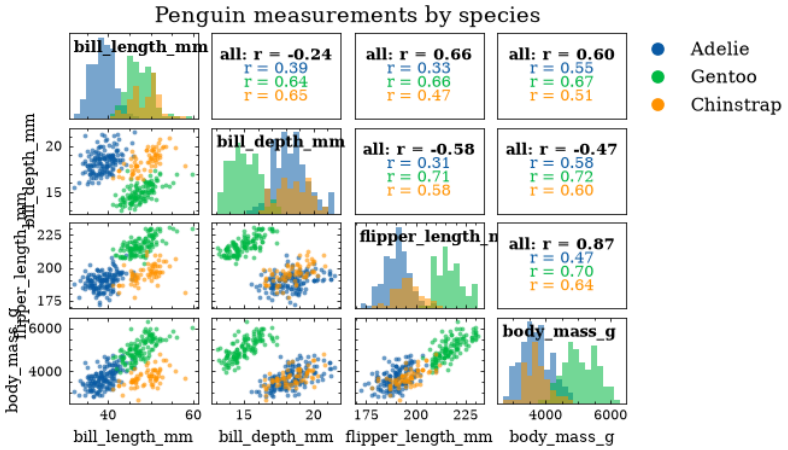
Key settings: **distribution** and **confidence** band, as for the Q-Q plot.

Pair Plot

A grid with one row and one column per numeric variable (`column_1` … `column_6`): a scatter for each pair below the diagonal, each variable's own histogram or density on the diagonal, and Pearson's correlation coefficient (or the mirrored scatter) above it. A `group` column colours points by group and reports the correlation within each group as well.

Required: at least two of `column_1` … `column_6` **Optional:** `group`

Matplotlib: `Axes.scatter`, `Axes.hist`



When to use it. The first look at a table with several measurements: which variables move together, which are skewed, which group separates where — before deciding what to model.

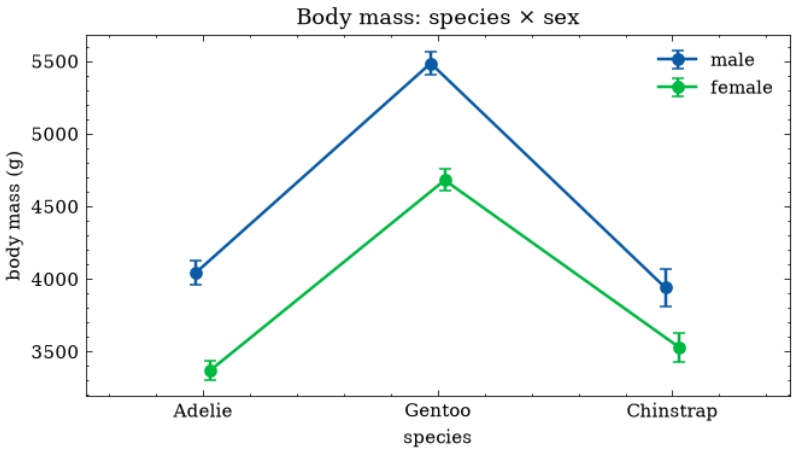
Key settings: **diagonal** (histogram, density, none), **upper** (correlation, scatter, none), **bins**, **max points** (scatters use a random sample of this many rows; histograms and correlations always use every row).

Interaction Plot

The mean of `y` at each level of a factor `x`, with one line per level of a second factor `trace`, and an error bar on every mean. Parallel lines mean the two factors act independently; lines that converge, diverge or cross are an **interaction** — the effect of one factor depends on the other. This is exactly what a two-way ANOVA tests.

Required: `x`, `y` **Optional:** `trace`

Matplotlib: `Axes.errorbar`



> **When to use it.** Designed experiments and any two-factor comparison: dose × sex, temperature × material, treatment × site.

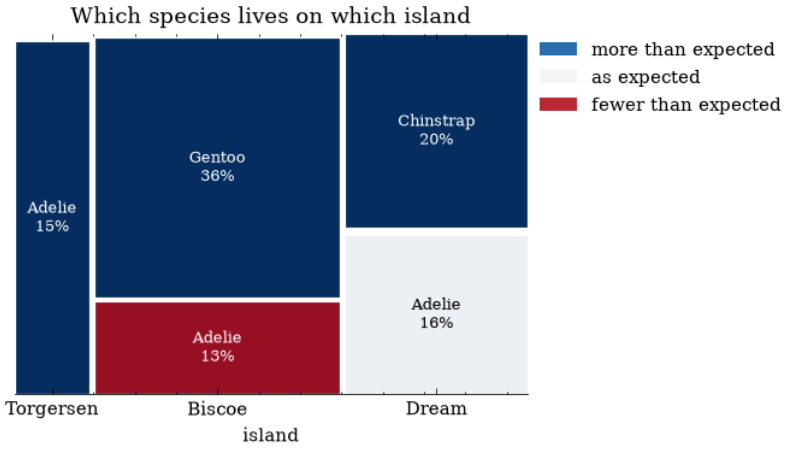
Key settings: **error bars** (standard error, confidence interval, standard deviation, or none), **confidence**, **dodge** (shift the lines sideways so error bars do not overlap).

Mosaic Plot

Two categorical variables as one square cut into tiles. Each column is as wide as its `x` category's share of the rows, and is split into tiles as tall as each `y` category's share within it — so each tile's **area** is the share of rows in that pair. If the variables are independent, every column is split the same way; uneven splits are the association. `weight` counts each row that many times (for a table already counted).

Required: `x`, `y` **Optional:** `weight`

Matplotlib: `patches.Rectangle`, `Axes.text`



> **When to use it.** Showing a contingency table; seeing **which** cells drive a significant chi-squared test.

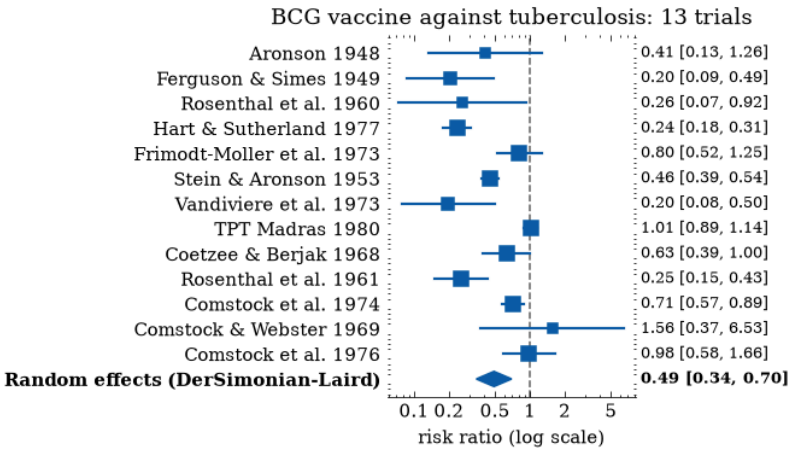
Key settings: **colour by** — the `y` category, or the **Pearson residual** (blue where there are more rows than independence predicts, red where fewer) — **labels** (percent, count, none), **gap** between tiles.

Forest Plot

One row per study, subgroup or coefficient: a square at the `estimate` and a horizontal line from `lower` to `upper` (its confidence interval), against a vertical line of no effect. `weight` scales each square's area so precise studies stand out; rows with a non-zero `summary` are drawn as diamonds (the pooled estimate).

Required: `label`, `estimate`, `lower`, `upper` **Optional:** `weight`, `summary`

Matplotlib: `Axes.plot`, `Axes.text`



When to use it. Meta-analyses, subgroup analyses, tables of regression coefficients or odds ratios.

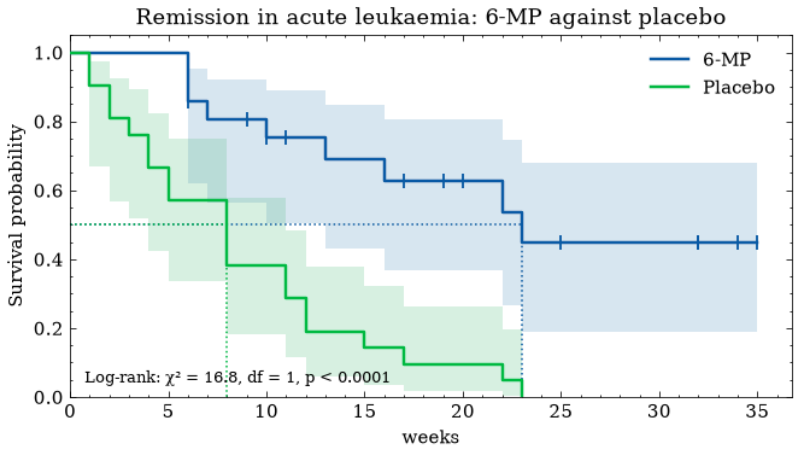
Key settings: **null value** (0 for differences, 1 for ratios), **log scale** (for ratios, so 0.5 and 2 are symmetric about 1), **marker size**, **show values** and their **format**.

Kaplan-Meier

Survival curves for time-to-event data: the estimated probability that the event (failure, relapse, churn) has **not** happened yet, as a step function of time. Rows whose event has not happened by the end of observation (**censored**, `event` = 0) are used for exactly as long as they were observed, which a plain average cannot do. A `group` column draws one curve per group and the log-rank test compares them.

Required: `time`, `event` (1 = event happened, 0 = censored) **Optional:** `group`

Matplotlib: `Axes.step`, `Axes.fill_between`



When to use it. Reliability (time to failure), clinical follow-up, customer retention — any "how long until" question where some cases have not reached the event yet.

Key settings: **confidence** band (Greenwood, log-log scale), **censor marks** (a tick at each censored time), **median line** (dotted guides where each curve crosses 50%: the median survival time), **log-rank** p-value on the chart, **as percent**.

Gridded data

Values sampled on a complete x/y grid: every combination of a set of x values and a set of y values has one row. The grid need not be evenly spaced, but it must be complete.

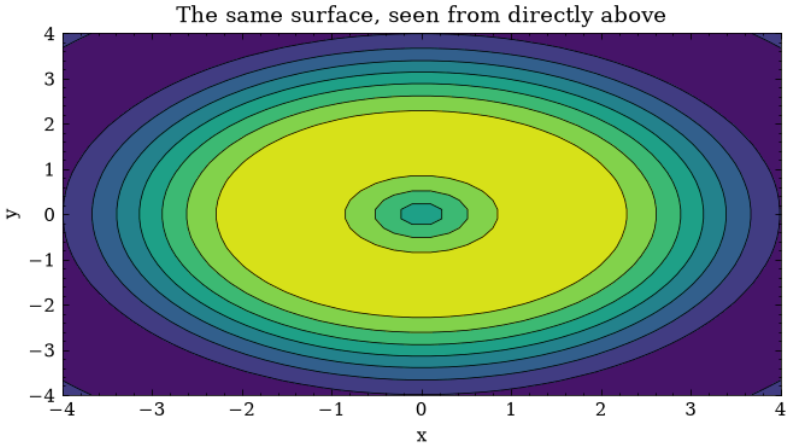
A chart that needs a complete grid draws nothing when the rows are not one, and says so in the log. It never fills missing cells by interpolation and presents the result as data. Use the **scattered** counterpart (Contour Plot (Scattered), Triangular Color Mesh, Surface Plot (Scattered)) for data measured wherever it could be measured.

Contour Plot

Lines of equal `z` (contours) over the x/y plane, or filled bands between them — a topographic map of a function of two variables.

Required: `x`, `y`, `z`

Matplotlib: `Axes.contour`, `Axes.contourf`, `Axes.clabel`



When to use it. Fields and response surfaces: temperature maps, design-of-experiments responses, potential energy surfaces.

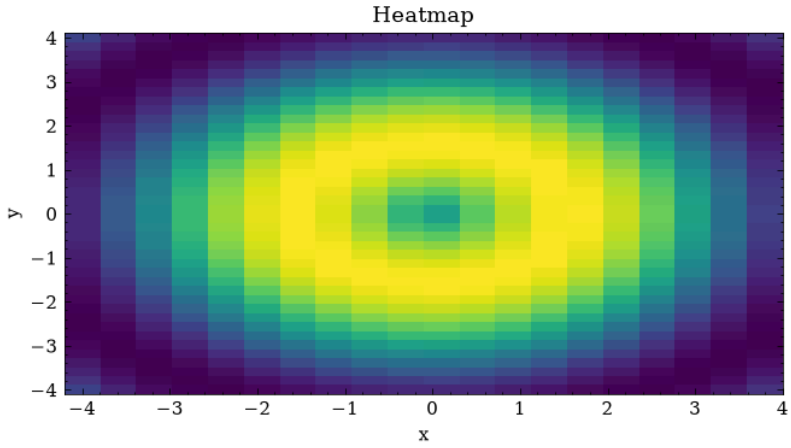
Key settings: **levels** (a number of levels, or an explicit list like `0, 5, 10, 20`), **filled**, **line overlay** on top of the bands, **label lines** with their values (with **format**, **font size**, **inline**), **extend** (colour values beyond the outer levels), **colour bar**, **show points** (mark where the samples are).

Heatmap

Each grid cell painted with the colour of its own value, with no smoothing between cells.

Required: `x`, `y`, `z`

Matplotlib: `Axes.pcolormesh`

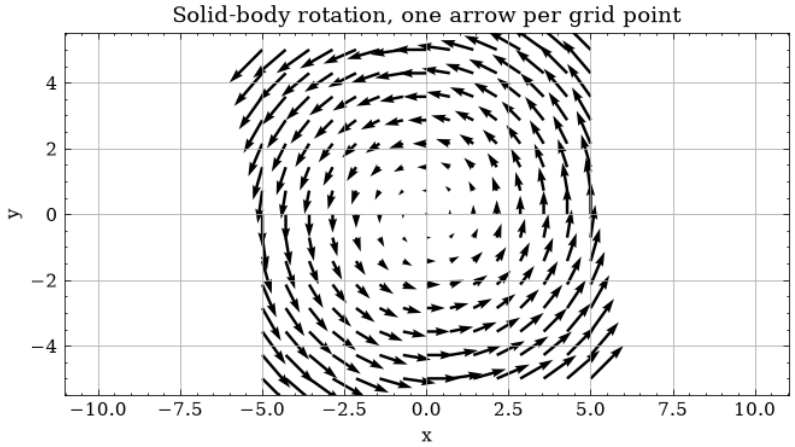


When to use it. Reading the exact value in each cell: correlation matrices, plate readers, confusion matrices, calendars.

Key settings: **shading** (`flat/nearest` keep each cell one colour — the honest reading of measured data; `gouraud` interpolates), **annotate** each cell with its value (with **format**, **font size**, **colour**; skipped automatically on very large grids), **colour bar**.

Quiver

An arrow at each (x, y) pointing in the direction of the vector (u, v), with length proportional to its magnitude. Works on scattered samples too.
Required: x, y, u, v **Optional:** color
Matplotlib: `Axes.quiver`



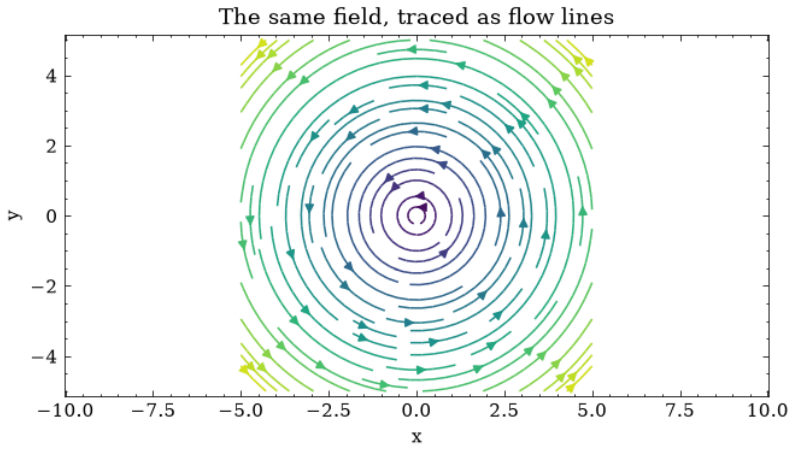
When to use it. Vector fields: wind, currents, gradients, displacement measurements.

Key settings: **scale** (data units per arrow length — fix it when comparing two axes), shaft **width**, **head width/length**, **pivot** (which part of the arrow sits on the point).

Stream Plot

Streamlines traced through the vector field (u, v): where a particle released in the flow would travel.
Required: x, y, u, v

(on an evenly spaced grid)
Matplotlib: `Axes.streamplot`



When to use it. Seeing the overall pattern of a flow — vortices, sources, separation — rather than the vector at each sample.

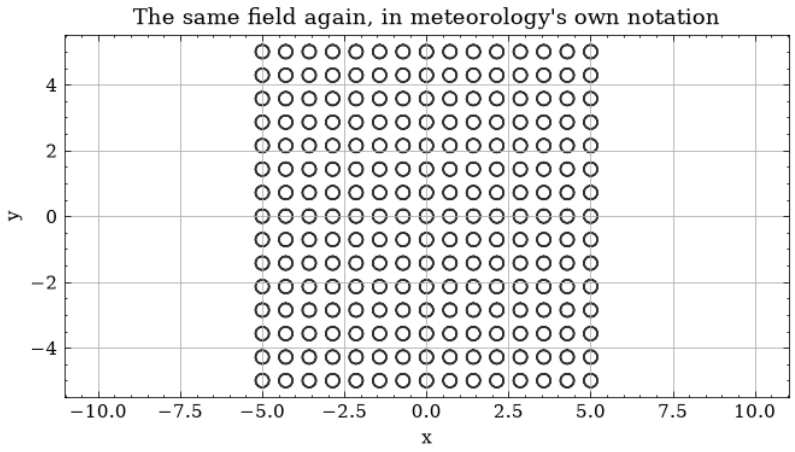
Key settings: **density** of the lines, **arrow size**, **broken streamlines**.

Wind Barbs

The meteorological symbol for a vector: a shaft pointing along (u, v) with flags and half-flags counting the speed.

Required: x, y, u, v

Matplotlib: `Axes.barbs`



When to use it. Wind data, where meteorologists read speed from the flags more precisely than from arrow length.

Key settings: **length**, **barb colour**, **flip barb** (the flags' side, which depends on the hemisphere).

Irregularly gridded data

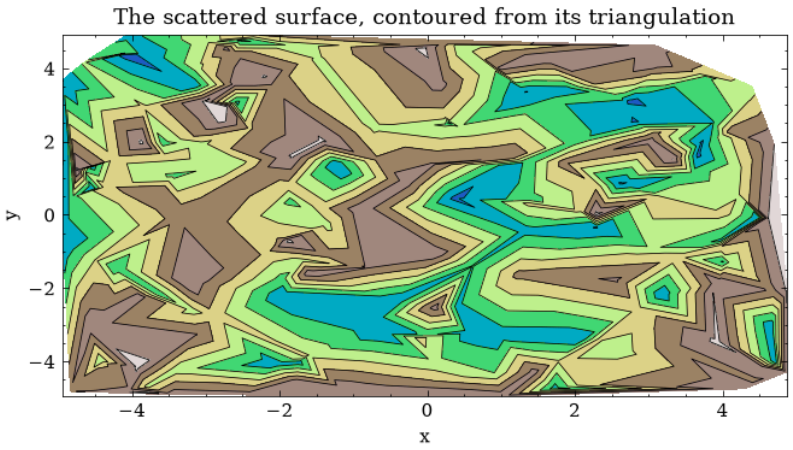
The same quantities as the gridded family, measured at arbitrary points. The points are joined into triangles (a Delaunay triangulation) and the chart is drawn over the triangles.

Contour Plot (Scattered)

Contours of z over scattered (x, y) samples, computed on their triangulation. Same settings as the gridded Contour Plot.

Required: x, y, z

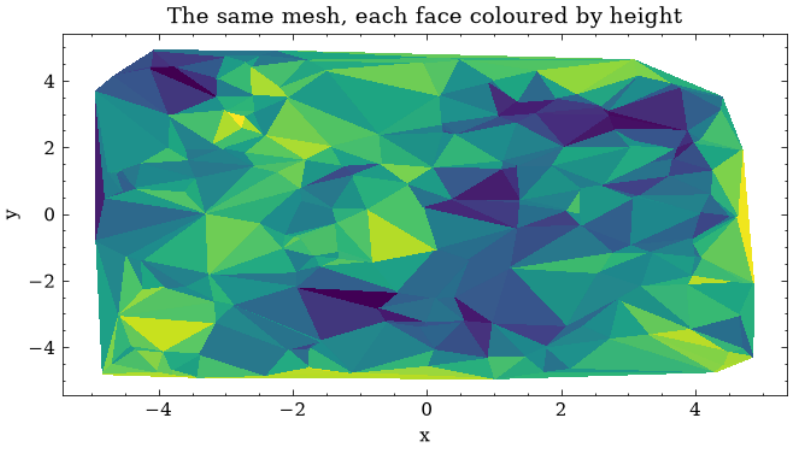
Matplotlib: `Axes.tricontour`, `Axes.tricontourf`



When to use it. Field measurements at irregular positions: boreholes, weather stations, probe points on a part.

Triangular Color Mesh

The triangulated samples, each triangle filled by value — the scattered counterpart of a heatmap.
Required: `x`, `y`, `z`
Matplotlib: `Axes.tripcolor`

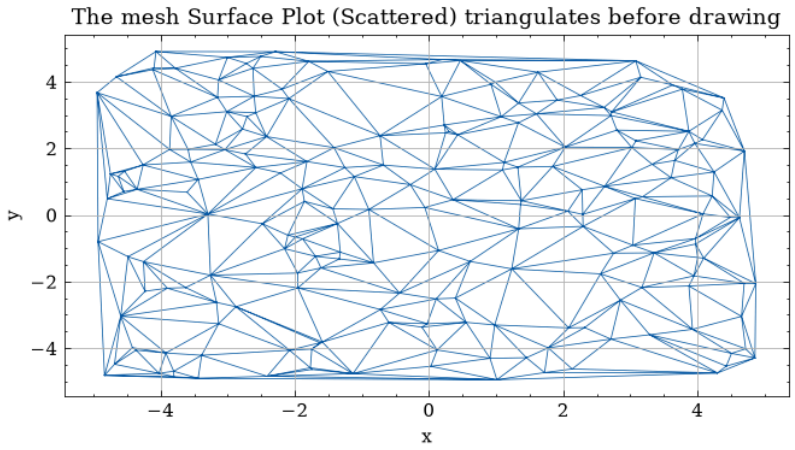


When to use it. Showing a scattered field without the smoothing that contour levels add.

Key settings: **shading** (`flat` gives each triangle the mean of its corners and shows the triangles; `gouraud` interpolates into a smooth field), **colour bar**.

Triangular Mesh

The triangulation itself, drawn as edges and vertices.
Required: `x`, `y`
Matplotlib: `Axes.triplot`



When to use it. Checking where the samples are, and what an interpolating chart will assume between them, before trusting a scattered contour or surface.

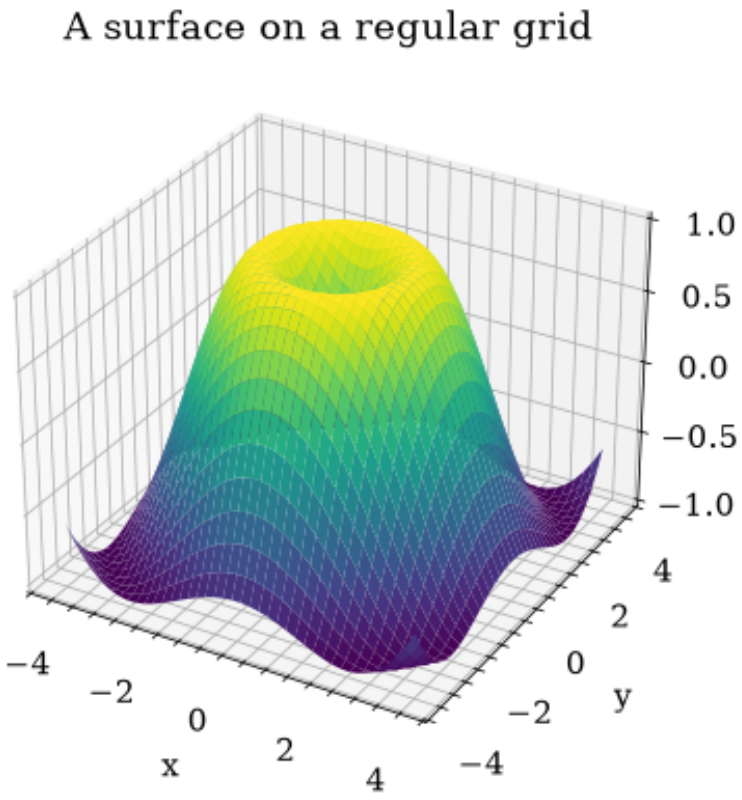
3D and volumetric data

Charts drawn in a three-dimensional box. Drag on the chart to rotate it; the **3D view** settings fix the angle exactly.

Surface Plot

A surface over a complete x/y grid, coloured by height.
Required: `x`, `y`, `z`

(complete grid)
Matplotlib: `Axes3D.plot_surface`

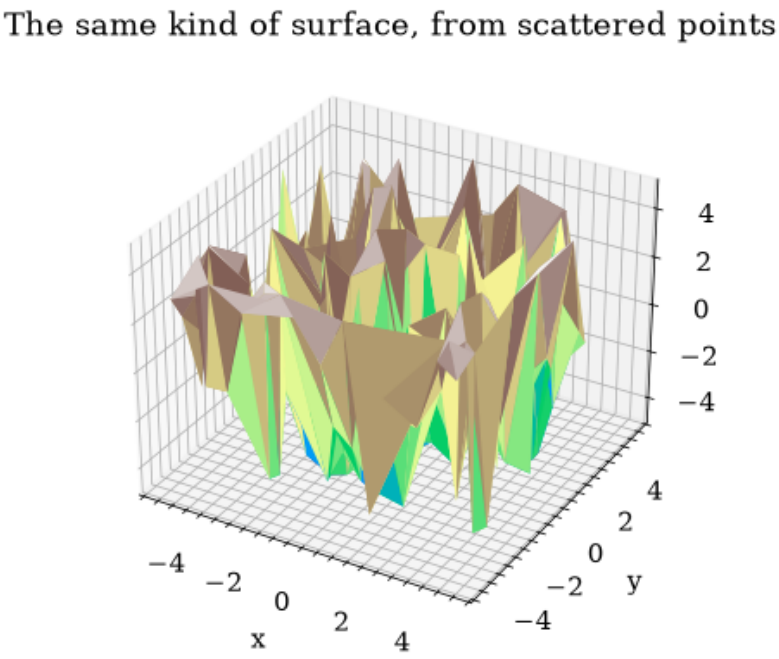


When to use it. Response surfaces and functions of two variables, when the shape in depth matters more than exact values (for exact values, use a Contour Plot or Heatmap).

Key settings: **circular mask** and **mask radius** (blank the grid outside a circle — a wafer-shaped surface), **row/column stride** (draw every Nth row or column of a large grid), **antialiased**.

Surface Plot (Scattered)

A surface over scattered x/y/z points, built from their triangulation.
Required: `x`, `y`, `z`
Matplotlib: `Axes3D.plot_trisurf`

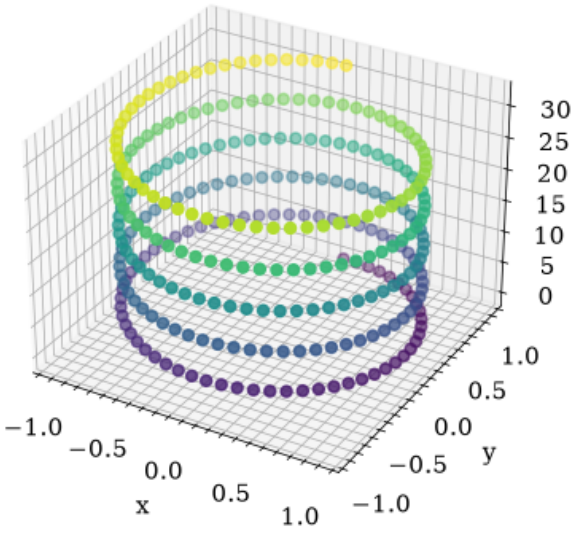


When to use it. As Surface Plot, for measurements not on a grid.

Scatter Plot (3D)

Points in three dimensions, optionally coloured and sized by further columns. Distant points fade (**depth shade**), the main depth cue a still image has.
Required: `x`, `y`, `z` **Optional:** `color`, `size`
Matplotlib: `Axes3D.scatter`

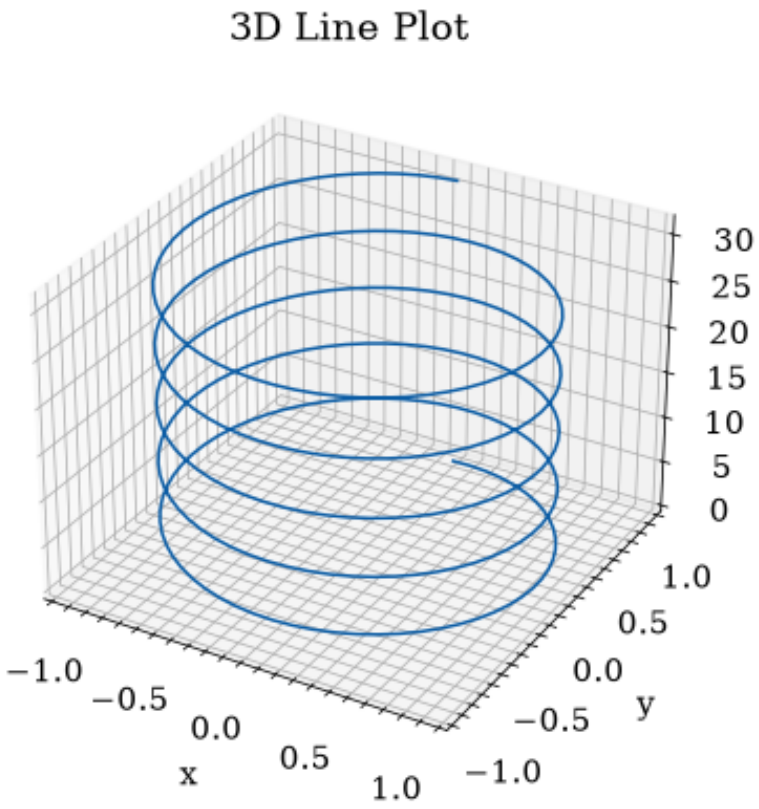
The same helix, as discrete points rather than a line



When to use it. Point clouds whose shape is the message: clusters, a plane, an outlier in three variables.

3D Line Plot

Points connected in row order through three dimensions.
Required: `x`, `y`, `z`
Matplotlib: `Axes3D.plot`



When to use it. Trajectories and parametric curves: a particle path, a robot's motion, a phase portrait.

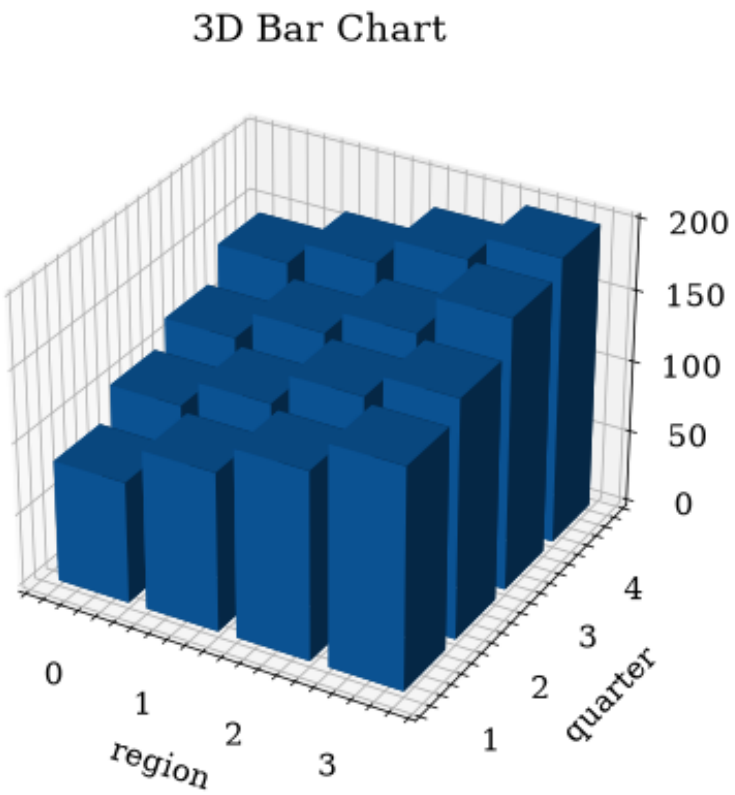
Key settings: line style and width, optional **marker** at each sample.

3D Bar Chart

A bar at each (x, y) rising to height `z`.

Required: `x`, `y`, `z`

Matplotlib: `Axes3D.bar3d`



> **When to use it.** A small table of values read both ways at once — a value per pair of categories — where a heatmap's colour alone is not enough to compare heights.

Key settings: bar **width** and **depth**, **bottom**, **shade**.

Data operations

A **data operation** takes the data of one or more series already on a chart and computes something from it: a smoothed curve, a fitted model, a spectrum, the position of the peaks, a statistical test. The result is added to the chart as new series (and, where it is data, as a new table), so it can be styled, exported and operated on again like anything else. Under each operation, **Computed with** names the SciPy, scikit-learn, statsmodels or NumPy functions that do the calculation, each linked to its documentation — the place to read exactly what a method does and what its parameters mean.

The Data operations page

The **Data operations** page of the navigation rail is a list, organized in sections like a system sidebar. Each row has an icon and a name; resting the pointer on a row shows a one-line description in the bar at the bottom of the page. To find an operation by what it does, use the search of **Guided analysis** or **Cmd/Ctrl+K** from anywhere (see Section 8.3).

Plot	**Plot** — creates a new chart (see Section 5).
Data	**Query Builder** — writes and saves SQL queries (see Section 4.4).
Analysis	Peaks, Roots, Calculus.
Statistics	Statistics, Outliers, Clustering, Control Chart, Transform, Decomposition.
Geometry	Geometry.
Signal Processing	Smoothing, Spectral Analysis, Filtering, Baseline Correction.
Modeling	Fit, Interpolation, Function, Regression, GP Regression.

An operation added by you or a plugin (see Section 12.2) appears under **Other** unless it is given a section.



Figure 4: The Data operations page.

How every operation window works

Every operation opens the same kind of window, operating on the figure currently shown:

- **On the left**, in tabs: the **Axis / Series** selector (which axis of the figure, and which of its series to read), the **Model** (the method, with a link to its documentation), and the **Parameters** of that model. Each parameter explains itself in its tooltip.
- **On the right**, the **Results** pane: a formatted report of what was computed — tables of numbers, fit statistics, warnings — and below it the log of the computation.
- **At the bottom**, the buttons: **Preview**, **Copy** (copies the report), **OK** and **Close**.

The workflow is always the same:

1. Choose the axis and tick the series to operate on.
2. Choose a model and set its parameters.
3. Click **Preview**. The calculation runs, the report fills the Results pane and the result is drawn on the chart, so you can judge it in place.
4. Adjust and **Preview** again as often as needed. Each Preview replaces the previous one.
5. Click **OK** to keep the result and close the window — or **Close** to discard the preview and leave the chart exactly as it was.

> Nothing is computed while you are editing. Changing a model, a parameter or the selected series only marks the current result as out of date; **Preview** and **OK** are what run the calculation. A slow operation can be set up calmly, and a half-typed number is never computed.

> A calculation that takes a while — a **Fit** above all — runs in the background. The window stays usable, a **Stop** button and a progress bar appear beside **Preview**, and **Stop** ends the calculation at its next step without changing anything. Closing the window stops it too.

What an operation writes

When **OK** is clicked:

- Results that are data (a smoothed curve, a spectrum, fitted values) are saved as new **tables** in the project. Their names start with an underscore (`_Smoothing_temperature`, …) so they are easy to tell apart from imported data in the table list.
- New **series** reading those tables are added to the chart: on the same axis when the result is in the same units as the source (a smoothed or fitted curve), on a new axis of the same figure when it is not (a derivative, a spectrum, a control chart).
- The report is added to the figure's **notes**: a pane below the chart that collects the reports of every operation applied to it, and that goes into exported reports (see Section 10).
- The operation, its parameters and its report are recorded in the project's **history** (see Section 10.4), so it can be reviewed later.

Statistics and **Geometry** are the two exceptions: Statistics writes no tables (it reports, and draws a chart only on request), and Geometry writes a query instead of a table.

One series or several

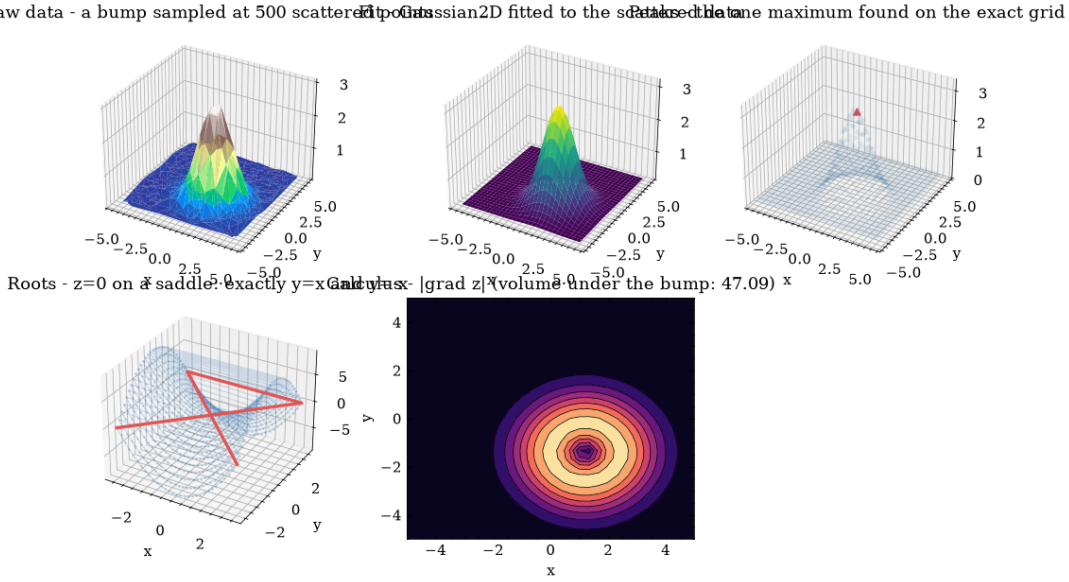
Most operations work on **each ticked series separately**, producing one result per series — smooth three series at once, get three smoothed curves. **Decomposition** needs several series and combines them; **Statistics** reports on each series and also compares them in pairs and as groups; **Spectral Analysis** pairs the first ticked series with each of the others for its two-signal estimators.

Curves, surfaces and fields

The **Axis / Series** selector labels each series with the shape it found:

`2D (x, y)`	An ordinary curve — every operation's usual case.
`3D on a grid (x, y, z)`	The rows form a complete x/y grid, so the surface is read exactly as measured.
`3D scattered (x, y, z)`	Points with no grid. Operations that need a grid interpolate onto one and leave everything outside the samples' convex hull blank, rather than inventing values where nothing was measured.
`vector field (x, y, u, v)`	Shown for information; no operation computes on vector fields yet.

Five operations also work on surfaces: **Fit**, **Function**, **Peaks**, **Roots** and **Calculus**. **Smoothing** has its own 2D and 3D models, and **Geometry** moves 3D series in space. Each is described below.



Analysis

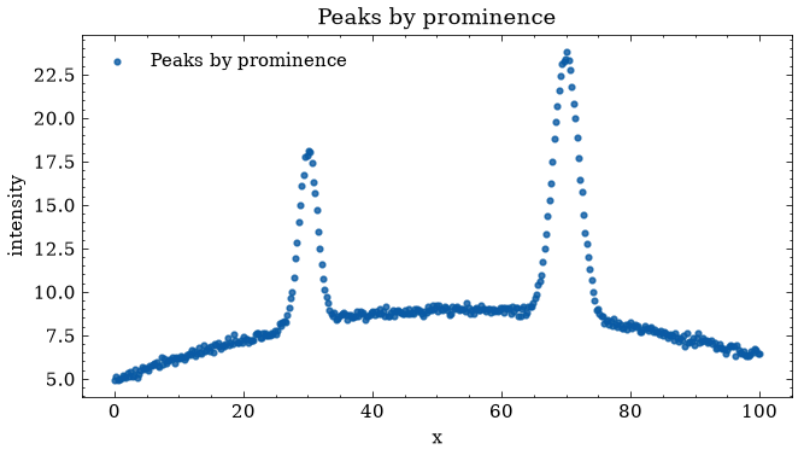
Peaks — find and measure peaks

Finds the local maxima (and/or minima) of a series and measures each one: position, height, **prominence**, width at half prominence and the x bounds of that width. The peaks are marked on the chart and listed in the report.

A raw search for local maxima on real data returns hundreds of hits, almost all noise, so Peaks is built around the filters that decide what counts as a peak:

Model	**Maxima**, **Minima**, or **Maxima and minima**.
Filter by	**Prominence** (default): how far a peak stands above the higher of the two valleys either side of it. It works on a sloping or raised baseline. **Height**: the value compared with zero, meaningful only when the baseline is at zero.
Threshold	The minimum prominence or height, as a fraction of the signal's full range (0.1 keeps peaks standing at least a tenth of the range above their surroundings).
Distance	Minimum separation in points; of two peaks closer than this, the more prominent is kept. This is what stops one noisy peak being reported as several.
Min width	Minimum width at half prominence, in points. Raise it to reject single-sample spikes.
Limit	Keep at most this many peaks, the most prominent first.

Computed with: `scipy.signal.find_peaks`, `scipy.signal.peak_prominences`, `scipy.signal.peak_widths`, `scipy.ndimage.maximum_filter`



When to use it. Spectroscopy and chromatography (peak positions and widths), finding cycles and extremes in any signal. The half-prominence bounds are exactly what **Calculus** needs to integrate a peak's area.

On a **surface**, Peaks finds local maxima and minima in two dimensions with the same parameters.

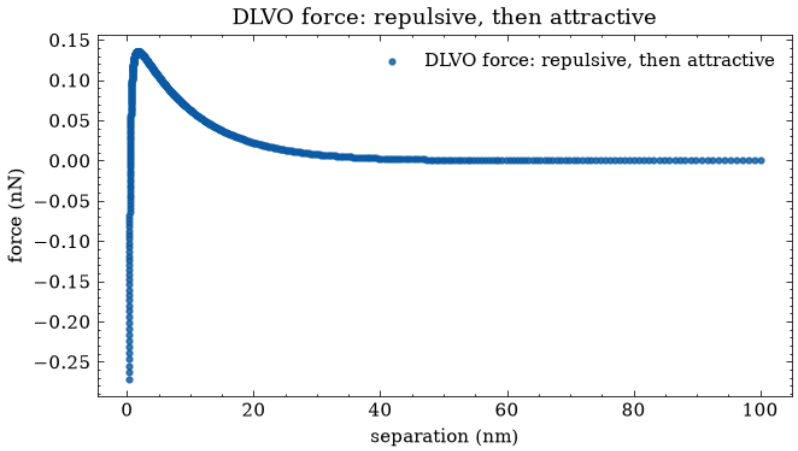
Roots — find where a series crosses a level

Finds every x at which the series crosses a chosen **level** — the zeros of y(x) − level. It first scans consecutive samples for a change of sign (each such pair **brackets** a crossing), then refines each bracket with a numerical solver on an interpolating curve. Because the solvers cannot leave their bracket, refining can only improve the answer. Each root is reported with its direction (rising or falling), the solver's iterations and its residual.

Model (solver)	**Brent** (default, fast and safe), **Bisection** (slowest, most robust), **TOMS 748** (often fewest iterations), **Newton (secant)**.

Level	The y value to solve for; 0 finds the zeros.
Interpolation	What the series does between samples: **Straight line** (claims nothing the samples do not say), **Cubic spline** or **Monotone cubic (PCHIP)** (better for smooth signals; a spline can overshoot noisy data into crossings that are not there).
Tolerance digits	How many decimal places of x the solver must settle.
Max iterations	Per crossing.
Limit	Keep the first N crossings in x order.

Computed with: `scipy.optimize.brentq`, `scipy.optimize.toms748`, `scipy.optimize.newton`, `scipy.interpolate.CubicSpline`, `scipy.interpolate.PchipInterpolator`



When to use it. Titration endpoints, the time a response falls to half its value, threshold crossings, the temperature at which a difference changes sign, the zero crossings of a derivative (maxima and minima).

On a **surface**, Roots traces the **level curve** z = level — a contour line — instead of a list of points.

Calculus — differentiate or integrate

Computes the numerical derivative or integral of a series. Two things that are easy to get wrong are built in:

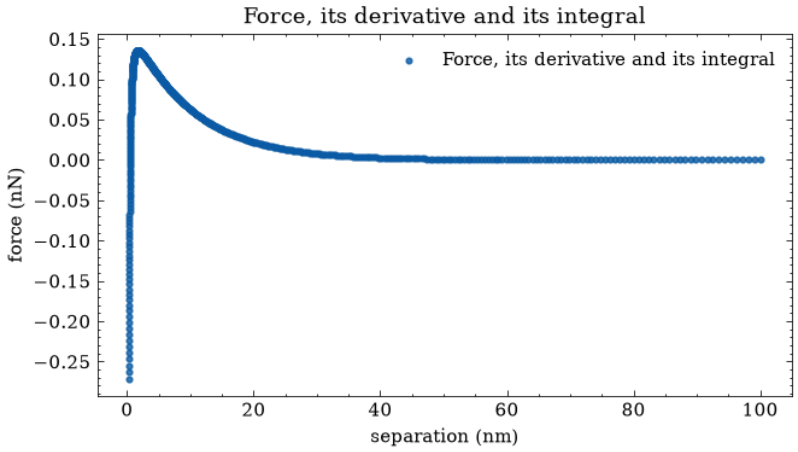
- **Smoothing inside the derivative.** Differentiation amplifies noise: a finite difference divides by a small step, so small noise becomes large noise. The default derivative fits a low-order polynomial over a moving window and differentiates that (Savitzky–Golay), smoothing and differentiating in one step.
- **Baseline removal inside the integral.** The area under a peak is meaningless if the signal does not return to zero, because a constant offset adds offset × width to the area.

Derivative (Savitzky-Golay)	Default. **Window** (points per local fit; larger smooths more), **polynomial order**, and **derivative order** (1 for the slope, 2 for the curvature).
Derivative (finite difference)	Plain central differences: exact on clean data, noisy on measured data.
Derivative (spline)	Differentiates a smoothing spline; **smoothing** 0 passes through every point, larger values allow the spline to depart from noisy data.
Integral (cumulative)	The running integral (trapezoidal rule), as a new curve.

Integral (total area)	One number: the area under the curve. **Simpson's rule** is more accurate on smooth data. **Baseline**: none, subtract the minimum, or subtract the straight line between the endpoints.
Derivative (surface gradient)	For a surface: the gradient magnitude at each point, with ∂z/∂x and ∂z/∂y alongside.
Integral (surface volume)	For a surface: the volume under it.

Derivatives are drawn on a new axis by default (a slope is in different units from the data). On a date axis, the derivative is expressed per natural unit of the sampling interval — per day for daily data — rather than per second.

Computed with: `scipy.integrate.trapezoid`, `scipy.integrate.simpson`, `scipy.integrate.cumulative_trapezoid`, `scipy.signal.savgol_filter`, `scipy.interpolate.UnivariateSpline`



When to use it. Rates of change (velocity from position, growth rate), finding inflection points, peak areas (concentration from a chromatogram), totals from rates (energy from power).

Statistics

Statistics — tests and summaries

Reports descriptive statistics and hypothesis tests on the ticked series, and can add a chart of what was tested. It is described fully in Section 9.

Outliers — detect anomalies

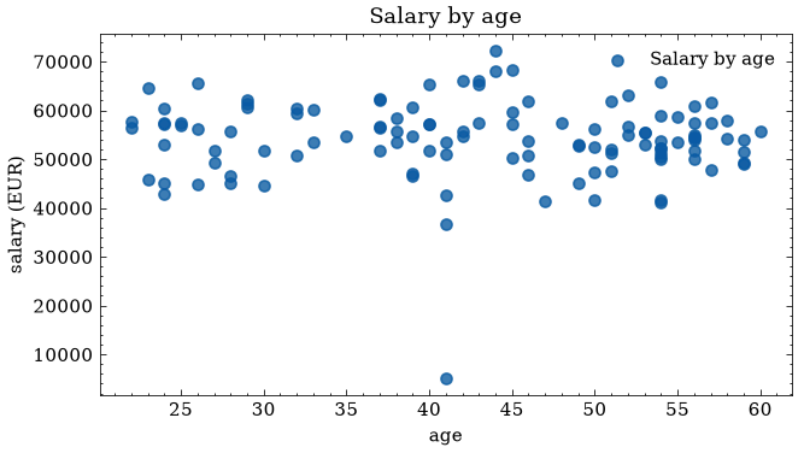
Flags the points of a series that do not belong with the rest, and either **hides** them from the chart or **colours** them.

Z-score threshold	Points more than **threshold** standard deviations from the mean. Simple, but the outliers themselves inflate the standard deviation.
Interquartile range	Points beyond Q1 − k · IQR or Q3 + k · IQR (**IQR factor** k, 1.5 by default — the box-plot rule). Robust to the outliers it is looking for.
Median absolute deviation	Like the z-score but built on the median and the MAD, so a few extreme values cannot hide each other.
Rolling median residual	Compares each point with the median of a moving **window** around it — finds spikes in a signal that trends or oscillates, where a global rule would flag the peaks of the signal itself.

Isolation Forest	Machine learning: points that are easy to isolate by random splits are anomalous. **Contamination** is the expected fraction of outliers.
Local Outlier Factor	Points in regions much sparser than their **neighbours'** regions.
One-Class SVM	Learns the boundary of the normal points; **nu** bounds the fraction treated as outliers.
Elliptic Envelope	Fits a robust ellipse (Minimum Covariance Determinant) to the cloud; points far outside it are outliers.

The last four judge a point by its position relative to the others in (x, y), not by its value alone, so they can find points that are unusual for their x even when their y is ordinary. Both coordinates are rescaled robustly first, so x and y in different units weigh equally.

Action: **Hide them** marks the rows in the table's `Hide` column, which every chart skips — the data stays in the table, and the rows can be un-hidden from the table editor (see Section 4.3). **Colour them** keeps every point on the chart and draws the outliers in the chosen **colour**.
Computed with: `sklearn.ensemble.IsolationForest`, `sklearn.neighbors.LocalOutlierFactor`, `sklearn.svm.OneClassSVM`, `sklearn.covariance.EllipticEnvelope`, `scipy.ndimage.median_filter`



> **When to use it.** Cleaning measurement glitches before a fit, flagging anomalous batches, checking how sensitive a result is to a few points.

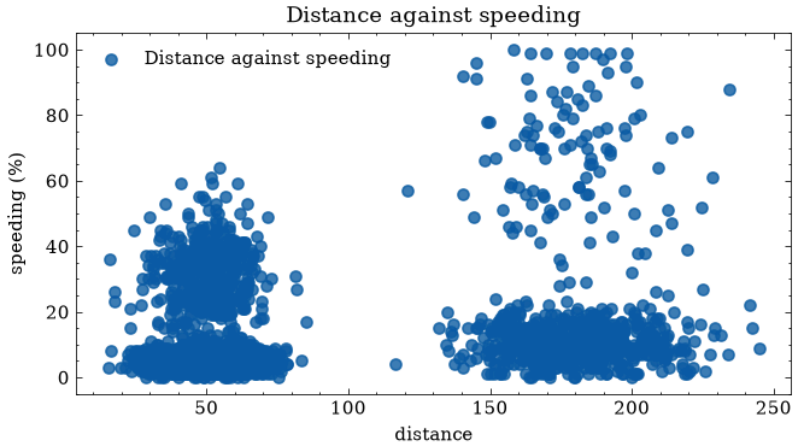
Clustering — group similar data

Groups the points of a series into clusters and writes each row's cluster number into a `ClusterId` column of the source table. The result is drawn either as **one series coloured by ClusterId** or as **separate series per cluster** (each with its own legend entry and style).

K-means / vector quantization	SciPy's k-means: a fixed **number of clusters**, each the set of points nearest one centre. **Whiten** rescales features first; **iterations** and **threshold** control convergence.
Hierarchical / agglomerative	Merges the closest points and groups step by step. **Linkage**: single, complete, average, weighted, centroid, median, Ward. Cut either at a number of clusters or at a **distance threshold**.
scikit-learn	K-Means, Mini-Batch K-Means, Bisecting K-Means, Agglomerative, **DBSCAN** and **OPTICS** (density-based: find clusters of any shape and leave noise points unassigned; set the

neighbourhood radius **eps** and **min samples**), Birch, Mean Shift, Spectral Clustering, Gaussian Mixture.

Features chooses which numeric columns of the series are used. A **max runtime** stops algorithms that scale badly on large tables.
Computed with: `scipy.cluster.vq.kmeans2`, `scipy.cluster.hierarchy.linkage`, `scipy.cluster.hierarchy.fcluster`, `sklearn.cluster.KMeans`, `sklearn.cluster.MiniBatchKMeans`, `sklearn.cluster.DBSCAN`, `sklearn.cluster.OPTICS`, `sklearn.cluster.AgglomerativeClustering`, `sklearn.cluster.SpectralClustering`, `sklearn.cluster.Birch`, `sklearn.cluster.MeanShift`, `sklearn.mixture.GaussianMixture`



When to use it. Finding natural groups — material phases, customer segments, operating regimes — and then analysing each group separately.

Control Chart — monitor process stability

Builds a Shewhart control chart and flags the points that signal a change in the process. The chart is drawn on a new axis with its centre line, control limits and, optionally, the A/B/C zones.

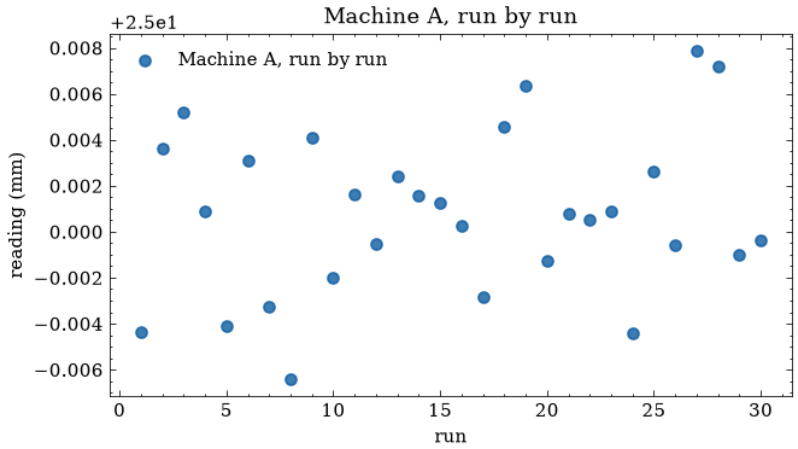
The key point of a control chart is where its limits come from: never from the standard deviation of all the data (a process that drifted has a large overall spread **because** it drifted), but from the variation **within** short stretches of it — the moving range for individual values, the range or standard deviation within subgroups otherwise.

Individuals (I-MR)	One measurement per point; sigma from the average moving range.
Moving range (MR)	The companion chart of point-to-point ranges.
X-bar and R	Means of consecutive **subgroups** of size n (2–25), sigma from the average range. For n up to about 8.
X-bar and S	The same, sigma from the subgroup standard deviations. Better for larger subgroups.
p	Fraction defective per sample. Needs a **sample size column**; limits widen and narrow with the sample size.
np	Count defective per sample, constant sample size.
c	Defects per unit, constant area of opportunity.
u	Defects per unit with a varying sample size (needs the sample size column).

Sigma limit	Distance of the limits from the centre, in sigmas; 3 is the Shewhart convention (about one false alarm per 370 points on a stable process).
Nelson rules	Besides points beyond the limits (rule 1), flag patterns inside them: 9 points in a row on one side of the centre (2), 6 steadily rising or falling (3), 14 alternating up and down (4), 2 of 3 beyond 2σ (5), 4 of 5 beyond 1σ (6), 15 in a row within 1σ (7), 8 in a row outside 1σ on both sides (8). The zone rules (5–8) are switched off on charts whose limits move with the sample size.
Exclude violations	Recompute the limits without the flagged points — the "trial limits, then revised limits" step. Use it only when the flagged points have a known cause that has been removed.
Draw	Centre line, limits, zones, and violations as separate markers.

The report lists each signalling point and the rule it broke. A chart with no signals says the process is **stable** — which is not the same as meeting a specification.

When to use it. Manufacturing and laboratory quality control, monitoring any process over time.

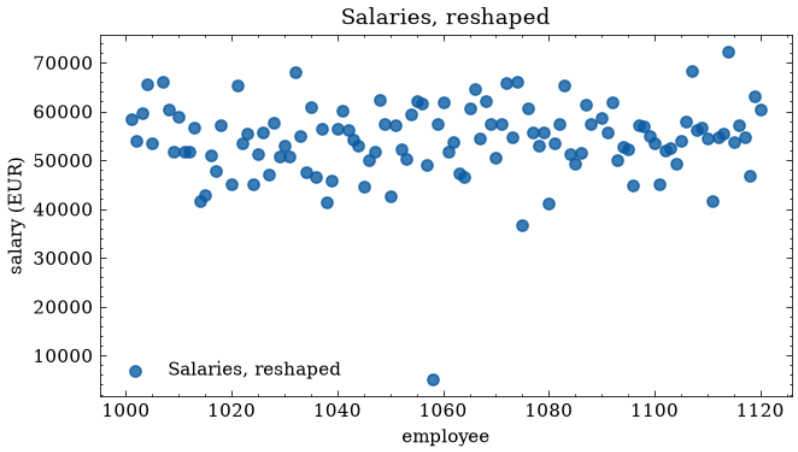


Transform — rescale or reshape a distribution

Replaces y with a transformed version of itself, point for point (same x, same number of points), on a new axis.

Power transform	Makes a skewed distribution closer to normal. **Yeo-Johnson** accepts any values; **Box-Cox** often fits better but needs strictly positive values. The fitted λ is reported.
Quantile transform	Maps values through their ranks onto a **uniform** or **normal** distribution. Very strong: any shape becomes the target shape.
Standard scaler	z-scores: subtract the mean, divide by the standard deviation.
Robust scaler	Subtract the median, divide by the IQR — unaffected by outliers. **With centering** and **with scaling** switch the two steps.

Computed with: `sklearn.preprocessing.PowerTransformer`, `sklearn.preprocessing.QuantileTransformer`, `sklearn.preprocessing.StandardScaler`, `sklearn.preprocessing.RobustScaler`



When to use it. Preparing data for a method that assumes normality or comparable scales — a test, a control chart, a clustering on several variables.

Decomposition — several series together

The one operation that combines **several** series into one calculation. Every ticked series is resampled onto a common grid of **n grid** points over their shared x range, giving a matrix with one column per series.

PCA	Principal components: the directions of greatest shared variation, ranked by the share of variance each explains (reported).
Fast ICA	Independent components: separates signals that were mixed together (the classic "cocktail party" problem).
NMF	Non-negative factors: parts that add up to the series, for data that cannot be negative (spectra, concentrations).
t-SNE, **Isomap**, **Locally Linear Embedding**	Manifold learning: places each grid sample as a point in 2D so that similar samples are close, drawn as a scatter on a new axis. Exploratory — the axes of the map have no units. **Perplexity** (t-SNE) and **neighbours** (Isomap, LLE) set how local the structure is.

Components sets how many to extract. PCA, ICA and NMF draw each component as a curve over the shared x.

Computed with: `sklearn.decomposition.PCA`, `sklearn.decomposition.FastICA`, `sklearn.decomposition.NMF`, `sklearn.manifold.TSNE`, `sklearn.manifold.Isomap`, `sklearn.manifold.LocallyLinearEmbedding`

When to use it. Finding the few patterns behind many correlated signals (several sensors, many spectra), separating mixed sources, exploring similarity.

Geometry

Geometry — move a series' coordinates

Rotates, translates, scales, mirrors or shears a series, or applies any matrix you type. It answers "put this where that is": rotate a scan onto a reference frame, shift a profile, flip it, scale a model onto measured units.

Rotate	Turns the series counter-clockwise about a centre, by an angle in degrees.
Translate	Adds a fixed amount to every x and y (and z).
Roto-translation	A rotation about the centre followed by a translation: a rigid motion; shape and size never change.
Scale	Multiplies x and y (and z) by their own factors about the centre. Unequal factors change the shape.
Mirror	Reflects across a horizontal, vertical or diagonal line through the centre.
Shear	Leans the series over: so much x added per unit of y, or y per unit of x.
Matrix	Any linear map you type. **2 × 2**: $x' = a_{11}x + a_{12}y$, $y' = a_{21}x + a_{22}y$ — every motion above except a move is one of these. **3 × 2** adds a third row of offsets, b_1 and b_2, added afterwards. The matrix acts about the centre; for the formula exactly as written, turn off **Centre on the data** and set the centre to 0, 0.

Centre on the data (on by default) uses the middle of the series' own extent as the fixed point — "turn this shape" rather than "turn it about the origin". Turn it off to type the centre.

In 3D. A series with a z column is moved in space: three rotation angles (about x, then y, then z, each by the right-hand rule), and translation, scale and centre gain a z value. Mirror, Shear and Matrix act in the x–y plane. The preview becomes a 3D view.

The Results pane shows a picture with no axes or labels: the source points in blue, the result in red (at most 200 of each) and, with **Show the deformation grid**, a square mesh around the source before and after (a cube in 3D), which makes a shear or an uneven scale easy to read.

> Geometry computes nothing and writes no table. The new series carries a SQL query — the source series' own query wrapped in the transform's arithmetic, with a 2D rotation written as `cos(radians(30))` so it stays readable. The moved series therefore **follows its source** (change the source and the transform applies to the new data), costs no extra space, and can be adjusted afterwards in the Query Builder.

Signal processing

Smoothing — reduce noise

Produces a smoothed copy of a series, drawn beside the original. The **data type** selector chooses between models for curves (1D), surfaces (2D) and volumes (3D).

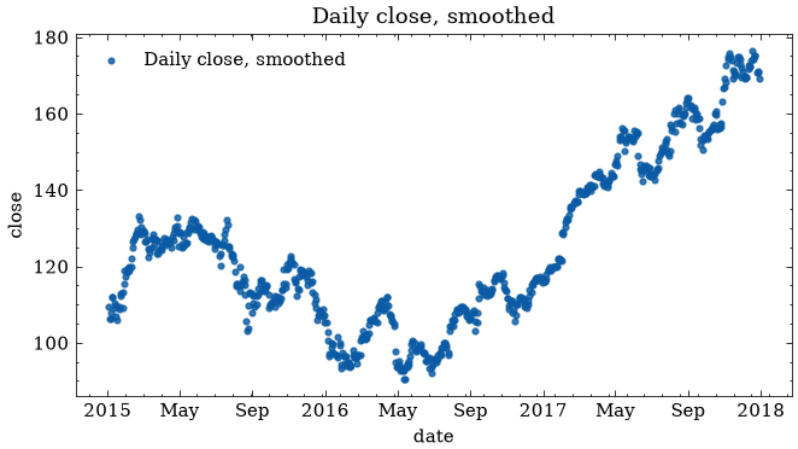
Curves (1D):

Moving Average	Mean over a sliding **window**, optionally centred. Simple; flattens peaks.
Savitzky-Golay	Local polynomial fit over a window. Smooths while preserving peak heights and widths better than a moving average. Can also return a derivative.

Gaussian Filter	Weighted average with a Gaussian kernel of width **sigma**.
Median Filter	Median over a window. Removes isolated spikes and keeps sharp steps.
Wiener Filter	Adaptive: smooths more where the local variance is close to the noise level.
Smoothing Spline	A spline of degree **k** with smoothing factor **s** (0 passes through every point).
LOWESS / LOESS	Locally weighted regression on a **fraction** of the data around each point, with robustness **iterations** that down-weight outliers.
Kalman Smoother	A state-space smoother; set the **process** and **measurement variances**. Good for tracking a slowly changing level in noisy readings.
FFT Low-Pass	Removes frequencies above a **cutoff ratio** of the spectrum.
Butterworth Filter	A classic low-, high- or band-pass filter of a given **order** and **cutoff**, applied forward and backward (no phase shift).
Wavelet Denoising	Thresholds the wavelet coefficients (**wavelet**, **level**, **threshold mode** and **factor**). Keeps sharp features that linear filters blur.
Whittaker-Eilers	Penalized least squares with smoothness **lambda** and difference **order**. Fast, handles any length.
Hodrick-Prescott	Separates a smooth trend from cycles (**lambda**: 1600 is the convention for quarterly data).
Total Variation Denoising	Removes noise while keeping steps sharp (**weight**). Ideal for piecewise-constant signals.

Surfaces (2D) and volumes (3D): Gaussian, Median, smoothing and rectangular bivariate splines, RBF (radial basis function) smoothing, FFT low-pass and Total Variation, each with per-axis sizes where it applies.

Computed with: `scipy.signal.savgol_filter`, `scipy.ndimage.gaussian_filter1d`, `scipy.signal.medfilt`, `scipy.signal.butter`, `scipy.signal.sosfiltfilt`, `scipy.signal.wiener`, `scipy.interpolate.UnivariateSpline`, `scipy.linalg.solveh_banded`, `statsmodels.nonparametric.smoothers_lowess.lowess`, `statsmodels.tsa.filters.hp_filter.hpfilter`



When to use it. Making a trend visible in noisy data, preparing a signal for peak finding or differentiation. When in doubt, start with Savitzky-Golay or LOWESS.

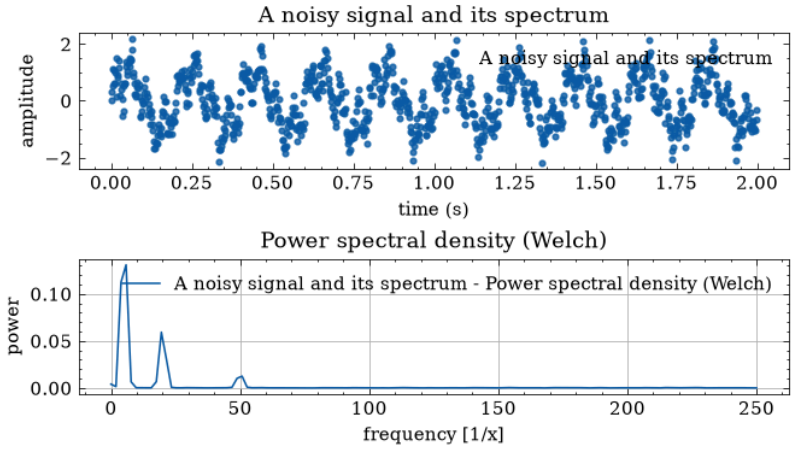
Spectral Analysis — analyse frequencies

Computes how a signal's energy is distributed over frequency, or how it correlates with itself or another signal over lag. The results are drawn on a new axis.

Power spectral density (Welch)	The standard estimate of power per frequency for a noisy signal: averages the spectra of overlapping segments.
Cross spectral density (Welch)	For a pair of signals: where they share power, and with what phase.
Coherence	The normalized cross spectrum: how linearly related two signals are at each frequency, from 0 to 1.
Magnitude / Phase (unwrapped) / Angle (wrapped) spectrum	The raw one-sided FFT, for deterministic signals where Welch's averaging would smear sharp peaks.
Laplace transform (damped FFT)	The FFT of the signal weighted by $e^{-\sigma t}$, for decaying or growing signals. **Damping σ** $= 0$ reduces it to the Fourier transform.
Wavelet power spectrum (Morlet)	Frequency content **over time** — how the spectrum changes along the signal. **Morlet w0** trades frequency resolution for time resolution; **scales** sets how many frequencies are evaluated.
Autocorrelation	How similar the signal is to itself shifted by each lag — reveals periodicity.
Cross-correlation	The same between two signals — reveals which one leads and by how much.

The **sampling frequency** is derived from the spacing of x (with a warning if the spacing is not uniform) or typed in. Welch estimators take a **segment length**, **overlap**, **window** and **detrend**;

spectra can be **one-sided** and in **decibels**; correlations take **max lags** and a **normalisation** (unbiased, biased, none). Two-signal estimators pair the **first** ticked series with each of the others.
Computed with: `numpy.fft.rfft`, `scipy.signal.welch`, `scipy.signal.csd`, `scipy.signal.coherence`, `scipy.signal.correlate`



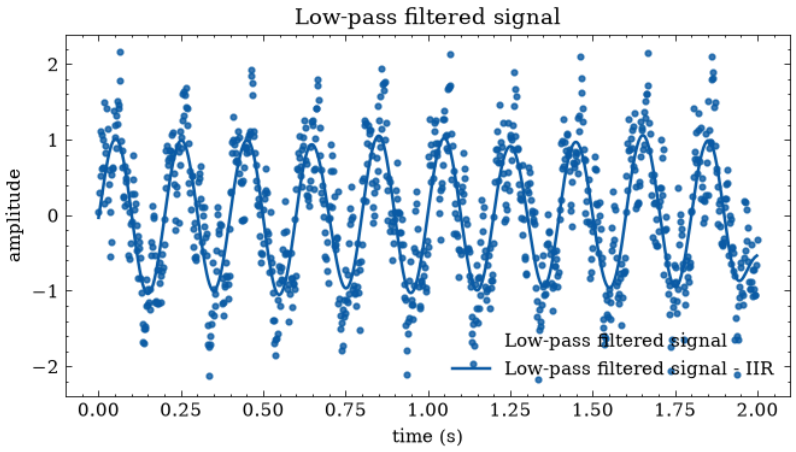
> **When to use it.** Vibration analysis, finding periodicities (seasonal cycles, mains hum), comparing two sensors, checking what a filter should remove.

Filtering — filter, detrend or demodulate

Changes the signal itself in the frequency domain — the complement of Spectral Analysis, which only measures it.

IIR filter	Family **Butterworth** (maximally flat), **Chebyshev I/II** or **Elliptic** (sharper roll-off with ripple: set **ripple** and **attenuation** in dB), or **Bessel** (best preserves wave shape). Response **lowpass**, **highpass**, **bandpass** or **bandstop**, with an **order** and one or two **cutoffs**. Applied forward and backward, so there is no phase shift.
FIR filter	Linear-phase filter with a number of **taps** and a **window**. No ringing on steps.
Analytic signal	Via the Hilbert transform: the **amplitude envelope**, **instantaneous phase** or **instantaneous frequency**. Used for AM demodulation and vibration envelopes.
Detrend	Removes a **linear** trend or a **constant** (the mean), on its own or before filtering.

The cutoffs are in the same units as the sampling frequency, which is taken from x as in Spectral Analysis.
Computed with: `scipy.signal.iirfilter`, `scipy.signal.firwin`, `scipy.signal.sosfiltfilt`, `scipy.signal.filtfilt`, `scipy.signal.hilbert`, `scipy.signal.detrend`



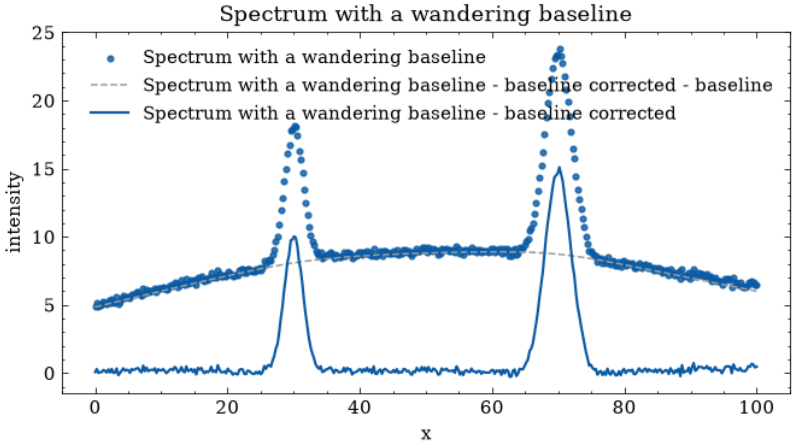
When to use it. Removing mains hum (bandstop at 50/60 Hz), separating slow drift from fast signal, extracting an envelope.

Baseline Correction — subtract a background

Estimates the slowly varying background under a spectrum or chromatogram and subtracts it. Peak heights and especially peak areas are wrong until this is done.

Asymmetric Least Squares (AsLS)	A smooth curve fitted to stay below the data: points above it cost little, points below cost a lot. **Lambda** sets its stiffness (larger bends less into narrow peaks), **p** its asymmetry (smaller pushes it further below; keep it well under 0.5), **iterations** the re-weighting passes. Copes with curved and overlapping baselines.
Rubber band	The lower convex hull of the data — a band stretched under it. No parameters, but it cannot follow a baseline that dips inward.

Draw the baseline adds the estimated background as its own series, so the correction can be checked by eye.
Computed with: `scipy.sparse.linalg.spsolve`



When to use it. Raman, IR, XRD and fluorescence spectra; chromatograms with drift — always before measuring peak areas.

Modeling

Fit — fit a model to data

Fits a mathematical model y = f(x; parameters) to a series by least squares and adds the fitted curve beside the data. It is the most complete operation in ChartLibre.

The model. The model tree lists the function library by family, with a search box:

Basic functions	Constant, linear, polynomials up to degree 10, reciprocal, logarithmic, power law, rational.
Growth and saturation	Exponential growth and decay, double exponential, saturation, Michaelis–Menten, Hill.
Sigmoidal curves	4- and 5-parameter logistic, Richards, Gompertz, error-function, tanh and arctan sigmoids, double-logistic pulse.
Peak functions	Gaussian, Lorentzian, pseudo-Voigt, Voigt, asymmetric Gaussian, Pearson VII.
Periodic functions	Sine, cosine, damped sine, linear chirp.
Statistical distributions	Normal, lognormal, Weibull, gamma, beta and Cauchy densities and distribution functions.
Reliability models	Exponential and Weibull survival, Weibull hazard.
Semiconductor and process	Arrhenius, inverse-temperature linear, dose-response power, Poisson yield, process window.
Adsorption and surface	Langmuir, Freundlich, Temkin isotherms.
NIST reference models	The models of the NIST StRD nonlinear regression benchmarks.
Surfaces	For z = f(x, y): plane, paraboloid, 2D Gaussian, saddle, ripple.
User functions / User surfaces	Your own, written with the Function Creator (see Section 12).

The **Multi-peak builder** creates a sum of **N** Gaussian, Lorentzian or pseudo-Voigt peaks plus an offset, optionally with one **shared width** (and shared η) for all peaks — for overlapping peaks in a spectrum.

The parameters. The table lists each parameter with its **initial value**, **lower and upper bounds** and a **fixed** flag. **Estimate** fills the initial values from the data — with the function's own estimator when it has one, or a random search of the parameter space when it does not. A good starting point matters: a least-squares fit started in the wrong place converges confidently to the wrong answer.

The algorithm.

Trust region (default)	Bounded least squares. Right for most fits.
Dogleg box	Bounded least squares, often better when a parame-

	ter ends up on a bound.
Levenberg-Marquardt	Classic unbounded least squares; bounds are ignored.
Nelder-Mead, **Powell**	Derivative-free local methods, for rough or noisy objective functions.
Differential evolution, **Dual annealing**, **Basin hopping**, **SHGO**, **Monte Carlo**	Global methods: search the whole box defined by the bounds, then finish with a local fit. Use them when the result depends on the starting point.

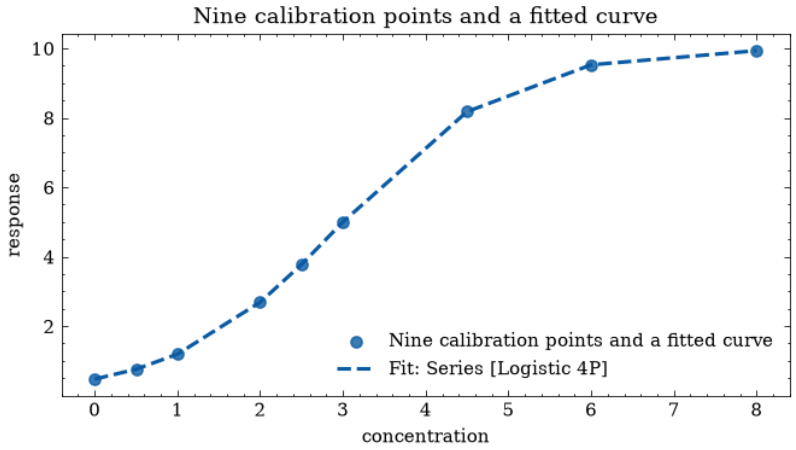
Loss sets how large residuals are weighted: plain least squares trusts every point; the robust losses (soft L1, Huber, Cauchy, arctan) limit how much one bad point can pull the fit. **Max evaluations** caps the work.

Weights says how much each point counts: **Equal**; **Relative** (each residual divided by its value, for data spanning orders of magnitude); or **Error bars**, each residual divided by the error in the series' **yerr** column, so a precise point counts more — the report then gives the chi-squared and the chi-squared per degree of freedom.

Global fit. Tick two or more series and the fit is global: one model fitted to all of them at once. Under the parameter table, **Shared** lists the parameters: a shared one has one value for every series (one row of the table), the others one value per series (a row each, named after its series). Fit, Preview and OK work as for one series; each series gets its own curve, report and output table, and the errors account for what the series share.

The report gives the fitted expression, each parameter with its standard error, t value, p value and 95% confidence interval, the parameter correlation matrix, and goodness of fit (R^2, RMSE, residual sum of squares, reduced chi-squared, AIC and BIC). Parameters can be copied, saved as CSV, or reused as the starting point of the next fit.

Extra output: 95% confidence band (where the true curve lies, given the parameter uncertainty), **95% prediction band** (where a new measurement would fall), a **residuals vs x** chart (structure in the residuals means the model is missing something) and a **measured vs fit** chart (a perfect fit lies on the diagonal). Fitted values and residuals are saved to the **output table**.
Computed with: `scipy.optimize.least_squares`, `scipy.optimize.minimize`, `scipy.optimize.differential_evolution`, `scipy.optimize.dual_annealing`, `scipy.optimize.basinhopping`, `scipy.optimize.shgo`



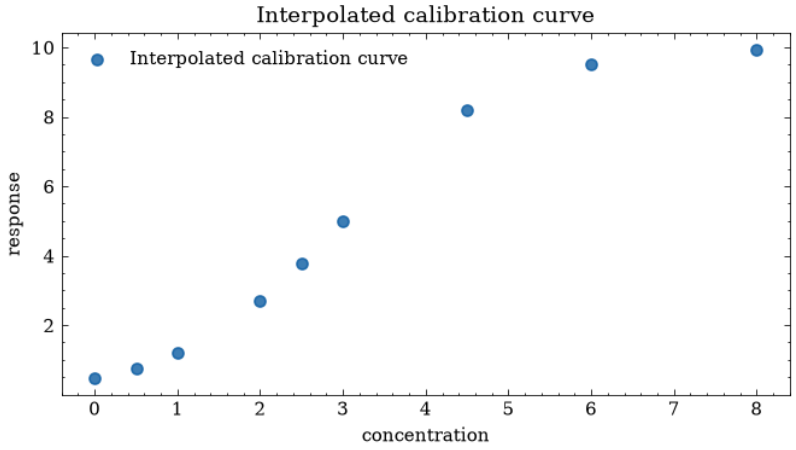
When to use it. Extracting physical parameters (a rate constant, a peak position, an EC50), calibration curves, comparing candidate models. ChartLibre's fit has been checked against the certified NIST StRD reference results.

Interpolation — fill gaps and resample

Evaluates a smooth or fitted curve through a series at new x values: to fill gaps, to resample onto a regular grid, or to extend a little beyond the data.

NumPy interp, **Linear**	Straight lines between points.
SciPy CubicSpline	A smooth cubic through every point, with a choice of **boundary condition**.
SciPy PCHIP	Monotone cubic: never overshoots between points — the safe choice for data that must not wiggle.
SciPy Akima	A cubic that is less affected by outliers than a natural spline.
SciPy spline family	UnivariateSpline / B-spline of degree **k** with smoothing **s**.
Polynomial	A least-squares polynomial of a given **degree**.
Exponential, **Logarithmic**, **Power**, **Gaussian**, **Sigmoid**	Simple fitted models, with optional starting parameters.

X spacing chooses where the new values are computed: a number of **points** spaced linearly, logarithmically, geometrically or at Chebyshev nodes over the data range (optionally **extended** by a percentage on both sides, or over an explicit **X range**); a fixed integer **step**; the **original data x** values; or a custom list of x values. **Allow extrapolation** permits values outside the data's x range.
Computed with: `numpy.interp`, `scipy.interpolate.CubicSpline`, `scipy.interpolate.PchipInterpolator`, `scipy.interpolate.Akima1DInterpolator`, `scipy.interpolate.make_interp_spline`, `scipy.interpolate.UnivariateSpline`, `scipy.optimize.curve_fit`



> **When to use it.** Putting two series on the same x grid before comparing or subtracting them; filling missing readings; producing a smooth curve for display.

Function — plot a function

Evaluates a function from the same library as Fit over a range and draws it, with no data and nothing fitted. Choose the function, type its parameter values, the **start** and **stop** of the range, the number of **points** and the **spacing** (linear, or logarithmic for log axes). For a surface function, the range is a grid in x and y.

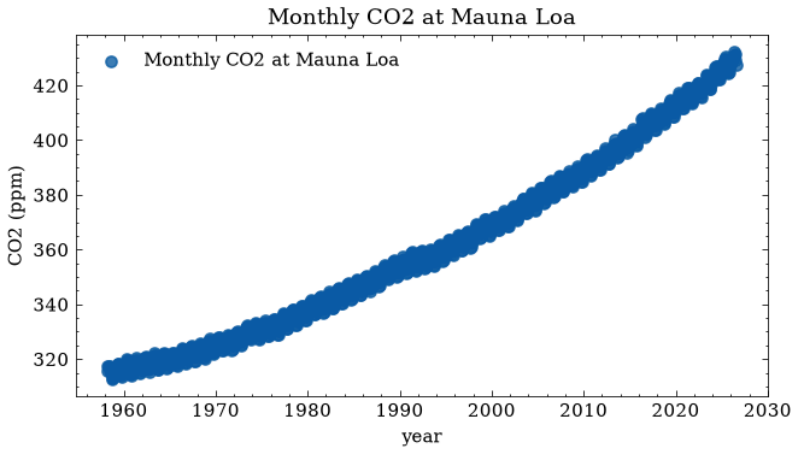
> **When to use it.** Seeing the shape of a model before choosing it, drawing a theoretical or reference curve beside measurements, checking starting values before a fit, generating synthetic test data.

Regression — robust and machine-learning regression

Fits a curve through a series with models that do not assume an algebraic formula, and evaluates it on a dense grid over the data's x range.

RANSAC	Robust straight line: fits random subsets (**max trials**) and keeps the one most points agree with. Ignores gross outliers completely.
Huber	Robust straight line that down-weights large residuals; **epsilon** near 1 is more robust, larger behaves like ordinary least squares; **alpha** regularizes.
Isotonic	The best-fitting curve that only rises (or only falls, or **auto**). No shape assumed beyond monotonicity.
Random Forest	Average of many regression trees (**estimators**, **max depth**). Flexible, step-like.
Gradient Boosting	Trees added one after another, each correcting the last (**learning rate**). Flexible and often accurate.

Computed with: `sklearn.linear_model.HuberRegressor`, `sklearn.linear_model.RANSACRegressor`, `sklearn.isotonic.IsotonicRegression`, `sklearn.ensemble.RandomForestRegressor`, `sklearn.ensemble.GradientBoostingRegressor`



> **When to use it.** A trend through outlier-heavy data (RANSAC, Huber), a calibration that must be monotone (Isotonic), a flexible non-parametric trend when no formula is known.

GP Regression — a fit with its own uncertainty

Gaussian process regression: a smooth curve through the data **and** a $\pm 2\sigma$ band saying how certain the curve is at each x — narrow where the data are dense, wide in gaps and beyond the ends. The result is three series: the mean and the two band edges.

RBF	Very smooth curves.
Matern ($\nu = 1.5$) / ($\nu = 2.5$)	Rougher curves (once / twice differentiable) — often more realistic for physical data.
Rational Quadratic	A mix of length scales — for data with both slow and fast variation.

Length scale (how quickly the curve may vary with x) and **noise level** are starting guesses; both are optimized during the fit, and the fitted kernel and log marginal likelihood are reported.
Computed with: `sklearn.gaussian_process.GaussianProcessRegressor`, `sklearn.gaussian_process.kernels.RBF`, `sklearn.gaussian_process.kernels.Matern`, `sklearn.gaussian_process.kernels.RationalQuadratic`, `sklearn.gaussian_process.kernels.WhiteKernel`

> **When to use it.** Interpolating sparse or expensive measurements with an honest uncertainty; deciding where the next measurement would be most informative.

Analysis

Where a data operation works on the series of a chart, an **analysis** works on the columns of a table: you say which column is the response and which ones might explain it, and it fits a model of the whole table. The **Analysis** page of the navigation rail lists them; each opens on the table selected in the Tables page. Each names, under **Computed with**, the library functions that fit it, linked to their documentation.

How every analysis window works

- **Select columns**, on the left: the table, its columns with their modelling type — ▲ continuous, ● nominal, ◐ ordinal — and the **Type** to change it. Under them, **Select role variables**: drag or click columns into the roles the analysis asks for (Y, X, groups, …). A role that is short of columns, or a column of the wrong type, is said before anything runs.
- **Parameters**, in the middle, where the analysis has any.
- **Results**, on the right: the report.
- **Preview** computes and shows the report; **OK** writes the results into the project — each row's prediction beside its data, the report's tables as tables — and draws a figure with the report in its notes. The calculation runs in the background, with **Stop**; OK is undoable.

The analyses

Distribution

One report per Y column: the quantile table (0% to 100%, at the (n + 1)p position), the moments, the normality tests and, with a lower and/or an upper specification limit, process capability — Cp and Cpk from the within sigma (the average moving range / 1.128), Pp and Ppk from the overall standard deviation, Cpm with a target, and the share out of specification, observed and expected. OK draws the histogram with its normal curve and the limits, beside the normal quantile plot.
Computed with: `numpy.quantile`, `scipy.stats.shapiro`, `scipy.stats.norm`

Fit Y by X

One Y against each X, the analysis chosen by their types: **Bivariate** (continuous by continuous: the least-squares line, correlations), **Oneway** (continuous by groups: means, ANOVA, Welch and Kruskal-Wallis, and every pair compared — Tukey HSD, Games-Howell for unequal variances, Dunn on the ranks with Holm's adjustment), **Contingency** (groups by groups: counts, row shares, chi-squared, Fisher) or **Logistic** (groups by continuous: the probability of each level along X, odds ratios, AUC). An ordinal column is a measure as Y and groups as X.
Computed with: `statsmodels.regression.linear_model.OLS`, `scipy.stats.tukey_hsd`, `scipy.stats.studentized_range`, `scipy.stats.chi2_contingency`, `statsmodels.discrete.discrete_model.Logit`, `statsmodels.discrete.discrete_model.MNLogit`

Fit Model

A linear model of one or more responses on the effects you build from the columns (main effects, factorial, response surface). Its **personality** says how: **Standard Least Squares**, **Stepwise** (backward or forward), **Generalized Linear Model** (normal, binomial, Poisson, gamma), **Nominal** or **Ordinal Logistic** for a categorical Y, and **Generalized**

Regression — Lasso, Ridge or Elastic Net, the penalty chosen by cross-validation, for many factors or factors that move together. Effect tests, estimates, the prediction profile; for a logistic fit the ROC curve, its AUC and the confusion matrix.
Computed with: `statsmodels.formula.api.ols`, `statsmodels.formula.api.glm`, `statsmodels.discrete.discrete_model.Logit`, `statsmodels.discrete.discrete_model.MNLogit`, `statsmodels.miscmodels.ordinal_model.OrderedModel`, `sklearn.linear_model.LassoCV`, `sklearn.linear_model.RidgeCV`, `sklearn.linear_model.ElasticNetCV`, `sklearn.metrics.roc_curve`

Partition — A decision tree of one response on many factors, as JMP's: **Split** and **Prune** one step at a time, or **Automatic** on validation rows; or a **Bootstrap Forest** or a **Boosted Tree**. Leaves and their rules, column contributions, and for any of the three the **permutation importance** (how much the fit drops when a column is shuffled) and **partial dependence** (the predicted response as one factor varies); for a nominal Y the ROC curve.
Computed with: `sklearn.tree.DecisionTreeRegressor`, `sklearn.tree.DecisionTreeClassifier`, `sklearn.ensemble.RandomForestRegressor`, `sklearn.ensemble.GradientBoostingRegressor`, `sklearn.metrics.roc_auc_score`

Discriminant — Which group a row belongs to, from continuous measures: linear or quadratic, with Wilks' lambda, the canonical analysis and plot, and each row's predicted group and probabilities.
Computed with: `scipy.linalg.eigh`, `scipy.linalg.cholesky`, `scipy.stats.f`

Partial Least Squares — A response from many X that move together — the wavelengths of a spectrum — through a few components, their number chosen by cross-validation; the variables' importance (VIP), loadings, coefficients and scores.
Computed with: `sklearn.cross_decomposition.PLSRegression`, `sklearn.model_selection.KFold`

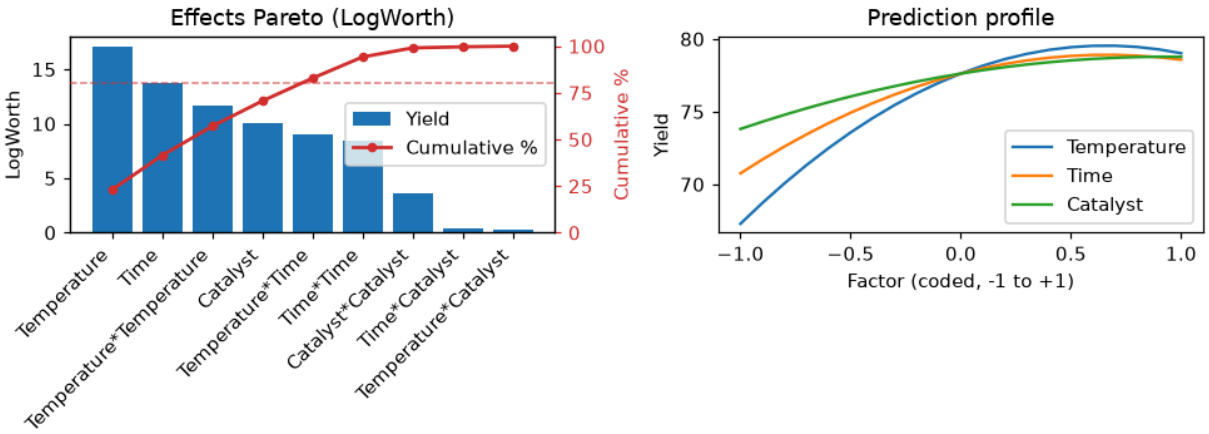


Figure 5: Fit Model on a Box-Behnken design: the effects ranked by LogWorth, and the prediction profile.

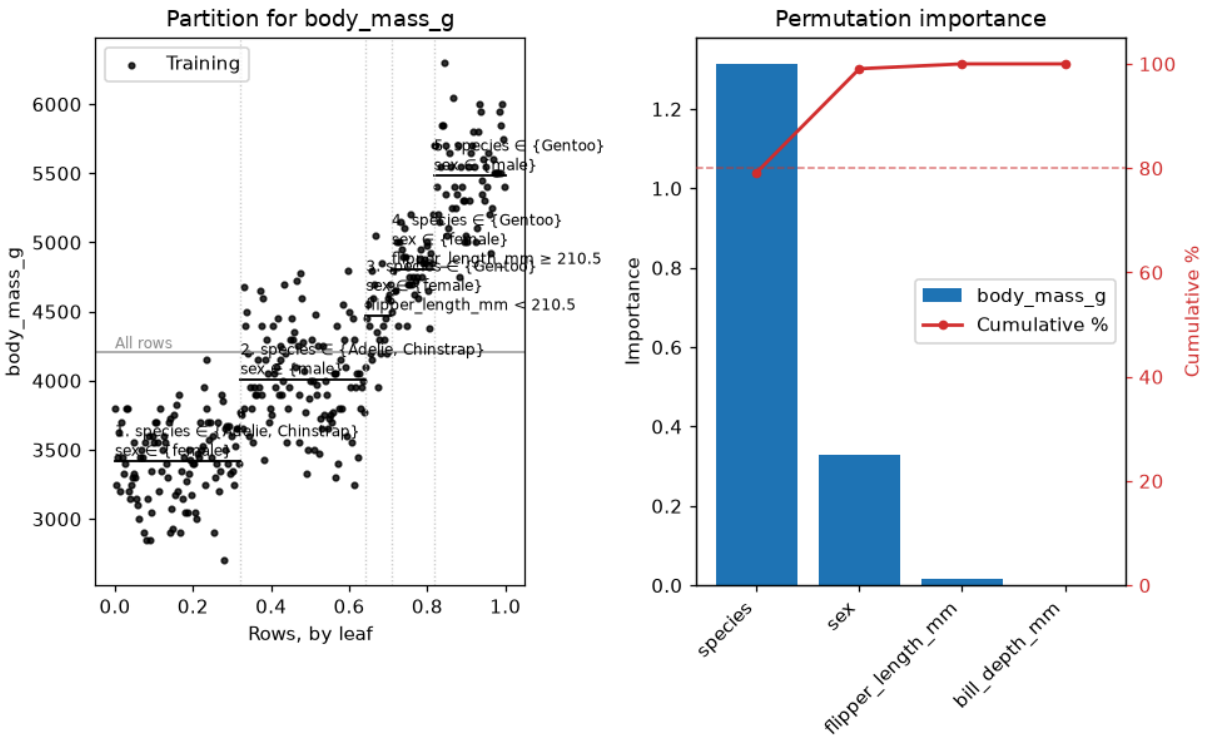


Figure 6: Partition of the penguins' body mass: the partition graph, each leaf as wide as its rows, and the permutation importance.

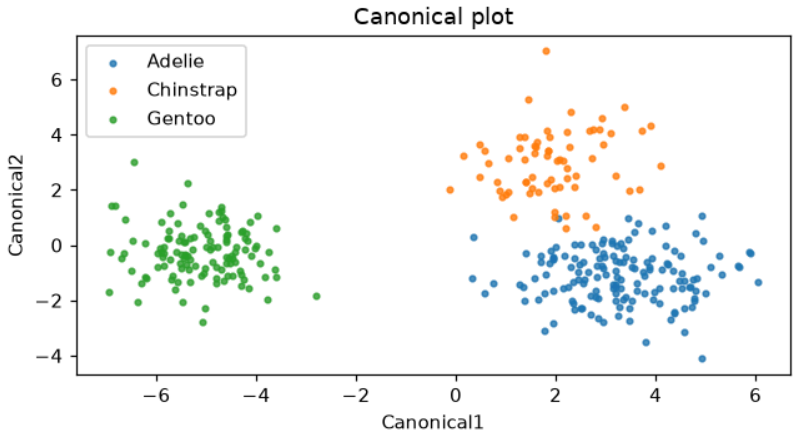


Figure 7: Discriminant analysis of the three penguin species from four measures: the canonical plot.

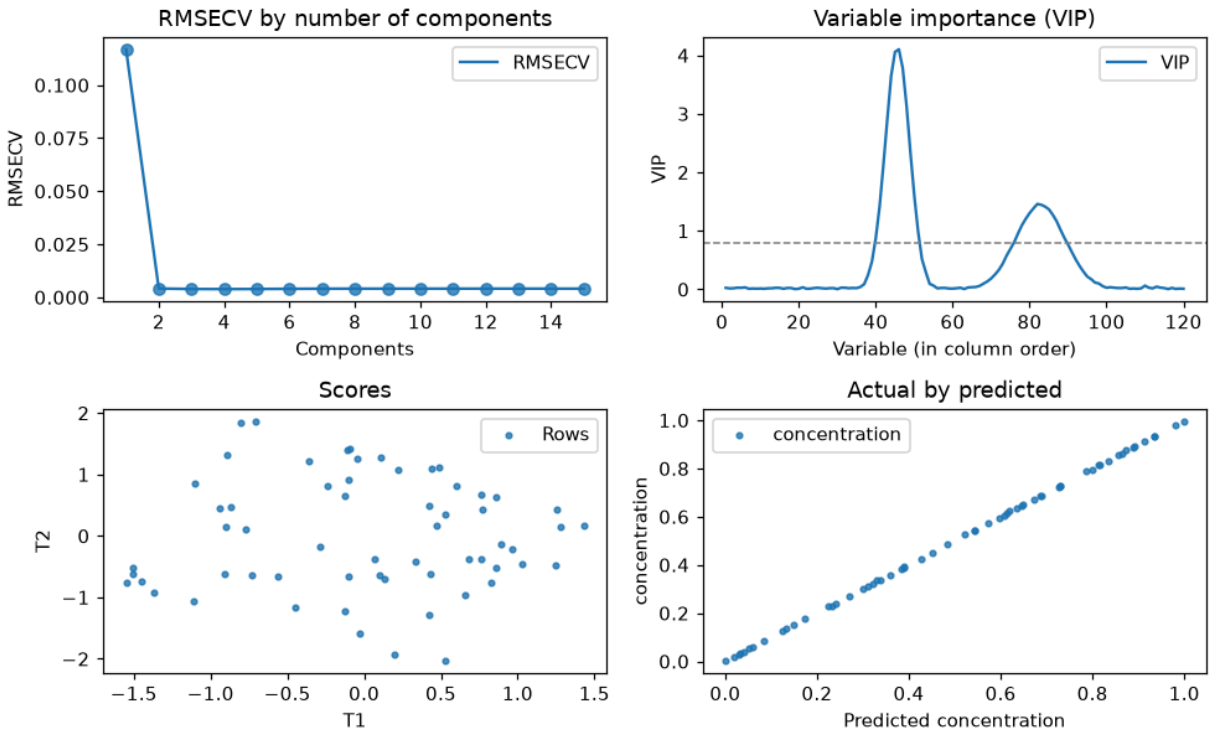


Figure 8: Partial Least Squares on a set of spectra: RMSECV by number of components, VIP along the wavelengths, scores, actual by predicted.

Guided analysis

Guided analysis starts from what you want to find out rather than from the name of a function. It follows the table selected in the Tables page and says what it holds — how many numeric columns, categories and dates.

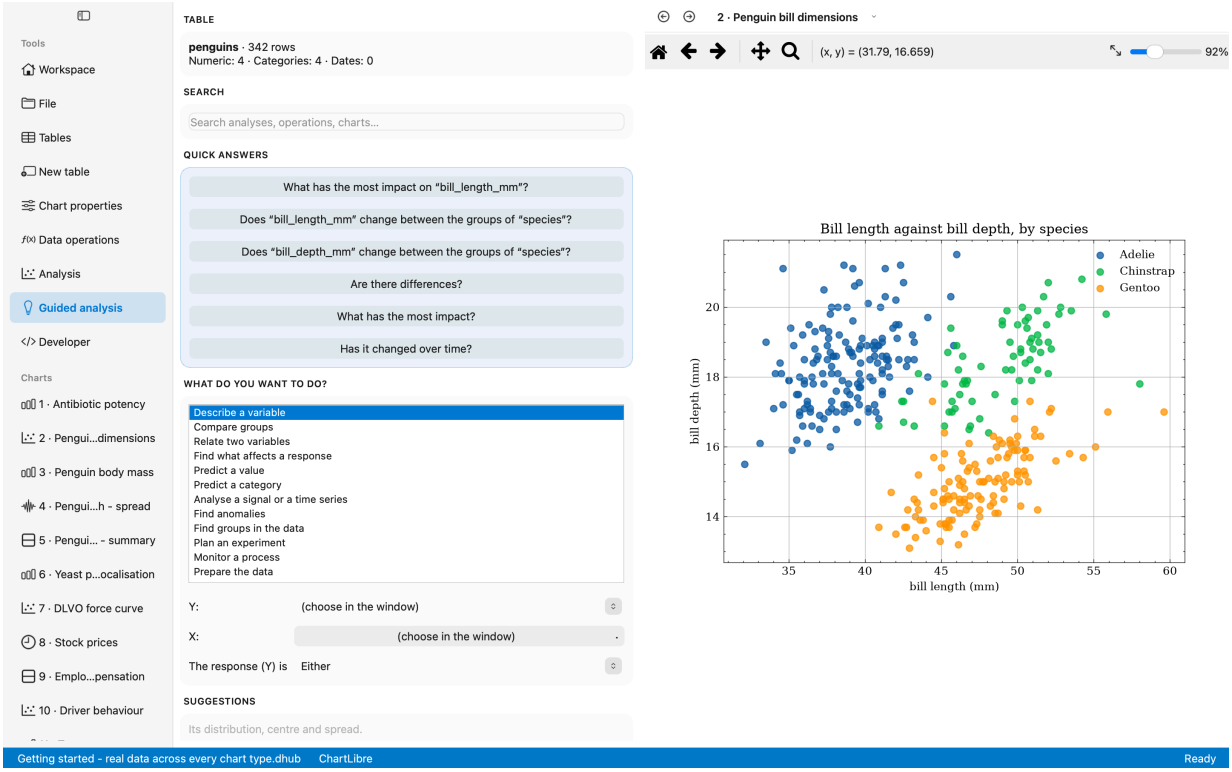


Figure 9: Guided analysis on the Palmer penguins table: what it holds, its own questions, the goals.

- **Search** finds any function by what it does, in English or Italian, and by the names other programs give it ("anova", "crosstabs", "control chart"). The same search opens from anywhere with **Cmd/Ctrl+K**.
- **Quick answers** ask a question in everyday words and answer it in words, the right test chosen for you:
  - **Are there differences?** — is a measure different from one group to another (t-test or ANOVA, Mann-Whitney or Kruskal-Wallis, chi-squared or Fisher, as the data call for);
  - **What has the most impact?** — the factors ranked by how much they matter for a result (a random forest on rows it had not seen, with a linear model's p-values);
  - **Has it changed over time?** — a steady trend (Mann-Kendall, Sen's slope) or a sudden change (Pettitt), whichever the values show.

  Computed with: `scipy.stats.ttest_ind`, `scipy.stats.mannwhitneyu`, `scipy.stats.kruskal`, `scipy.stats.tukey_hsd`, `scipy.stats.chi2_contingency`, `scipy.stats.fisher_exact`, `sklearn.ensemble.RandomForestRegressor`, `statsmodels.stats.anova.anova_lm`, `scipy.stats.theilslopes`

  Above them, questions asked of the selected table itself — "Does "weight" change between the groups of "line"?" — open the answer with its columns already chosen.
- **What do you want to do?** lists goals — describe, compare, relate, explain, predict, monitor… With **Y** and **X** chosen from the table, **Open** starts the suggested analysis or chart with them cast. Suggestions the table cannot feed come last, greyed, saying what they lack.

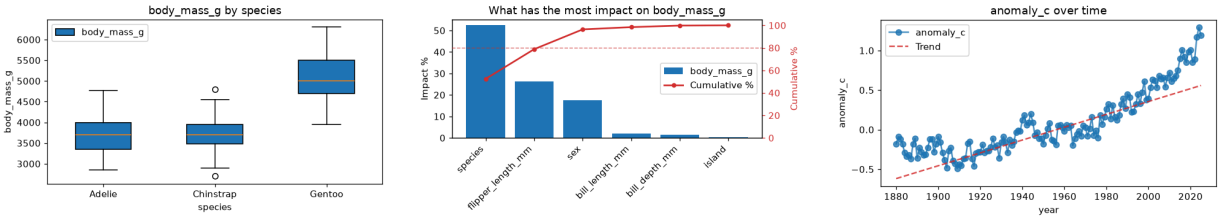
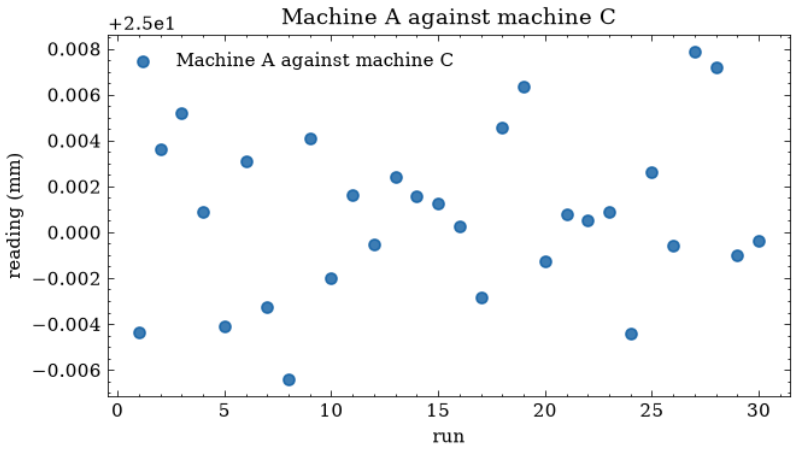


Figure 10: The three quick answers on real data: penguin body mass by species, what has the most impact on it, and NASA's global temperature anomaly since 1880.

> The demo **Quick answers – three questions asked of real data** has each quick answer already asked, its answer in the chart's notes.

Statistics

Statistics (in the **Statistics** section of Data operations) answers the questions people ask of their data before or instead of drawing a conclusion: what is the centre and spread, is it normal, do these groups differ, are these variables associated. It reads the series ticked in its list and writes a formatted report in the Results pane; it changes no data and writes no tables.



Using the window

1. Tick the series to analyse. One series is enough for descriptive, one-sample and normality statistics; tests that compare need two or more.
2. Choose the **Model** — the family of statistics to report, or **All statistics** for every family that applies.
3. Set the options the model uses (reference value, alternative hypothesis, …).
4. **Preview** computes the report. **Copy** copies it, with its formatting, for a lab notebook or an e-mail.
5. To also draw a chart, tick **Add a chart** and pick one. **Match the model** chooses the chart that shows what the model measures; **Preview** draws it on a new axis of the current figure, **OK** keeps it, and **Close** removes it again.

Reference value	The value one-sample tests compare the centre with (H_0: mean = reference).
Alternative	**Two-sided** (different), **greater** or **less** — the direction of the alternative hypothesis, for the tests that support it.
Trimmed mean	The percentage cut from each end for the trimmed mean.
Anderson method	How the Anderson–Darling p-value is obtained: interpolated from tables, or by Monte Carlo (with a number of **resamples** and a **batch** size).
Rank by	For distribution fitting: AIC (default), BIC or the KS statistic.
Try every SciPy distribution	Fit all 100 continuous distributions instead of the 15 common ones. Slow, and many of the extra candidates fit no real measurement.

How to read a p-value: it is the probability of seeing a difference at least this large **if there were no real difference**. A small p-value (conventionally below 0.05) says the data are hard to explain by chance alone. It does not say how large or important the difference is — read the effect sizes (Cohen's d, η^2, Cramér's V) for that — and a large p-value does not prove there is no difference, only that these data do not show one.

Models on values

These models read the numeric values of each ticked series (its `y`, or its `value` for distribution charts).

Descriptive statistics

For each series: count, mean, trimmed mean, geometric and harmonic means (for positive data), median, mode and how often it occurs, standard deviation, variance, standard error of the mean, minimum, quartiles, maximum, range, interquartile range, median absolute deviation, skewness, kurtosis, entropy, and the 95% confidence interval of the mean.

Charts: Histogram, ECDF.
Computed with: `scipy.stats.sem`, `scipy.stats.skew`, `scipy.stats.kurtosis`, `scipy.stats.iqr`, `scipy.stats.median_abs_deviation`, `scipy.stats.trim_mean`, `scipy.stats.gmean`, `scipy.stats.hmean`, `scipy.stats.mode`

One-sample tests

Is the centre of the series equal to the reference value?

One-sample t-test	Compares the mean; assumes roughly normal data (or a large sample).
Wilcoxon signed-rank	Compares the median without assuming normality; assumes a symmetric distribution.
Sign test	Counts values above and below the reference. Assumes almost nothing; least powerful.

Charts: Histogram, ECDF.
Computed with: `scipy.stats.ttest_1samp`, `scipy.stats.wilcoxon`, `scipy.stats.binomtest`

Normality / shape tests

Does the series look like a sample from a normal distribution?

Shapiro-Wilk	The most powerful general test for small and medium samples.
D'Agostino-Pearson	Combines skewness and kurtosis; needs at least about 20 values.
Skewness test / Kurtosis test	Each part separately: is the distribution lopsided, are its tails heavier or lighter than normal?
Anderson-Darling	Gives extra weight to the tails.
Jarque-Bera	Skewness and kurtosis for large samples.
Kolmogorov-Smirnov vs normal	The largest gap between the sample's ECDF and the fitted normal CDF.

With thousands of values, every test rejects normality for tiny departures that do not matter in practice; with ten values, none of them can detect much. Look at the Q-Q plot as well.

Charts: Histogram, ECDF, **Q-Q Plot**, **P-P Plot**.

Computed with: `scipy.stats.shapiro`, `scipy.stats.anderson`, `scipy.stats.normaltest`, `scipy.stats.jarque_bera`, `scipy.stats.kstest`, `scipy.stats.skewtest`, `scipy.stats.kurtosistest`

Paired-sample tests

For two series measured on the **same units** — before and after on the same parts, two instruments on the same samples. Values are paired by row (or by matching x). For every pair of ticked series:

Paired t-test	Is the mean difference zero?
Wilcoxon signed-rank paired test	The same without assuming normal differences.
Sign test	Counts which of the two is larger in each pair.

Chart: Scatter Plot of one series against the other.
Computed with: `scipy.stats.ttest_rel`, `scipy.stats.wilcoxon`

Independent-sample tests

For two groups of **different units** — parts from two machines, patients in two arms. For every pair of ticked series:

Welch t-test	The default reading: compares means without assuming equal variances.
Student t-test	Assumes equal variances.
Mann-Whitney U	Compares the distributions by rank; no normality needed.
Levene's test and **Brown-Forsythe**	Are the variances equal? (Brown-Forsythe uses the median, and is less affected by non-normal data.)
Difference of means and **Cohen's d**	The size of the difference in standard deviations (about 0.2 small, 0.5 medium, 0.8 large).

Chart: Box Plot.
Computed with: `scipy.stats.ttest_ind`, `scipy.stats.mannwhitneyu`, `scipy.stats.levene`

Group comparison (ANOVA)

All ticked series together, as groups:

One-way ANOVA	Do the group means differ? With η^2, the share of variance explained by the groups.
Welch's ANOVA	The same without assuming equal variances — read this one when Levene's test says the variances differ.
Kruskal-Wallis	The rank-based version, with ε^2 as effect size.
Levene's test	Equal variances across all groups.
Tukey's HSD	Which pairs differ: every pair with its difference, adjusted p-value and 95% interval, so that testing many pairs does not inflate false positives.

Chart: Box Plot.
Computed with: `scipy.stats.f_oneway`, `scipy.stats.kruskal`, `scipy.stats.tukey_hsd`

Correlation / association

For every pair of ticked series, paired by row: **Pearson** r (linear association), **Spearman** ρ (monotone association, by rank), **Kendall** τ (rank agreement, robust for small samples) and the least-squares **regression slope**. Correlation measures association, not cause.

Chart: Scatter Plot.

Computed with: `scipy.stats.pearsonr`, `scipy.stats.spearmanr`, `scipy.stats.kendalltau`, `scipy.stats.linregress`

Distribution fit

Fits candidate distributions (normal, lognormal, gamma, Weibull, exponential, …) to each series by maximum likelihood and ranks them by AIC, BIC or the KS statistic, with their fitted parameters. AIC is the default because the KS statistic alone always favours distributions with more parameters.

Chart: Histogram with the chosen fitted distribution — **best fit** follows whichever candidate ranks first.
Computed with: `scipy.stats.rv_continuous.fit`, `scipy.stats.kstest`

Models on tables

The last three models read several **columns** of each series at once rather than its values alone. They find the columns by the series' roles, so a series drawn as the matching chart already has them: a Mosaic Plot series for a contingency table, an Interaction Plot series for a two-way ANOVA, a Kaplan-Meier series for survival. A series without the columns a model needs is listed as **Not analysed**, with the columns it is missing.

Contingency table (chi-squared, Fisher)

Are two categorical variables associated?
Required: a row variable (`x` or `group`), a column variable (`y` or `trace`) **Optional:** `weight` — a count per row, for a table already counted

The report shows the table of counts with its totals, then:

Pearson chi-squared	The standard test of independence, with its degrees of freedom. When some expected counts are below 5, the report says so and recommends Fisher's test.
Chi-squared, Yates' correction	For 2 × 2 tables: a more conservative version.
Likelihood-ratio G	An alternative to chi-squared, with the same reading.
Fisher exact	Exact for 2 × 2 tables (with the odds ratio); for larger tables, a Monte Carlo estimate with a fixed seed, so the same data always give the same p-value.
Cramér's V	The strength of the association, from 0 (none) to 1 (complete).

Chart: Mosaic Plot — shade by Pearson residual to see which cells drive the result.
Computed with: `scipy.stats.chi2_contingency`, `scipy.stats.fisher_exact`

Two-way ANOVA

Do two factors affect a response, and does the effect of one depend on the other?
Required: the first factor (`x`), the second factor (`trace` or `group`), the response (`y` or `value`)

The ANOVA table has one row for each factor, one for their **interaction** and one for the residual, with sums of squares, degrees of freedom, F, p-value and partial η^2. Sums of squares are **Type II**, so the result does not depend on which factor is listed first; for a balanced design (the same number of values in every cell) they equal the classic sequential ones. The cell means and counts are listed too. Every combination of levels must have at least one value.

Read the interaction first: if it is significant, the effect of one factor depends on the level of the other, and the main effects alone do not tell the whole story.

Chart: Interaction Plot.
Computed with: `statsmodels.formula.api.ols`, `statsmodels.stats.anova.anova_lm`

Survival (Kaplan-Meier, log-rank)

How long until an event, when some cases have not had it yet?
Required: `time` (or `x`), `event` (1 = happened, 0 = censored) **Optional:** `group`

For each group: number of cases, events and censored cases, and the **median survival time** with its 95% confidence interval. The median is **not reached** when the curve never falls to 50%, and the upper end of the interval is ∞ when it never closes. With two or more groups, the **log-rank test** says whether the curves differ.

Chart: Kaplan-Meier.

Reliability of the results

The statistics are computed by SciPy and statsmodels and are checked by ChartLibre's test suite against published reference values — among them SciPy's documented ANOVA example, Agresti's party-identification table for the contingency tests, Fisher's tea-tasting experiment for the exact test, the Freireich 6-MP leukaemia trial for Kaplan–Meier and the log-rank test, and the certified NIST StRD data sets for curve fitting.

Saving, reports and templates

Saving a figure

Save figure (chart menu, or Ctrl+Shift+S / Cmd+Shift+S) saves the active figure as an image: **PNG** or **JPEG** (pixels), or **SVG** or **PDF** (vector — sharp at any size, the format journals ask for). The dialog opens on the default format chosen in Settings. The resolution is the figure's own **DPI**, set in Figure options — 200 unless set otherwise, sharp in print; 300 is common for journals — and the picture is trimmed to its content. The DPI is an export resolution only: on screen every chart is drawn at the screen's own density, whatever its DPI.

For a figure that must match a journal's column width, set the figure **size** in Figure options before saving; fonts keep their point size, so the text in the saved file is the size it will be printed at.

Copy puts the figure on the clipboard, to paste into a slide or a document.

Reports

A **report** is a document for people who read results rather than queries: each figure drawn again, with its notes (the reports of the operations applied to it), and optionally the project's tables and history. The SQL behind the charts is deliberately left out.

Project report	**File** page → **Export** → **Project report**: every figure of the project, then — if ticked — the tables with their size and notes, and the operation history.
Export report...	Chart menu: the same report for **this figure alone**, titled after it.

The options are the same for both:

Format	**HTML** — one self-contained file with the pictures inside it, which opens in any browser and can be e-mailed as it is. **PDF** — the same page laid out on A4.
Pictures	**PNG**, or **SVG** for pictures that stay sharp at any zoom (HTML only; the PDF always uses PNG).
Resolution	The DPI the figures are drawn at (72–600).
Contents	For the project report: include the tables, and the operation history.

Export... asks where to save the file; the window shows its progress while the figures are drawn, and closes when the report is written. A figure that cannot be drawn is reported in the log and skipped, so one broken chart does not stop the report.

Templates

A **template** keeps the **look** of a figure and none of its content: the figure's style sheet and layout, each axis' options except its title, labels, limits and annotations, and each series' colour, marker and line style. Applied to another figure, it changes how that figure looks and nothing it says.

Save this look as a template...	Chart menu → **Templates**. Give it a name.
Apply a template...	Gives the active figure the look of a saved template, axis by axis and series by series, in order. Undo takes it back.
Delete a template...	Forgets a template.

Templates are kept in the `user/templates` folder of the ChartLibre installation, not in a project, so one template serves every project.

> **When to use it.** Making every figure of a paper or a thesis look the same — the same fonts, grid, tick style and marker for each series — without setting it by hand on each one.

History

Every data operation applied with **OK** is recorded in the project: when it ran, which operation and model, on which series, with which parameters, which tables it wrote, and its report. The record lives in the `.dhub` file, so it travels with the project.

- **Project history...** (on the **Developer** page) lists every operation in the project, oldest first.
- **History...** (right-click a table on the Tables page) lists the operations that read from or wrote to that table.

The history makes an analysis reproducible: anyone opening the project can see exactly how each result table and curve was obtained. It is also included in the project report.

Settings, styles and the log

Settings

Settings (the ChartLibre menu on macOS, the rail on Windows and Linux) holds the preferences of this installation — they belong to the computer, not to a project:

App style	The look of the interface. The platform's native style, plus any Qt styles installed on the system (Breeze, Fusion, …) and **Browse...** for a Qt stylesheet (`.qss`) of your own.
Language	**Auto** follows the operating system's language, falling back to English. Each language is listed under its own name: English, **Italiano** and **Français** are complete; **Español** and **Deutsch** are only started and still read mostly English.
Default export format	The format **Save figure** proposes: PNG, JPEG, SVG or PDF.
SQL guard	**Refuse SQL that would change the database** (on by default, shown in red): `UPDATE`, `DELETE`, `DROP` and the like are refused when a query is saved or typed and when a project is opened. Switch it off only if you know why you need to.
DeepL API key	Used by **Edit Localization** to translate the interface with DeepL. The key is kept on this computer only. Without one, Edit Localization translates with Google, MyMemory or Argos Translate. Free-plan keys end in `:fx`.

The style and the language are applied the next time ChartLibre starts, as the window says: changing them in a running application would leave half the window in the old style or language.

Chart styles

A **Matplotlib style** sets the base look of every chart in a figure: colours, fonts, line widths, grid, tick direction. Each figure picks its style in **Figure options → Style**: Matplotlib's built-in styles, the **SciencePlots** styles for journal figures, the **Apps** styles that look like JMP, Minitab and SPSS charts, or a `.mplstyle` file of your own. **Apply to all figures** gives every figure of the project the same style (undoable). A figure with no style of its own uses the **Default chart style** chosen in **Settings**.

The **Edit** button beside the style opens the **Matplotlib Style Editor**:

- a **style stack** to combine several styles (each overriding the previous one);
- a **parameter editor** to add, change or reset individual settings (rcParams), with an editor suited to each — colour picker, check box, number, line style, marker, colormap, font list;
- a live **preview** of the current figure with the edited style;
- **Save** writes the result as a `.mplstyle` file, which can then be chosen for any figure.

The log

The **Log viewer** (on the **Developer** page) shows what ChartLibre has done and what went wrong: projects opened, imports, operations, rows a chart could not use, errors. It is the first place to look when a chart is emptier than expected, and the thing to attach to a bug report.

Clear log (in red) empties the log after asking for confirmation.

Credits

Credits (in the ChartLibre menu on macOS, the **Menu** button elsewhere) names the application and its version, and the open-source libraries it is built on — Qt and PySide6, Matplotlib, NumPy, pandas, SciPy, statsmodels, scikit-learn and others, each with its installed version and licence. It also credits the work of others that ships with ChartLibre or that it draws on — the SciencePlots styles by John D. Garrett, the system icon sets — and the sources of the demo data: the Palmer penguins, NOAA's Mauna Loa CO_2 record, WDC-SILSO's sunspot number, NASA's GISTEMP, the USGS earthquake catalogue, the UCI Yeast dataset, Anscombe's quartet, Will Burtin's antibiotic data, the NIST StRD reference data sets and others. Each name links to its source.

Advanced: extending ChartLibre

ChartLibre discovers chart types and data operations the same way: by scanning a folder for Python classes that directly subclass a known base class, at import time — no registration list to edit and keep in sync. Dropping a well-formed file into the right folder is enough for it to appear in the application.

The **Developer** page offers a form for each of the sections below, so writing the file by hand is the fallback, not the only way:

Renderer Helper a short form (name, category, description, link, required/optional roles) that scaffolds a new chart-type file.

Data Operation Builder the same, for a new data-operation dialog - the scaffolded file already runs, passing each selected series through unchanged, ready to have the real computation dropped in.

Function Creator the same, for a new fit function - the scaffolded `execute(x, p)` already evaluates a real polynomial in the declared parameters.

Edit Localization a table editor for a language's translation catalogue, filterable to untranslated strings only, with a button to create a new language from scratch and an optional machine-translation pass to draft from — with Google, MyMemory, DeepL when a DeepL key is set in Settings, or Argos Translate, which runs on this computer once `ctranslate2` and `sentencepiece` are installed (`pip install ctranslate2 sentencepiece`) and downloads each language's model (85–150 MB) the first time it is used.

Every scaffolded file from the first three goes under `user/` at the project root (`user/charts/`, `user/data_operations/`, `user/functions/`) rather than into `app/`'s own shipped source - kept separate on purpose, so a packaged build's own files are never touched and a person's own additions are never confused with what shipped.

Writing a custom chart renderer

A chart type lives entirely in one file under `app/charts/`, as a class that subclasses `BaseAxisRenderer` (from `app.charts.base`) **directly** — the discovery scanner in `app/scanners/axis_renderer_scanner.py` matches that exact base-class name in the source, even when the class also inherits behaviour from another renderer.

The class is described entirely through its own attributes:

`Name`	Display name, and the `chart_type` value stored in the database. Renaming it orphans existing axes unless an alias is added to `CHART_TYPE_ALIASES` in the scanner.
`Category`	Which family of plot this is, following Matplotlib's own taxonomy (e.g. "Pairwise data", "Statistical distributions").
`Description` / `Link`	Shown in the chart picker and in the axis properties panel.
`RequiredRoles` / `OptionalRoles`	Column names the series' SQL query must (or may) produce — see Section 6.
`Kwargs`	Matplotlib keyword arguments forwarded verbatim to the plot call, as `{name: metadata}`. The metadata (`default`, `type`, `min`/

	`max`, `kind`, `group`, `description`) drives the generated editor UI automatically.
`Options`	Settings the renderer consumes itself and never forwards to Matplotlib — same metadata shape, opposite destination.
`MaxSeries`	How many series this renderer can draw on one axis, or `None` for any number.

The class then implements one method:

```
def render_axis(self, ax, series: list[SeriesData], options: dict | None = None) -> None:
    ...
```

which receives one `SeriesData` per series — already queried, with its columns in `sd.df` — and draws them onto `ax` (a Matplotlib `Axes`).

`sd.df` is a `SeriesFrame` (`app.data.series_frame`): columns of NumPy arrays read straight from SQLite, read the way a DataFrame is (`sd.df["x"]`, `"x" in sd.df.columns`, `sd.df.loc[mask, "color"]`), with `sd.df.to_pandas()` available for the rare operation that genuinely needs a DataFrame. The developer guide's §4.1 covers it.

Practical steps:

1. Create `app/charts/my_chart.py`.
2. Import `BaseAxisRenderer` and, if useful, `SeriesData` from `app.charts.base`.
3. Define a class, e.g. `MyChartAxisRenderer(BaseAxisRenderer)`, setting `Name`, `Category`, `Description`, `RequiredRoles`/`OptionalRoles`, and optionally `Kwargs`/`Options`.
4. Implement `render_axis` using ordinary Matplotlib calls on `ax`.
5. Restart the application (or reopen the chart-creation window) — the new type appears in the picker with no further wiring.

`app/charts/text.py` is a good, compact, complete example to read or copy as a starting point: a handful of roles, a short `Kwargs` block, and a straightforward `render_axis`.

Writing a custom data operation

A data operation lives in `app/data_operations/`, as a class that subclasses `DataOperationDialogBase` (from `app.data_operations.dialog_base`) **directly** — discovered the same way, by `app/scanners/data_operation_scanner.py`.

The base class supplies the whole dialog shell (series picker, parameter form, preview pane, action buttons) and the plumbing that turns a result into a new series or table on the chart. A subclass sets a few class attributes —

`Name`	Display name shown in the Data operations list.
`Description`	One-line summary shown next to the name.
`Icon`	An inline SVG source string for the operation's icon.

— and overrides the hooks the base class calls at the right moments. The ones every operation implements are:

`build_parameter_selector()`	Build the parameter form. Simple operations describe their parameters declaratively with the helpers in

	`app.data_operations.parameter_spec` (`FloatParam`, `IntParam`, `ChoiceParam`, …) rather than laying out widgets by hand.
`prepare_job()` / `finish_job()`	Read the widgets into a **job** whose `run()` does the calculation without touching the window, and turn its outcome into results. With these two, **Preview** and **OK** work unchanged, and `evaluate(..., background=True)` runs the job on a worker thread with the shared **Stop** button — see the developer guide. An operation that does not need that can override `compute_results()` instead.
`result_series_spec(s)()`	The series metadata (name, roles, style) the base class writes to the chart. The DataFrame comes from the result's own `to_df()`, and the table name from the `RESULT_TABLE_PREFIX` class attribute, unless you override them.
`format_results()`	Render the results as the text/HTML shown in the preview pane.
`refresh_results()`	Recompute and redraw the preview after a parameter changes.

Practical steps:

1. Create `app/data_operations/my_operation_dialog.py`.
2. Subclass `DataOperationDialogBase`, set `Name`, `Description`, `Icon`.
3. Declare parameters (e.g. with `FloatParam`/`IntParam`/`ChoiceParam`) and implement `build_parameter_selector`.
4. Implement `prepare_job`/`finish_job` (or `compute_results`), `result_series_spec`, and `format_results`. Put the arithmetic in a Qt-free module under `app/analysis/`, so it can be tested on its own.
5. Restart the application — the operation appears in the Data operations list, under **Other** until its name is added to a section.

`app/data_operations/function_dialog.py` is the smallest complete dialog and a reasonable starting template; `app/data_operations/calculus_dialog.py` is a good second read for an operation that consumes an existing series rather than generating one from scratch.

Equivalents in other programs

Coming from JMP, SPSS, Minitab, Origin or Excel? Find the task you know in the left column: the second says where it is in ChartLibre, the others where it is in each program. A dash means the program has no direct equivalent. The search (the **Search** frame of the Analysis panel, or *Ctrl+K* - *Cmd+K* on macOS) knows these names too: typing “Fit Y by X”, “Crosstabs” or “Control Chart Builder” finds the ChartLibre function.

> Menu paths are those of JMP 18, SPSS 29, Minitab 22, Origin 2024 and Excel 365 with the Analysis ToolPak. They are indicative: versions move things, and an equivalent does not always compute exactly the same statistics or offer the same options.

Task	ChartLibre	JMP	SPSS	Minitab	Origin / Excel
Descriptive statistics	Data operations > Statistics > Descriptive	Analyze > Distribution	Analyze > Descriptive Statistics > Descriptives / Explore	Stat > Basic Statistics > Display Descriptive Statistics	Origin: Statistics > Descriptive Statistics; Excel: Data Analysis > Descriptive Statistics
Distribution: quantiles, moments, normality	Analysis > Distribution	Analyze > Distribution	Analyze > Descriptive Statistics > Explore / Frequencies	Stat > Basic Statistics > Graphical Summary	Origin: Statistics > Descriptive Statistics > Statistics on Columns
Process capability (Cp, Cpk, Pp, Ppk)	Analysis > Distribution (spec limits)	Distribution > Process Capability; Analyze > Quality and Process > Process Capability	Analyze > Quality Control > Control Charts (Process Capability statistics)	Stat > Quality Tools > Capability Analysis > Normal	Origin: Statistics > Quality > Process Capability Analysis
Normality test	Data operations > Statistics > Normality / shape	Distribution > Continuous Fit > Normal (Goodness of Fit)	Analyze > Descriptive Statistics > Explore > Normality plots with tests	Stat > Basic Statistics > Normality Test	Origin: Statistics > Descriptive Statistics > Normality Test
One-sample t-test	Data operations > Statistics > One-sample tests	Distribution > Test Mean	Analyze > Compare Means > One-Sample T Test	Stat > Basic Statistics > 1-Sample t	Origin: Statistics > Hypothesis Testing > One-Sample t-Test
Two-sample and paired tests	Data operations > Statistics > Independent / Paired-sample tests	Analyze > Fit Y by X > Oneway > t Test; Analyze > Specialized Modeling > Matched Pairs	Analyze > Compare Means > Independent-Samples / Paired-Samples T Test	Stat > Basic Statistics > 2-Sample t / Paired t	Origin: Statistics > Hypothesis Testing > Two-Sample t-Test; Excel: Data Analysis > t-Test
One-way ANOVA	Data operations > Statistics > Group comparison (ANOVA)	Analyze > Fit Y by X > Oneway > Means/Anova	Analyze > Compare Means > One-Way ANOVA	Stat > ANOVA > One-Way	Origin: Statistics > ANOVA > One-Way ANOVA; Excel: Data Analysis > Anova: Single Factor
Y by X: the analysis chosen by the types	Analysis > Fit Y by X	Analyze > Fit Y by X	Analyze > Compare Means / Correlate / Crosstabs / Regression (one per type)	Stat > ANOVA > One-Way; Stat > Regression > Fitted Line Plot; Stat > Tables	—
Pairwise comparisons after ANOVA	Analysis > Fit Y by X (Oneway)	Fit Y by X > Oneway > Compare Means (All Pairs, Tukey HSD); Nonparametric > Steel-Dwass	Analyze > Compare Means > One-Way ANOVA > Post Hoc (Tukey, Games-Howell)	Stat > ANOVA > One-Way > Comparisons (Tukey, Games-Howell)	Origin: Statistics > ANOVA > One-Way ANOVA > Means Comparison

Task	ChartLibre	JMP	SPSS	Minitab	Origin / Excel
Two-way ANOVA	Data operations > Statistics > Two-way ANOVA	Analyze > Fit Model (two factors and their interaction)	Analyze > General Linear Model > Univariate	Stat > ANOVA > General Linear Model	Origin: Statistics > ANOVA > Two-Way ANOVA; Excel: Data Analysis > Anova: Two-Factor
Correlation	Data operations > Statistics > Correlation / association	Analyze > Multivariate Methods > Multivariate	Analyze > Correlate > Bivariate	Stat > Basic Statistics > Correlation	Origin: Statistics > Descriptive Statistics > Correlation Coefficient; Excel: Data Analysis > Correlation
Contingency table, chi-squared	Data operations > Statistics > Contingency table	Analyze > Fit Y by X > Contingency	Analyze > Descriptive Statistics > Crosstabs	Stat > Tables > Cross Tabulation and Chi-Square	Origin: Statistics > Descriptive Statistics > Cross Tabulation
Distribution fit	Data operations > Statistics > Distribution fit	Distribution > Continuous Fit	Analyze > Nonparametric Tests > One Sample (Kolmogorov-Smirnov)	Stat > Quality Tools > Individual Distribution Identification	Origin: Statistics > Descriptive Statistics > Distribution Fit
Linear model, ANOVA, response surface	Analysis > Fit Model	Analyze > Fit Model (Standard Least Squares)	Analyze > General Linear Model > Univariate; Analyze > Regression > Linear	Stat > Regression > Fit Regression Model; Stat > DOE > Analyze Response Surface Design	Origin: Statistics > Fitting > Multiple Linear Regression; Excel: Data Analysis > Regression
Stepwise regression	Analysis > Fit Model > Stepwise	Analyze > Fit Model (Stepwise)	Analyze > Regression > Linear (Method: Stepwise)	Stat > Regression > Fit Regression Model > Stepwise	—
Logistic regression	Analysis > Fit Model > Nominal / Ordinal Logistic	Analyze > Fit Model (Nominal / Ordinal Logistic); Fit Y by X > Logistic	Analyze > Regression > Binary Logistic / Multinomial Logistic / Ordinal	Stat > Regression > Binary / Ordinal / Nominal Logistic Regression	Origin: Statistics > Fitting > Logistic Regression
Generalized linear model	Analysis > Fit Model > Generalized Linear Model	Analyze > Fit Model (Generalized Linear Model)	Analyze > Generalized Linear Models > Generalized Linear Models	Stat > Regression > Poisson Regression	—
Decision tree	Analysis > Partition	Analyze > Predictive Modeling > Partition	Analyze > Classify > Tree	Stat > Predictive Analytics > CART Classification / Regression	—
Curve fitting, nonlinear regression	Data operations > Fit	Analyze > Fit Y by X > Bivariate > Fit Polynomial; Analyze > Specialized Modeling > Nonlinear	Analyze > Regression > Curve Estimation / Nonlinear	Stat > Regression > Fitted Line Plot / Nonlinear Regression	Origin: Analysis > Fitting > Nonlinear Curve Fit; Excel: chart Trendline
Robust and machine-learning regression	Data operations > Regression	Analyze > Predictive Modeling > Bootstrap Forest / Boosted Tree	—	Stat > Predictive Analytics > Random Forests / TreeNet	—
Gaussian process	Data operations > GP Regression	Analyze > Specialized Modeling > Gaussian Process	—	—	—
Outliers	Data operations > Outliers	Analyze > Screening > Explore Outliers	Analyze > Descriptive Statistics > Explore (Outliers)	Stat > Basic Statistics > Outlier Test	Origin: Statistics > Descriptive Statistics > Grubbs Test
Cluster analysis	Data operations > Clustering	Analyze > Clustering > K Means Cluster / Hierarchical Cluster	Analyze > Classify > K-Means Cluster / Hierarchical Cluster	Stat > Multivariate > Cluster K-Means / Cluster Observations	Origin: Statistics > Multivariate Analysis > K-Means Cluster Analysis

Task	ChartLibre	JMP	SPSS	Minitab	Origin / Excel
Principal components	Data operations > Decomposition	Analyze > Multivariate Methods > Principal Components	Analyze > Dimension Reduction > Factor	Stat > Multivariate > Principal Components	Origin: Statistics > Multivariate Analysis > Principal Component Analysis
Control charts	Data operations > Control Chart	Analyze > Quality and Process > Control Chart Builder	Analyze > Quality Control > Control Charts	Stat > Control Charts	Origin: Plot > Specialized > QC (X bar R) Chart
Pareto chart	Chart: Pareto Chart	Analyze > Quality and Process > Pareto Plot	Analyze > Quality Control > Pareto Charts	Stat > Quality Tools > Pareto Chart	Excel: Insert > Statistic Chart > Pareto
Cause-and-effect diagram	Chart: Fishbone Diagram	Analyze > Quality and Process > Diagram	—	Stat > Quality Tools > Cause-and-Effect	—
Survival analysis	Data operations > Statistics > Survival; Chart: Kaplan-Meier	Analyze > Reliability and Survival > Survival	Analyze > Survival > Kaplan-Meier	Stat > Reliability/Survival > Distribution Analysis (Right Censoring) > Nonparametric	Origin: Statistics > Survival Analysis > Kaplan-Meier Estimator
Smoothing, moving average	Data operations > Smoothing	Analyze > Fit Y by X > Bivariate > Flexible > Kernel Smoother / Fit Spline	Transform > Create Time Series (moving averages, smoothing)	Stat > Time Series > Moving Average / Single Exp Smoothing	Origin: Analysis > Signal Processing > Smooth; Excel: Data Analysis > Moving Average
Spectrum, FFT	Data operations > Spectral Analysis	Analyze > Specialized Modeling > Time Series > Spectral Density	Analyze > Forecasting > Spectral Analysis	—	Origin: Analysis > Signal Processing > FFT; Excel: Data Analysis > Fourier Analysis
Digital filters, detrending	Data operations > Filtering	—	—	—	Origin: Analysis > Signal Processing > FFT Filters / IIR Filter
Peaks	Data operations > Peaks	—	—	—	Origin: Analysis > Peaks and Baseline > Peak Analyzer
Baseline subtraction	Data operations > Baseline Correction	—	—	—	Origin: Analysis > Peaks and Baseline > Peak Analyzer (Subtract Baseline)
Derivative, integral	Data operations > Calculus	—	—	—	Origin: Analysis > Mathematics > Differentiate / Integrate
Interpolation, missing values	Data operations > Interpolation	—	Transform > Replace Missing Values	—	Origin: Analysis > Mathematics > Interpolate/Extrapolate
Transformations (log, Box-Cox, standardize)	Data operations > Transform	Cols > New Formula Column > Transform	Transform > Compute Variable	Calc > Standardize; Stat > Control Charts > Box-Cox Transformation	Origin: Analysis > Mathematics > Normalize
Design of experiments	Tables > Create a table from a DOE	DOE > Custom Design / Classical	Data > Orthogonal Design > Generate	Stat > DOE > Create Factorial / Response Surface Design	—
Summary table (group and aggregate)	Edit table > Group and aggregate	Tables > Summary	Data > Aggregate	Stat > Tables > Descriptive Statistics	Excel: Insert > PivotTable
Charts	Data operations > Plot	Graph > Graph Builder	Graphs > Chart Builder	Graph	Origin: Plot; Excel: Insert > Charts